CEN

WORKSHOP

AGREEMENT

CWA 14890-1

March 2004

ICS 35.240.15

English version

Application Interface for smart cards used as Secure Signature Creation Devices - Part 1: Basic requirements

This CEN Workshop Agreement has been drafted and approved by a Workshop of representatives of interested parties, the constitution of which is indicated in the foreword of this Workshop Agreement.

The formal process followed by the Workshop in the development of this Workshop Agreement has been endorsed by the National Members of CEN but neither the National Members of CEN nor the CEN Management Centre can be held accountable for the technical content of this CEN Workshop Agreement or possible conflicts with standards or legislation.

This CEN Workshop Agreement can in no way be held as being an official standard developed by CEN and its Members.

This CEN Workshop Agreement is publicly available as a reference document from the CEN Members National Standard Bodies.

CEN members are the national standards bodies of Austria, Belgium, Cyprus, Czech Republic, Denmark, Estonia, Finland, France, Germany, Greece, Hungary, Iceland, Ireland, Italy, Latvia, Lithuania, Luxembourg, Malta, Netherlands, Norway, Poland, Portugal, Slovakia, Slovenia, Spain, Sweden, Switzerland and United Kingdom.



EUROPEAN COMMITTEE FOR STANDARDIZATION
COMITÉ EUROPÉEN DE NORMALISATION
EUROPÄISCHES KOMITEE FÜR NORMUNG

Management Centre: rue de Stassart, 36 B-1050 Brussels

© 2004 CEN All rights of exploitation in any form and by any means reserved worldwide for CEN national Members.

Ref. No.:CWA 14890-1:2004 D/E/F

Table of Contents

1 Scope 7

2 References 8

3 Abbreviations and notation 11
3.1 Abbreviations. 11
3.2 Notation and coding conventions 12

4 Definitions 14

5 Selection of ESIGN-application 16
5.1 Trusted environment versus untrusted environment 16
5.2 Application Flow 17
5.3 Selection of ESIGN application 19
5.4 Selection of cryptographic information application 19
5.5 Concurrent usage of signature applications 20
5.6 Security environment selection 20
5.7 Key selection 21
5.8 Basic Security Services 21

6 User verification 22
6.1 Knowledge based user verification 22
6.2 Biometric user verification 25

7 Digital Signature Service 27
7.1 Signature generation algorithms 27
7.2 Activation of digital signature service 27
7.3 General Aspects 28
7.4 Signature Generation 29
7.5 Selection of different keys, algorithms and input formats 30
7.6 Read certificates and certificate related information 32

8 Device authentication 34
8.1 Certification authorities and certificates 34
8.2 Authentication environments 37
8.3 Key transport and key agreement mechanisms 38
8.4 Key transport protocol 39
8.5 Device authentication with privacy protection 48
8.6 Authentication summary 58
8.7 Symmetric authentication scheme 59
8.8 Compute session keys from key seed $K_{IFD/ICC}$ 62
8.9 Compute send sequence counter SSC 63
8.10 Post-authentication phase 63
8.11 Reading the Display Message. 63
8.12 Updating the Display Message 65

9 Secure Messaging 67
9.1 CLA byte 67
9.2 TLV coding of command and response message 67
9.3 Treatment of SM-Errors 68
9.4 Padding for checksum calculation 68
9.5 Message structure of Secure Messaging APDUs 68

9.6 Send sequence counter (SSC) . . . 71
9.7 Use of TDES . . . 71

10 Key Generation . . . 74
10.1 Key generation and export using SK.ICC.AUT . . . 74
10.2 Key generation and export with dynamic SM . . . 76
10.3 Key parameter import . . . 77
10.4 Public Key Export . . . 80
10.5 Write certificate . . . 84
10.6 Setting keys in static secure messaging . . . 84

11 Key identifiers and parameters . . . 86
11.1 Key identifiers (KID) . . . 86
11.2 Public Key parameters . . . 86
11.3 DSA with ECC public key parameters . . . 87
11.4 RSA Diffie-Hellman key exchange parameters . . . 90

12 APDU data structures . . . 91
12.1 CRTs . . . 91
12.2 Key transport device authentication protocol . . . 94
12.3 Privacy device authentication protocol . . . 96

13 AlgIDs, Hash- and DSI Formats . . . 98
13.1 Algorithm Identifiers and OIDs . . . 98
13.2 Hash Input-Formats . . . 99
13.3 Formats of the Digital Signature Input (DSI) . . . 101

14 CV_Certificates and Key Management . . . 103
14.1 Level of trust in a certificate . . . 103
14.2 CV Certificates in the ICC . . . 103
14.3 The role of the CA . . . 104
14.4 CV_Certificates and key management . . . 104
14.5 Signature-Certificates . . . 106
14.6 Authentication Certificates . . . 106
14.7 Certificate Data Elements . . . 106
14.8 C_CV.IFD.AUT . . . 113
14.9 C_CV.CA.CS-AUT . . . 114
14.10 C.ICC.AUT . . . 115

15 Files . . . 116
15.1 File structure . . . 116
15.2 FileIDs . . . 117
15.3 EF.DIR . . . 117
15.4 EF.SN.ICC . . . 117
15.5 EF.DH . . . 118
15.6 EF.C.ICC.AUT . . . 119
15.7 EF.C.CA_{ICC}.CS-AUT . . . 119
15.8 EF.C_X509.CH.DS . . . 120
15.9 EF.C_X509.CA.CS (DF.ESIGN) . . . 121
15.10 EF.DM . . . 121

16 Cryptographic Information Application . . . 122
16.1 ESIGN cryptographic information layout example . . . 124

Annex A. Dev. auth. - cryptographic view . . . 131

A.1 Algorithms for authentication with key exchange or key negotation 131
A.2 Device authentication with key transport 131
A.3 Device authentication with key negotiation 136
A.4 Device authentication with privacy protection 139
A.5 The “Grandmaster Chess Attack” 144

Annex B. Personalisation scenarios 145

Index 147

Foreword

This document is part of a series of standards for secure signature creation devices (SSCDs), developed to support the EU-directive on electronic signatures. It is dedicated to smart cards as an important representation of SSCDs (e.g. SIMs, USB tokens). The key issue of this document is to enable interoperability, so that smart cards from different manufacturers can interact with different kind of signature creation applications. The specification is applicable to smart cards supporting file system oriented applications as well as for smart cards supporting object oriented applications (e.g. Java applets).

Requirements for SSCDs expressed in relevant EU standards and CWAs (e.g. CWA related to protection profile for SSCDs) are taken into account. In particular they are:

- *Requirement 1: The format for electronic signatures and their certificates shall be interoperable*

  Signatures will be verified in different applications and environments, unknown to the signer. Formats of signatures and certificates therefore need to be standardized in order to ensure interoperability.

- *Requirement 2: The device interface (physical, logical and application interface) shall be interoperable at least for the same device type (smart card, USB-device etc).*

  A signer should be able to use his signing device in different applications and environments, without having to install specific software drivers depending on the manufacturer of the device. An analogy is the present standardization of SIM cards, which today are “plug-compatible” where any SIM card can be inserted in any mobile phone.

This document describes the European standardization activities and solutions for smart cards as a specific type of a **S**ecure **S**ignature **C**reation **D**evice (SSCD).

However, the focus, solely on European activities, is not as restrictive as it seems at first view because national non-European projects are likely to take this work into account. Smart cards are selected as representatitives for SSCDs because of the following reasons:

- smart cards are computers in their own and under sole control of the signatory with the size of a business card or even smaller in case of the SIM cards for mobile phones.
- There are already successfully evaluated products on the market that are approved as high security products.
- smart cards can be used as representatives for SSCD’s by fulfilling the requirements from existing CWAs (see (F[6] , G1[4] , G2[5], EU Directive[3]).
- smart cards do already use international and national standards on physical, logical and application layers respectively, which can be build as a base for the elaboration of an European standard (see the ISO/IEC 7816 series, esp. ISO/IEC 7816-4,8,-15) according to the EU directive [3].

This standard consists of two parts

- Part1 - “Basic Requirements” describes the mandatory specifications for an SSCD to be used in compliance with the ESIGN-G1 and F specifications.
- Part 2 - “Optional Features” describes additional features relevant for use with SSCDs.

The error handling of commands is out of scope of this document.

1 Scope

This document specifies the application interface to SmartCards during the usage phase, used as Secure Signature Creation Devices (SSCD) to enable interoperability and usage of those cards on a national or European level.

The specification is based on the EU directive on electronic signatures [3] and takes into account E-SIGN documents and standards mentioned in the scope to be respected.

The functionalities described in this document map the general requirements of the EU directive to asymmetric techniques as required by the corresponding protection profile [6] and cover additional services, useful in signature environments.

The specification is separated into two parts.

Part 1 describes the mandatory services for the usage of Smart Cards as SSCDs. This covers the signing function, storage of certificates, the related user verification, establishment and use of trusted path and channel, key generation and the allocation and format of resources required for the execution of those functions and related cryptographic token information [17].

Part 2 describes optional services based on the same technology as available in signature devices. This covers key decipherment and client(card holder) server authentication, signature verification and related cryptographic token information [17].

2 References

The following standards contain provisions which, through reference in this text, constitute provisions of this part of ESIGN specification. At the time of publication, the editions indicated were valid.

[1] CWA 14890-1:2004, Application Interface for smart cards used as Secure Signature Creation Devices - Part 1 - Basic requirements

[2] CWA 14890-2:2004; Application Interface for smart cards used as Secure Signature Creation Devices - Part 2 - Optional Features

[3] EU Directive 1999/93/EC of the European Parliament and the council of 13 December 1999 on a Community framework for electronic signatures

[4] CWA 14170:2004, *Security Requirements for Signature Creation Applications*

[5] CWA 14171:2004, *General Guidelines for electronic signature verification*

[6] CWA 14172-1 to 8: 2004, *EESSI Conformity Assessment Guidance*

[7] ETSI SR 002 176: Electronic Signatures and Infrastructures (ESI); Algorithms and Parameters for Secure Electronic Signatures

[8] ISO 3166 : 1993, Codes for the representation of names of countries.

[9] ISO/IEC 7812-1 : 1993, Identification cards - Issuer identification - Part 1 : Numbering system.

[10] ISO/IEC 7816-3 CD Draft: 2003, Identification cards - Integrated circuit(s) cards with contacts - Part 3 : Electronic signals and transmission protocols.

[11] ISO/IEC FCD 7816-4: 2003 (Draft) Identification cards - Integrated circuit(s) cards with contacts, Part 4: Interindustry commands for interchange, Working draft dated 2003-09-16, ISO SC17 Document WG4N1750

[12] ISO/IEC FDIS 7816-5 : 2003, Identification cards - Integrated circuit(s) cards with contacts - Part 5 : Registration for application providers, FDIS dated 2003-09-15, ISO SC17 Document WG4N1751

[13] ISO/IEC FDIS 7816-6 : 2003, Identification cards - Integrated circuit(s) cards with contacts - Part 6 : Interindustry data elements for interchange - FDIS dated 2003-09-16, ISO SC17 Document WG4N1752.

[14] ISO/IEC FDIS 7816-8: 2003 , Integrated circuit(s) cards with contacts, Part 8: Commands for security operations. FDIS dated 2003-09-16, ISO SC17 Document WG4N1753.

[15] ISO/IEC FDIS 7816-9: 2003 (Draft), Integrated circuit(s) cards with contacts, Part 9: Interindustry commands for card and file management. FDIS dated 2003-09-16, ISO SC17 Document WG4N1754.

[16] ISO/IEC FDIS 7816-11: 2003(E), Integrated circuit(s) cards with contacts, Part 11: Personal verification through biometric methods. FDIS dated 2003-07-31, ISO SC17 Document WG4N1749

[17] ISO/IEC FDIS 7816-15: 2003 (Draft), Integrated circuit(s) cards with contacts, Part 15: Cryptographic Information Application, FDIS dated 2003-02-12

[18] ISO/IEC 8825-1: 1998, Information technology - ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER).

[19] ISO 9564-1:2002, Banking - Personal Identification Number (PIN) management and security - Part 1: Basic principles and requirements for online PIN handling in ATM and POS systems.

[20] ISO/IEC 9796-2:2002, Information technology - Security techniques -- Digital signature schemes giving message recovery - Part 2: Integer factorization based mechanisms (revision)

[21] ISO/IEC 9797-1 : 1999, Information technology - Security techniques - Message Authentication Codes (MACs) - Part 1: Mechanisms using a block cipher”

[22] ISO/IEC 9798 : 1998, Information technology - Security techniques - Entity authentication - Part 3: Mechanisms using digital signature techniques

[23] ISO/IEC 9979 : 1998, Information technology - Security techniques - Procedures for the registration of cryptographic algorithms.

[24] ISO/IEC 10116 : 1991, Information technology - Information technology - Security techniques - Modes of operation of an n-bit block cipher algorithm.

[25] ISO/IEC 10118-1 : 2000, Information technology - Security techniques - Hash functions - Part 1 : General.

[26] ISO/IEC 10118-2 : 2001, Information technology - Security techniques - Hash functions - Part 2 : Hash functions using an n-bit block cipher algorithm.

[27] ISO/IEC 11770-3 : 1999, Information technology — Security techniques — Key management — Part 3, Mechanisms using asymmetric techniques

[28] ISO/IEC 15408-1, Edition:1999-12, Information technology - Security techniques - Evaluation criteria for IT security - Part 1: Introduction and general model

[29] ISO/IEC 15408-2, Edition:1999-12, Information technology - Security techniques - Evaluation criteria for IT security - Part 2: Security functional requirements

[30] ISO/IEC 15408-3, Edition:1999-12, Information technology - Security techniques - Evaluation criteria for IT security - Part 3: Security assurance requirements

[31] ISO/IEC 19794-2: Biometrics – Biometric Data Interchange Formats – Part 2: Finger Minutiae (Working Draft)

[32] Wireless Identity Module Part: Security Version 12-July-2001, Wireless Application Protocol WAP-260-WIM-20010712-a, www.wapforum.org

[33] PKCS #1 v2.1: RSA Cryptography Standard, RSA Laboratories, June, 14, 2002, / www.rsasecurity.com/rsalabs/pkcs/pkcs-1/

[34] “New Directions in Cryptography” by Diffie, W. and M. E. Hellman, IEEE Transactions on Information Theory, Volume IT-22 Number 6, November 1976, pages 644-654.

[35] ISO/IEC 14888-1 : JTC1.28.08.01 “Digital signatures with appendix”

[36] ISO/IEC 14888-3 : JTC1.28.08.03 “Certificate-based mechanisms”

[37] EMV2000, Integrated Circuit Card Specification for Payment Systems Book 3, "Application Specification Version 4.0" December, 2000

[38] EuroCrypt 2002, paper by J.H. An, Y. Dodis and T. Rabin, "On the security of joint signature and encryption"

[39] “The order of encryption and authentication for protecting communications (or: How secure is SSL?)” by Hugo Krawczyk. In Advances in Cryptology - CRYPTO 2001, volume 2139 of Lecture Notes in Computer Science, pages 310-331. Springer-Verlag, 2001.

[40] DIN V66291-1, Chipcards with digital signature application/function according to SigG and SigV - Part 1: Application Interface, 15 December 1998, DIN Deutsches Institut für Normung e.V.: Berlin.

[41] DIN V66291-4, Chipcards with digital signature application/function according to SigG and SigV - Part 4: Basic Security Services, 17 October 2000, DIN Deutsches Institut für Normung e.V.: Berlin.

[42] “The Internet Key Exchange (IKE)” by Harkins, D. and D. Carrel, RFC2409, November 1998. URL: ftp://ftp.rfc-editor.org/in-notes/rfc2409.txt

[43] “Security Analysis of IKE's Signature-Based Key-Exchange Protocol. in Advances in Cryptology” by Canetti, R. and H. Krawczyk, Crypto 2002. 18-22 August 2002, Santa Barbara, CA:Lecture Notes in Computer Science Vol. 2045. Springer-Verlag. p. 143-161.

[44] "The Elliptic Curve Digital Signature Algorithm (ECDSA)" by D. Johnson, A. Menezes and S. Vanstone, International Journal of Information Security, 1 (1), pp. 36-63, 2001.

[45] IEEE P1363, "IEEE Standard Specifications for Public-Key Cryptography", 2000.

[46] FIPS PUB 198, “The Keyed-Hash Message Authentication Code (HMAC), 6 March 2002, *csrc.nist.gov/publications/fips/fips198/fips-198a.pdf*

[47] “Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS#1” by D. Bleichenbacher. Advances in cryptography - Crypto 1998

[48] ANSI X9.63, “Public Key Cryptography For The Financial Services Industy: Key Agreement and Key Transport Using Elliptic Curve Cryptography”,

3 Abbreviations and notation

The abbreviations and notation is in accordance with those given in ESIGN-F [6].

3.1 Abbreviations

AID Application Identifier
APDU Application protocol data unit
AKI Authority Key Identifier
AKP Asymmetric key pair
AM Access Mode
AT CRT for Authentication
AUT Authentication
BCD Binary Coded Digit
BER Basic Encoding Rules
C Certificate
CA Certification Authority
CAR CA Reference
CC Cryptographic Checksum
CCT CRT for cryptographic checksum
CG Cryptogram
CH Cardholder
CHA Certificate Holder Authorisation
CHN Cardholder Name
CHR Certificate Holder Reference
CIA Cryptographic Information Application
CIO Cryptographic Information Object
CLA Class byte
CRT Control Reference Template
CPI Certificate Profile Identifier
CS CertSign
CT CRT for confidentiality
CV Card Verifiable
CWA CEN Workshop Agreement
C/S Client/Server
D[key](msg) Decipherment of <msg> with <key>
DE Data Element
DES Data Encryption Standard
DF Dedicated File
DH Diffie-Hellman
DIR Directory
DO Data Object
DS Digital Signature
DSA Digital Signature Algorithm
DSI Digital Signature Input
DST CRT for Digital Signature
E[key](msg) Encipherment of <msg> with <key>
EF Elementary File
ECC Elliptic Curve Cryptography
FCI File Control Information
FID File Identifier
h(msg) hash value of <msg>
HB Historical Bytes
HT CRT for hash code
ICC Integrated Circuit(s) Card
ICV Initial Chaining Value
ID Identifier.
IFD Interface Device
INS Instruction byte
KE Key Encipherment
KID Key Identifier
MAC[key](msg) Message Authentication Code of <msg> built with <key>
MF Master File
MSE MANAGE SEC. ENVIRONMENT
NAD Node Address
OID Object Identifier
P1-P2 Parameter bytes
PK Public Key
PKI Public Key Infrastructure
PKCS Public Key Cryptography Standards
PI Padding Indicator
PIN Personal Identification Number
PRND Padding Random Number
PSO PERFORM SEC. OPERATION
PV Plain Value
RC Retry Counter
RCA RootCA
RD Reference Data
RFU Reserved for Future Use
RID Registered application provider indentifier
RND Random number
RSA Sig-Alg. of Rivest, Shamir, Adleman
SCA Signature-Creation Application
SHA Secure Hash Algorithm
SE Security Environment
SIG( ) Signature of
SK Secret Key
SN Serial Number
SM Secure Messaging
SSC Send Sequence Counter
SSCD Secure Signature Creation Device
SW1-SW2 Status bytes
TDES Triple-DES
TOE Target of evaluation
TPDU Transmission protocol data unit
TLS Transport Layer Security
TLV Tag, Length, Value
TSF TOE Security Functions
WIM Wireless Identity Module
WS Workshop

3.2 Notation and coding conventions

For the purpose of this part of ESIGN specification, the following notation and conventions apply:

Coding conventions

'0' to '9' and 'A' to 'F	The sixteen hexadecimal digits
"..."	ASCII-string
(B1)	Value of byte B1
B1 || B2	Concatenation of bytes B1 (the most significant byte) and B2 (the least significant byte)
(B1 || B2)	Value of the concatenation of bytes B1 and B2
#	Number

Notation

For keys and certificates the following simplified Backus-Naur notation is used:

<object descriptor>::= <key descriptor> | <certificate descriptor>

<key descriptor>::= <key>[<index1>].
<keyholder> [<index2>].
<usage list>

<key>::= <asymmetric key> | <symmetric key>

<asymmetric key>::= <secret key> | <public key>

<secret key>::= SK

<public key>::= PK

<index1>::= 1 | 2 | ...

<keyholder>::= <cardholder> | <device> | <organisation>

<device> ::= <integrated circuit(s) card> | <interface device> | <personalisation system> | <server> | <identity verification system>

<organisation> ::= <certification authority> | <card manufacturer>

<cardholder>::= CH

<certifcation authority>::= CA | RCA

<card manufacturer>::= CM

<integrated circuit(s) card>::= ICC

<interface device>::= IFD

<personalisation system>::= PS

<server> ::= S

<identity verification system> ::= IVS <index2> used for indication of qualification or role

<usage list>::= <usage>[<index3>] | <usage list> & <usage>[<index3>]

<usage>::= <digital signature> |
<authentication> |
<key encipherment> |
<CertSignature>

<digital signature>::= DS

<authentication>::= AUT

<key encipherment>::= KE

<CertSignature>::= CS | CS-DS | CS-AUT | CS-KE
<index3> used for indication of algorithm or components (e.g. ICC/IFD or C/S)

<certificate descriptor>::= <certificate>[<index1>].
<certholder>[<index2>].
<usage list>

<certificate>::= C | <CV certificate> |
<X.509 certificate> |
<X.509 attribute certificate>

<CV certificate> ::= C_CV

<X.509 certificate> ::= C_X509

<X.509 attrib. cert.>::= C_X509A

<certholder>::= <cardholder> | <device> | <organisation>

Note: Indexes shall only be applied to avoid ambiguouity.

Examples:

1. SK.RCA.CS = Secret key of root CA for signing certificates, whereby no binding of the certified key pair to a special usage is specified

2. SK.RCA.CS-DS = Secret key of root CA for signing certificates, whereby the certified key pair shall be used for signing DS certificates only

3. SK1.CH.DSRSA = Secret key no. 1 of the cardholder for digital signature with RSA

4. SK.CH.AUTC/S = Secret key of the cardholder for client/server authentication

5. PK.CA.CS-AUTC/S = Public key of CA for verification of certificates related to client/server authentication

6. PK.CA.CS-AUTICC/IFD = Public key of CA for verification of certificates related to ICC/IFD authentication

7. SK.CA.CS-AUT&AUT = Secret key of CA for signing of authentication certificates and for use in authentication procedures

4 Definitions

For the purpose of the ESIGN specification, the following definitions apply. These definitions are in compliance with those given in the revision of ISO/IEC 7816-4.

4.1 Answer-to-Reset file : Elementary file which indicates operating characteristics of the card.

4.2 command-response pair : Set of two messages : a command followed by a response

4.3 data unit : The smallest set of bits which can be unambiguously referenced.

4.4 data element : Item of information seen at the interface for which are defined a name, a description of logical content, a format and a coding.

4.5 data object : Infyormation seen at the interface which consists of a tag, a length and a value (i.e., a data element). In this specification, data objects are referred to as BER-TLV, COMPACT-TLV and SIMPLE-TLV data objects. Refer to ISO/IEC ISO/IEC 7816-4 [11].

4.6 dedicated file : File containing file control information and, optionally, memory available for allocation. It may be the parent of EFs and/or DFs.

4.7 DF name : String of bytes which uniquely identifies a dedicated file in the card.

4.8 directory file : Elementary file defined in part 4 of ISO/IEC 7816.

4.9 elementary file : Set of data units or records which share the same file identifier. It cannot be the parent of another file.

4.10 file control parameters : Logical, structural and security attributes of a file.

4.11 file identifier : A 2-bytes binary value used to address a file.

4.12 file management data : Any information about a file except the file control parameters (e.g., expiration date, application label).

4.13 internal elementary file : Elementary file for storing data interpreted by the card.

4.14 master file : The mandatory unique dedicated file representing the root of the file structure.

4.15 message : String of bytes transmitted by the interface device to the card or vice-versa, excluding transmission-oriented characters as defined in part 3 of ISO/IEC 7816.

4.16 parent file : The dedicated file immediately preceding a given file within the hierarchy.

4.17 password : Data which may be required by the application to be presented to the card by its user. A password in the context of this specification, is a string of numbers and/or ASCII characters.

4.18 path : Concatenation of file identifiers without delimitation. If the path starts with the identifier of the master file, it is an absolute path.

4.19 private key Secret part of an asymmetric key pair e.g. signature creation data as specified in the EU directive for electronic signatures.

4.20 provider : Authority who has or who obtained the right to create a dedicated file in the card.

4.21 public key : public part of an asymmetric key pair.

4.22 record : String of bytes which can be handled as a whole by the card and referenced by a record number or by a record identifier.

4.23 record identifier : Value associated with a record that can be used to reference that record. Several records may have the same identifier within an elementary file.

4.24 record number : Sequential number assigned to each record which uniquely identifies the record within its elementary file.

4.25 retry counter : a counter being used to count the number of erroneous usages of a related (security) object. If the object (e.g. password entry) was used correctly (correct password entered) then the retry counter is reset to its initial value. A typical value of a retry counter is 3.

4.26 secret key : Either a symmetrical key or private part of a public key pair.

4.27 trusted environment : an operating environment that avoids the fraud with data that is seen at the communication interface by the definition of its physical location, physical security protection or physical access conditions or other measures to counterfy tampering or intrusion of data. (see 8.2.3 “Specification of the environment” on page 8-37.

4.28 trusted channel : A means by which a TSF and a remote trustet IT product can communicate with necessary confidence to support the TSP.

4.29 trusted path : A means by which a user and a TSF can communicate with necessary confidence to support the TSP.

4.30 untrusted environment : an operating environment that allows tampering and intrusion of data available at the communication interface. The decision whether an environment is trusted cannot always be made on system level and therefore might have to be taken by the card holder.

4.31 usage counter a counter updated each time a related object is used. Opposite to retry counters, usage counters are not reset to their initial value after the object was successfully used.

4.32 working elementary file : Elementary file for storing data not interpreted by the card.

5 Selection of ESIGN-application

The EU directive uses a different legal terminalogy as those specifications that describe the technical implementation of these directive. Figure 5-1 shows a comparison of terminology in order to map the mechanisms described in the EU directive.

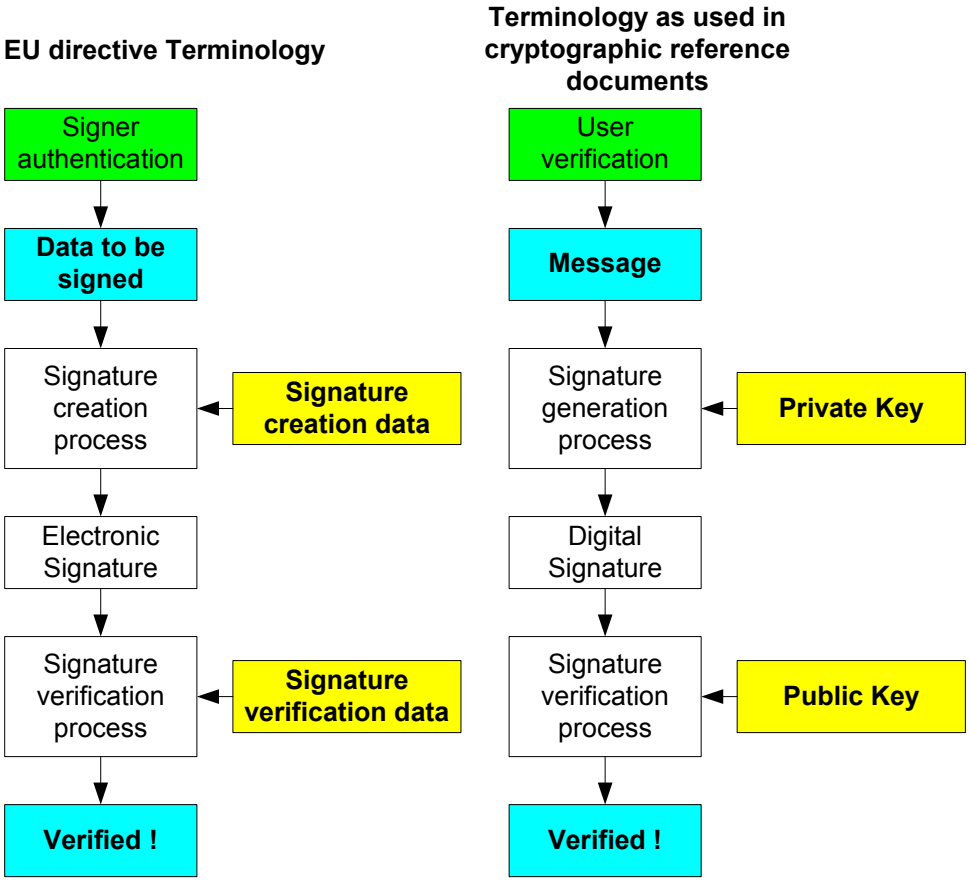


Figure 5-1. Signature process comparison of terminology

For the rest of this document the terminology as used in cryptographic reference documents (right figure) will be used.

5.1 Trusted environment versus untrusted environment

According to the definitions in “trusted environment :” on page 4-15 and “untrusted environment :” on page 4-15 this specification describes additional mechanisms in order to accomplish the required security as claimed in the protection profile [6]. The additional mechanisms according to this specification are

- Device authentication
- Secure messaging

5.2 Application Flow

The following diagrams demonstrate a typical application flow as described in [4].

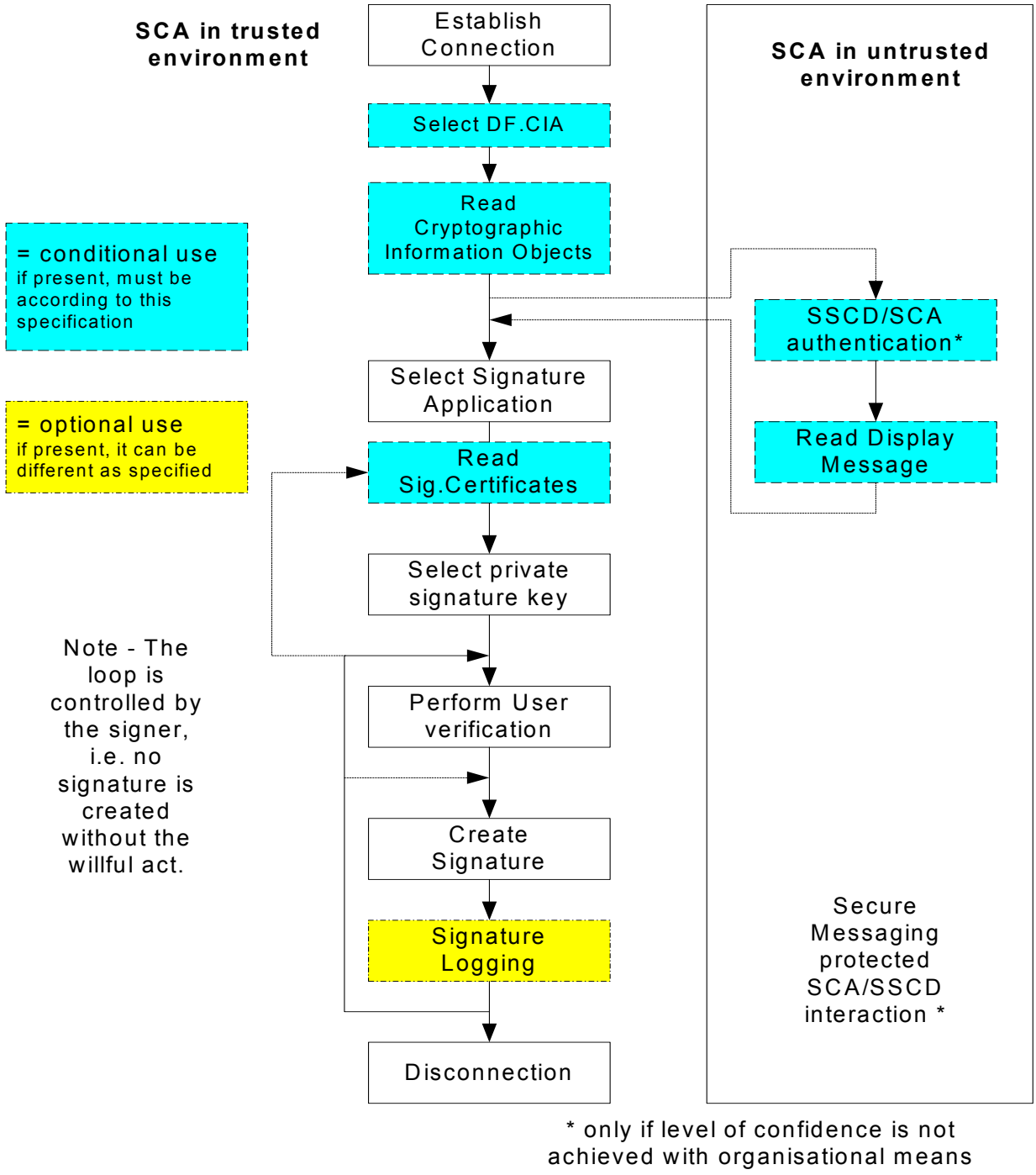


Figure 5-2. Interaction sequences between SCA and SSCD in compliance with EU directive

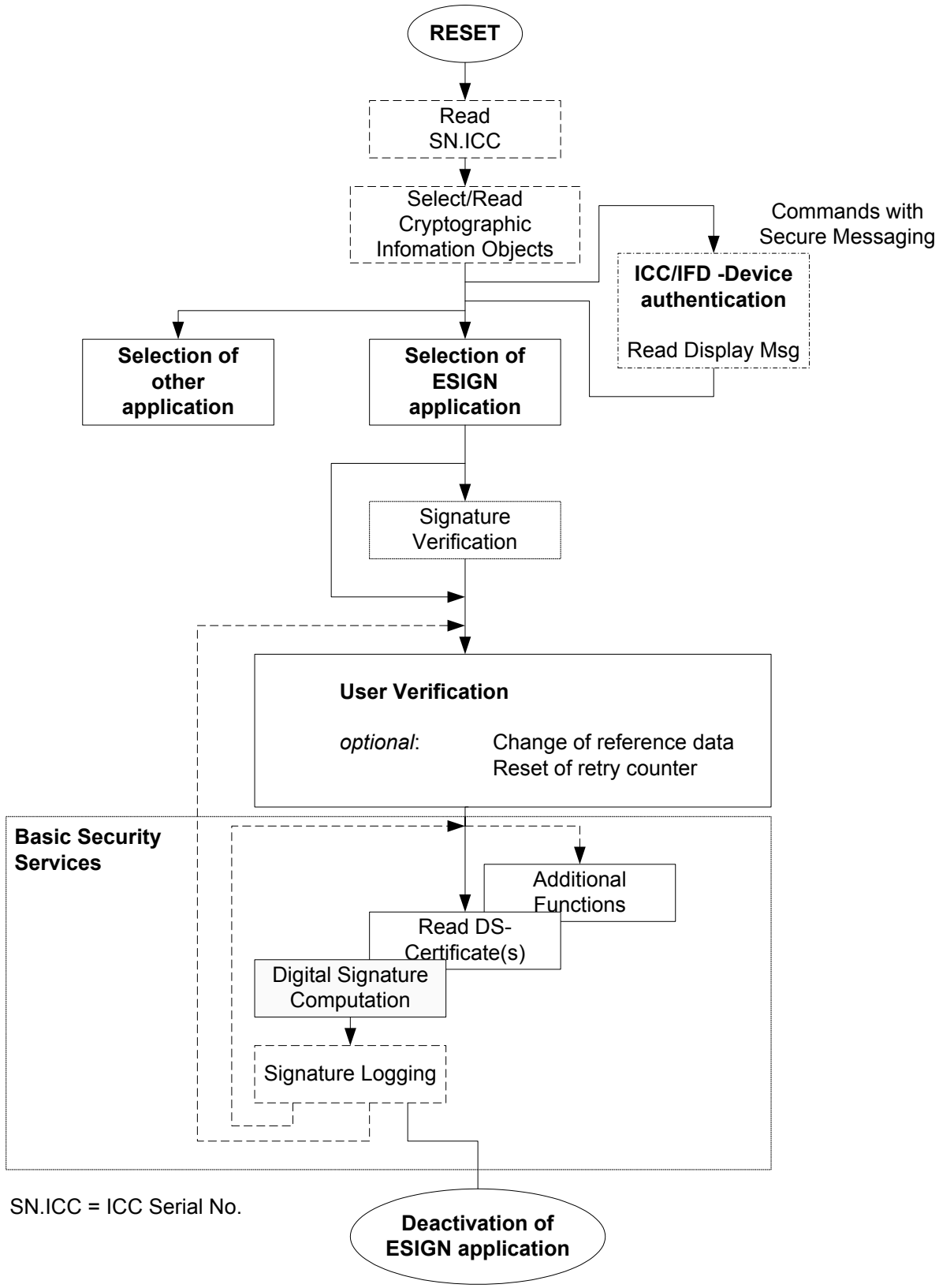


Figure 5-3. Flowchart example for ICCs with ESIGN-Application as specified in this standard

The following scenario displays the typical command flow, when an application is started

5.3 Selection of ESIGN application

An ESIGN application is selected by its Application Identifier (AID). The RID is registered by the ISO registration authority and has the following value:

A0 00 00 01 67 .

The entire AID has the following value

AID = A0 00 00 01 67 || “ESIGN” = A0 00 00 01 67 45 53 49 47 4E

with

Category = ‘A.....’ (international)

PIX = ‘ESIGN’ = 5 bytes

Execution Flow:

```
SELECT (  INS = ‘A4’             // SELECT
          P1  = ‘04’,            // select by DF Name
          P2  = ’00’ or ‘02’,    // return FCI info (see remark below)
          data = AID.ESIGN)      // application identifier
```

For the description of the SELECT command refer to ISO/IEC 7816-4, chapter 7.1.1.

Remarks:

P2 It is left to the particular application to use other specified values from ISO/IEC 7816-4, this specification does not mandate the retrieval of the file control information. JAVA cards, however, require P2 = ‘00’ or P2 = ‘02’. In this case, unless otherwise specified the FCI contains the following data objects:

data field contains the AID.ESIGN.

Response The FCI shall at least contain the DO '84' (DfName)

5.4 Selection of cryptographic information application

The cryptographic information application (CIA) belonging to the ESIGN application has in compliance with ISO/IEC 7816-15 the following AID:

AID.CIA_ESIGN = ´E8 28 BD 08 0F A0 00 00 01 67 || “ESIGN” (15 bytes)

The selection of the DF.CIA(ESIGN) is done with a DFname as shown above.

5.5 Concurrent usage of signature applications

Concurrent usage of signature applications can be realized using the mechanisms of logical channel as defined in ISO/IEC 7816-4.

5.5.1 Methods of channel selection

The mechanism for channel selection is optional.

A channel may be selected by

- implicit selection with CLA byte of SELECT command
- explicit selection with the MANAGE CHANNEL command

Both methods are described in [11] and will not be discussed in the specification.

5.5.2 Security issues on multiple channels

In order to access the resources of the ICC from another channel the ‘shareable’ bit shall be set in the FCI of the application DF.

Each invocation of an ESIGN-Application has its own security status.

Under the scope of the ESIGN application, after establishing a channel that requires device authentication it is not allowed to open another channel without device authentication.

5.6 Security environment selection

A signature card may contain more than one authentication- or signature key and one or more user reference data respectively. In this case the IFD needs to select the appropriate keys to be involved in authentication or signature operations and/or user verification (e.g.PIN, Password or biometric data) respectively. The command MANAGE SECURITY ENVIRONMENT (refer to ISO/IEC 7816-4, chapter 7.5.11.) is used in order to specify the desired keys to be involved in the next operation.

A security environment selection is applicable in the context of the following operations

- User verification (PIN, Password or biometric)
- signature creation as described in chapter 7 “Digital Signature Service” on page 7-27.
- signature verification as described in volume 2.
- Verification of card verifiable certificates as described in chapter 14 “CV_Certificates and Key Management” in volume 1 page 14-103
- Key decipherment as described in volume 2.
- Authentication services as described in volume 2.

The actual application of the MANAGE SECURITY ENVIRONMENT is discussed in the appropriate chapters.

5.7 Key selection

The selection of keys is performed by an appropriate command MSE (Manage Security Environment) if not specified otherwise (e.g. in the parameters of the command header, see 6.1.1 on page 6-22). An appropriate key ID is submitted and stored in the ICC in order to specify the key to be used with the next command(s). Key selection has to be done prior to security operation unless the key was previously selected.

A key ID is typically a number represented by one or more bytes describng a non-standardized value which is application- or operating system specific. The IFD may retrieve the link between a certificate and a corresponding KeyID from the DF.CIA application data as described in chapter 16 “Cryptographic Information Application” on page 16-122.

5.8 Basic Security Services

A basic security service, opposite to an extended security service, is a set of SSCD functions that serve the purpose of signature generation, verification, authentication or key en-/decipherment.

An extended security service comprises the installation of data and keys, card life cycle management and other functionalities that extend the scope described in the EU directive [3].

The two volumes of this document specify the following basic security services.

- **digital signature service** for the advanced electronic signatures services of the cardholder (mandatory), see chapter 7 “Digital Signature Service” on page 7-27.
- **authentication service** as required for specific authentication procedure (optional).
- **Key decipherment** service (optional)

6 User verification

User verification is part of the application flow as described in Figure 5-3 on page 5-18.

Two general methods of user verification are available

- Knowledge based user verification (e.g. password) (mandatory)
- Biometric user verification (optional)

Specific local verification data, dedicated to protect the use of the private key for signature generation shall be applied. If several private keys for signature generation are in the card, their use may optionally be protected by different verification data.

For protection of the client/server authentication function and the key decipherment function separate (global or local according to the security policy) verification data shall be used.

In either of these cases the verification data might be subject of alteration.

The following commands, as defined in ISO/IEC 7816-4 [11], are used to the manage verification process:

- VERIFY
- CHANGE REFERENCE DATA
- RESET RETRY COUNTER.

Regardless from the usage of these commands at any time it shall be ensured that the usage of the signature key is protected by a preceding user verification

6.1 Knowledge based user verification

Knowledge based user verification requires a password to be entered by the user. The actual verification is done with the VERIFY command. European applications do frequently restrict passwords to numbers only. As such they are also known as 'PIN'. Find further information under 6.1.2 .

6.1.1 Password verification

Execution Flow

```
VERIFY ( INS = '20'     // VERIFY
         P1 = '00'      //
         P2 = keyref    // password reference
         Data = verification data)
```

For the structure and coding of the VERIFY command refer to ISO/IEC 7816-4, chapter 7.5.6.).

6.1.1.1 Password reference value (KID)

The parameter P2 codes the reference value of the password.

This standard specifies the following KIDs

KID = ’8x’ Key reference to local reference data for protection of the use of a private signature key (min 6 digits or characters); the value of the KID and the purpose of the signature password shall be indicated in the cryptographic information objects.

Also other passwords may be of the type “local”

KID = ’0x’ Key reference to global reference data; the value of the KID and the purpose of the password shall be indicated in the cryptographic information objects.

Note: In order to support EMV 2000 applications only the three least significant bits should be used to differentiate keys. See “EMV2000, Integrated Circuit Card Specification for Payment Systems Book 3, "Application Specification Version 4.0" December, 2000” on page 2-10 [37].

6.1.2 Presentation formats

The data field codes the verification data in one of the following formats

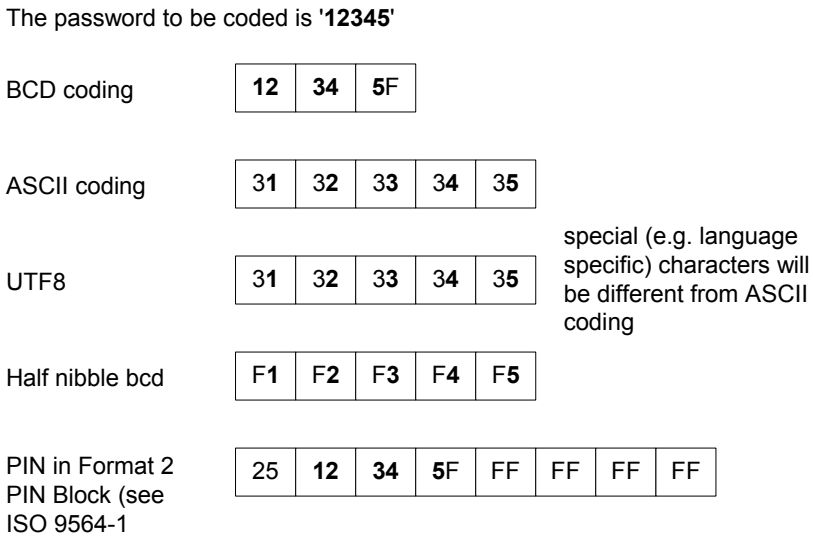


Figure 6-1. Example of coding ’12345’ in different presentation formats

Figure 6-1 demonstrates the coding when the different presentation formats are used. The actual presentation format may be retrieved from information presented in DF.CIA (see 16 “Cryptographic Information Application” on page 16-122), unless implicitly known.

Passwords protecting the usage of signature keys, shall have a minimum of 6 digits or characters.

All other passwords shall have a minimum of 5 digits or characters.

Note: The specifications are in compliance with appropriate protection profiles e.g. ESIGN-F [6].

6.1.3 Retry counters

In order to avoid exhaustive search attacks, the password shall be protected by an appropriate retry counter.

In order to avoid an exhaustive search attack on the password under a known resetting code, the resetting code should be protected by an appropriate usage counter with a usage counter limit with a low value (typically U = 3).

After successful presentation, the appropriate retry counter is automatically reset to its initial value (typically N = 3).

For the definition of the counter types refer to 4 “Definitions” on page 4-14.

6.1.4 Password Change

The CHANGE REFERENCE DATA command for changing the global password may be used at any time convenient for the cardholder.

The change of DF-specific (local) password can only be performed after selection of the appropriate DF.

Execution Flow

```
CHANGE REFERENCE DATA(  INS = ’24’       // CHG REF DATA
                       P1 ='00'          // no algorithm spec.
                       P2 ∈ [’0x’,’8x’]   // KID of referenced password
                       data = <password> || < new password >)
```

For the description of the command refer to ISO/IEC 7816-4, chapter 7.5.6..

The length of the existing Reference Data is known in the card, so that neither a delimiter nor padding for filling up fixed formats is necessary.

6.1.5 Reset of RC and setting a new password

After N (N as specified by application) subsequent false presentations of the password, the password is locked and does not allow further usage of the protected functions.

With the ISO/IEC 7816-4 command RESET RETRY COUNTER, the cardholder can initiate the reset of the RC to its initial value N. The Resetting Code shall have a minimum length of 6 digits[1].

It is also possible to define a new password with the RESET RC command.

The reset of DF-specific (local) retry counter can only be performed after selection of the appropriate DF.

Execution Flow

```
RESET RETRY COUNTER(  INS = ‘2C’          // RESET RETRY COUNTER
                     P1 ∈ [‘00’..’03’]     //
                     P2 ∈ [‘0x’,’8x’]      // KID of referenced password
                     data : see below)
```

For the coding of the RESET RETRY COUNTER command refer to ISO/IEC 7816-4, chapter 7.5.10

1 ‘digit’ can also be a character as part of a password

The content of the data field depends on the value of P1

P1 = '00' Resetting code followed without delimitation by new reference data

P1 = '01' Resetting code

The RESET RETRY COUNTER with P1=01 will not change the security status, i.e. for using the signature creation key verify command is still required.

P1 = '02' New reference data

P1 = '03' Data field absent

After successful presentation of the Resetting Code, the retry counter of the referenced password is automatically reset to its initial value (typically N = 3).

6.2 Biometric user verification

The verification of biometric data mandates a device authentication and the usage of secure messaging in order to avoid a replay attack. Hence this case will always be considered as untrusted environment as specified in 5.1 “Trusted environment versus untrusted environment” on page 5-16.

6.2.1 Retrieval of the Biometric Information Template

Prior to the verification, the SCA may need information about

- AlgID of the matching algorithm and key reference of the biometric reference data
- biometric type (e.g. fingerprint)
- biometric type instance (e.g. left pointer finger enrolled)
- format owner of the biometric data
- format type of the biometric data

This information may be found in the DF.CIA description (see 16 “Cryptographic Information Application” on page 16-122).

Alternativly this information may be provided in the Biometric Information Template (BIT), see ISO/IEC 7816-11 [16]. The BIT may be present in the DF.CIA or be retrieved with the GET DATA command.

Execution Flow (optional)

```
GET DATA (  INS = ‘CA’       // GET DATA
            P1-P2 = '7F61’   // Example: group template for multiple BITs
            data = absent)   // no data
```

Response APDU:BIT (Biometric Information Template(s))

For the coding of the command GET DATA see ISO/IEC 7816-4, chapter 7.4.2.

6.2.2 Performing the biometric user verification

Secure messaging is required when executing commands on biometric user verification.

Execution Flow

```
VERIFY ( INS = '20'|'21'        // VERIFY
         P1 = '00'              //
         P2 = key reference     // biometric data reference
         data = CG(biometric verification data) || CC)
```

If biometric verification data shall be presented in a biometric verification data DO (see ISO/IEC 7816-11) then INS code '21' shall be used.

Note:

1. The data field contains SM-DOs.
2. The AlgID, if required, is presented in the BIT or DF.CIA resp.
3. The value of keyref is to find in the BIT or DF.CIA resp.
4. If the length of data exceeds 254 bytes, then command chaining may be applied.

For the coding of the VERIFY command see ISO/IEC 7816-4, chapter 7.5.6.

6.2.3 Reset of RC

For resetting the RC refer to 6.1.5 "Reset of RC and setting a new password"

6.2.4 Biometric verification data format

Biometric verification formats are biometric type dependent.

For interoperability standardized formats are to be used (see e.g. ISO/IEC 19794-2 [31]).

7 Digital Signature Service

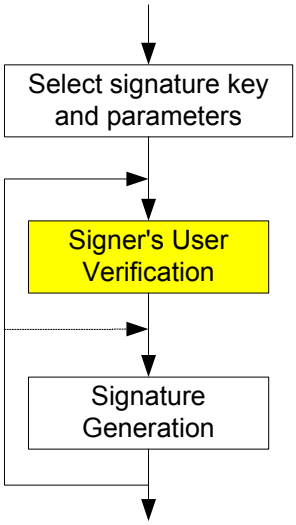
This chapter covers “electronic signatures” according to the EU Directive 5.1.

7.1 Signature generation algorithms

For the signature generation algorithmes as referenced in European and national algorithm catalogs shall be applied [7].

7.2 Activation of digital signature service

In order to enable the digital signature service a user verification shall be applied prior to signature generation.



Note -The loop is controlled by the signer, i.e. no signature is created without the willful act.

Figure 7-1. Signer’s User Verification

Refer to Figure 5-2 on page 5-17 for the overview of the signature generation flow.

For the description of the user verification refer to 6 “User verification” on page 22.

7.3 General Aspects

A digital signature process consists of several steps. Some of them are done outside the ICC, some of them are done inside the ICC, and some could be done on either side.

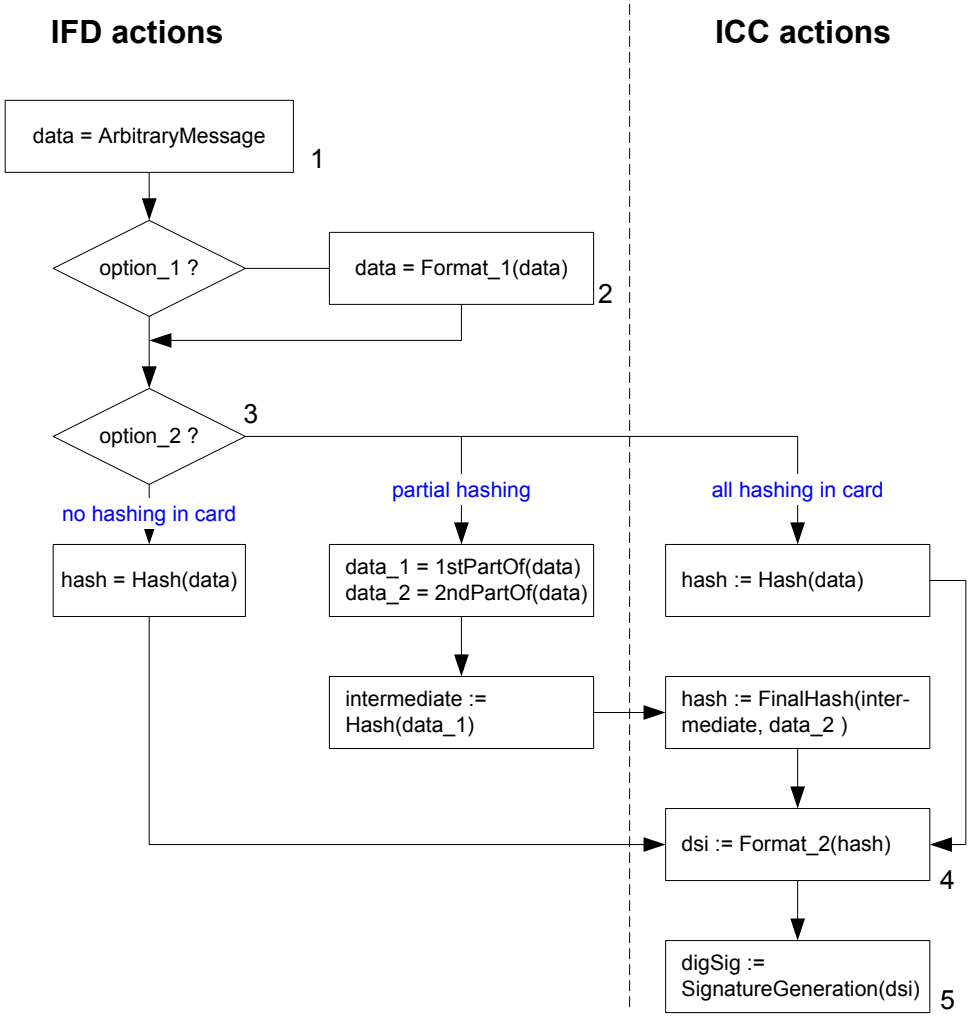


Figure 7-2. Data-Flow chart for a signature generation process

In principle, the signature generation process works as follows:

1. The signature generation process starts with an arbitrary message.

2. Optionally the original message is formatted in an application specific way. Typically signature attributes and other format information besides the actual document to be signed are added to the arbitrary message. The specification of the optional format mechanism Format_1 is outside the scope of this document.

3. The data has to be hashed by applying a hash function which is on the list of the applicable hash functions. The hash result could be considered as a compressed version of the text. Depending on option_2 the hashing will be done either

   - completely outside the ICC,

- partly outside and partly inside the ICC, thus data is split into two parts,
- completely inside the ICC

depending on the card functionality and the software organisation.

4. Depending on the signature algorithm the hash value has to be formatted by a format mechanism Format_2. Several format mechanisms have been defined, some of them are evaluated as “weak” and should not be used. The algorithm catalogues cover these aspects and propose formatting schemes which are approved until a specified date.

5. The final dsi (Digital Signature Input) is the string to be transformed by the signature algorithm. The result is the Digital Signature digSig.

7.4 Signature Generation

The data to be hashed or the final digest respectivly are sent to the ICC with the PSO: HASH command as described in ISO/IEC 7816-8 [14].

Execution Flow 1a of 2 (all hashing in ICC)

```
PSO( INS = ‘2A’,          // PERFORM SECURITY OPERATION
     P1 = ‘90’,           // return hash code if required by Le
     P2 = ‘80’,           // plain value
     data = plain data to be hashed)
```

Execution Flow 1b of 2 (partial hashing in ICC)

```
PSO( INS = ‘2A’,          // PERFORM SECURITY OPERATION
     P1 = ‘90’,           // return hash code if required by Le
     P2 = ‘A0’            // input template for hash computation
     data =               ‘90’ L90 <intermediate hash result> ||
                          ‘80’ L80 <data to be hashed> ;
```

Execution Flow 1c of 2 (all hashing outside ICC)

```
PSO( INS = ‘2A’,          // PERFORM SECURITY OPERATION
     P1 = ‘90’,           // return hash code if required by Le
     P2 = ‘A0’            // input template for hash computation
     data = ‘90’ L90 <hash value>
```

For the structure and content of the APDU refer to ISO/IEC 7816-8, chapter 5.2..

For the content of the command data field or DO ‘A0’ resp. (HASH Input) refer to 13.2 “Hash Input-Formats” on page 99.

As the hash value is available in the ICC after each of the variants, typically the response data field is absent. After hashing the PSO: COMPUTE DIGITAL SIGNATURE command has to be sent to the ICC in order to create a signature from the hash value and other parameters which are determined by the corresponding algorithm. That algorithm is either implicitly determined or it has been set prior to the signing operation by a MSE:SET command.

Execution Flow 2 of 2

```
PSO( INS = ‘2A’,          // PERFORM SECURITY OPERATION
     P1 = ‘9E’,           // COMPUTE DIGITAL SIGNATURE
     P2 = ‘9A’,           // data to be signed
     data = absent)       // data already in ICC
```

The digital signature input format is specified through the appropriate signature algorithm, padding format and, if required, the digest info. The ICC shall derive the digital signature input with the parameters specified by the selected security environment.

The format of the digital signature input is described in 13.3 “Formats of the Digital Signature Input (DSI)” on page 101.

For the structure and content of the APDU refer to ISO/IEC 7816-8, chapter 5.2.

The command APDU data field is absent as the hash value is already available in the card.

The response data field contains the plain signature without further TLV coding.

T	L	V	
-	-	SIG	= unformatted signature
-	-	‘9000’	= SW1 SW2 (or specific status bytes)

Table 7-1. Response data field for COMPUTE DIGITAL SIGNATURE

7.5 Selection of different keys, algorithms and input formats

If the card supports more than one key and an implicit selection is not available, then the private signature key needs to be selected.

If the card supports several digital signature algorithms or different Digital Signature Input formats (e.g. according to PKCS #1 Version 2.0 (variant of version 1.5) (default) and ISO/IEC 9796-2 (Digital signature scheme 1), (see 13 “AlgIDs, Hash- and DSI Formats” on page 98), then the algorithm or the DSI format respectively has to be selectable. However, it is possible, that e.g. an algorithm is implicitly specified if a key is selected.

It is further possible that an algorithm, a key or a signature input format is not implicitly known, nor uniquely specified.

In all of the above cases the appropriate parameter has to be selected explicitly with a MANAGE SECURITY ENVIRONMENT command as described in ISO/IEC 7816-4, chapter 7.5.11.

This can be achieved by

- restore an existing SE →7.5.1
- modify a KeyID/[AlgID] in a template →7.5.2

7.5.1 Restore an existing SE

Execution Flow

```
MSE( INS = ‘22’,          // MANAGE SECURITY ENVIRONMENT
     P1 = ‘F3’,           // RESTORE
     P2 = SE#,            // number of security environment
     data = absent);
```

For the structure and content of the APDU refer to ISO/IEC 7816-4, chapter 7.5.11.

Note: The actual SE# is described in Part 2 of this specification.

The response data field is absent.

7.5.2 Modify the DST of a current SE

If a signing key is not implicitly known by the ICC, it has to be selected with the MANAGE SECURITY ENVIRONMENT command. Optionally an algorithm identifier can be supplied in order to specify variations of the algorithm (e.g. for padding formats).

Execution Flow for selection of signature key

```
MSE( INS = ‘22’                      // MANAGE SECURITY ENVIRONMENT
     P1 = ‘41’,                      // SET signature creation
     P2 = ‘B6’,                      // CRT for DST
     data = ‘84 L84 xx’ || [‘80’L80 yy]) // KeyID || [AlgId]
```

For the structure and content of the APDU refer to ISO/IEC 7816-4, chapter 7.5.11.

For the algorithm ID see Table 13-1 on page 13-98.

The response data field is absent.

7.6 Read certificates and certificate related information

If more than one signature key is present in the card, the user may want to select the signature key for a certain signature creation. In his certificates there are fields indicating e.g. whether it is a private or professional certificate. Therefore the user may want to have a look on these fields in order to select the proper certificate and with it the associated private key. In the CIOs there is a link between a certificate and the associated private key and an indication where the certificates are to be found (in the card or on a server), in which case the card will provide an url and a certificate serial number.

7.6.1 Read certificate related CIOs

CIOs related to certificates denote which certificates are present in the ICC or are to be found on a server and which private key corresponds to the public key present in the certificate (see 16 “Cryptographic Information Application”).

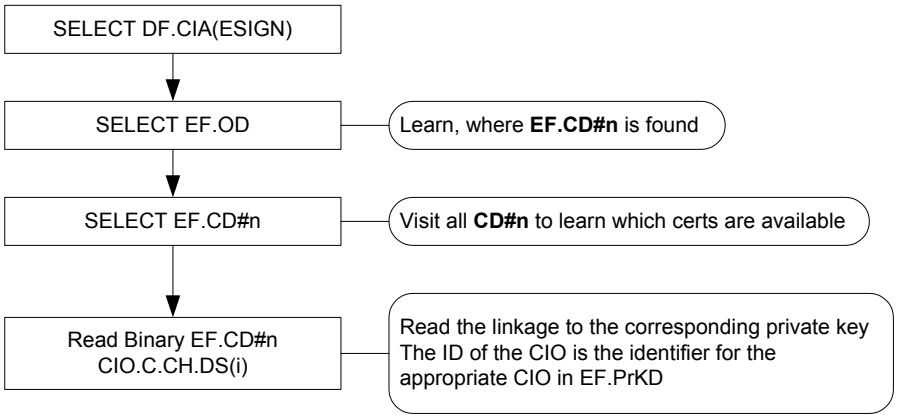


Figure 7-3. Reading certificate related information (certificate is present in the ICC)

The certificate related information contains the ID of the appropriate CIO in EF.PrKD which contains the KeyId of the private key in question.

7.6.2 Read signer’s certificate from ICC

If present, the corresponding certificate file(s) may be read from the ICC. Prior to reading the appropriate file needs to be selected. The location (e.g. path or (short) FileID) is denoted in the corresponding CIO. in EF.CD#n.

The actual certificate is stored in EF.C.CH.DS(i). After selection of EF.C.CH.DS(i) according to the path information found in the CIO.C.CH.DS(i), the certificate may be read from the ICC.

Execution Flow:

```
READ BINARY( INS = ‘B0’         // READ BINARY
             P1 = ‘00’,         // Example: offset = 0x0000
             P2 = ‘00’          //
             Le = ‘00’,         // read all data

Response = data.EF.C.CH.DS(i)
```

- The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.2.3.

- The structure and coding of EF.C.CH.DS(i) will typically be an X.509 certificate format, but it could also be some other format. The actual format is application specific and out of the scope of this standard.

Note: If the expected length exceeds 256 bytes, then the READ BINARY might have to be called several times with an appropriate offset.

7.6.3 Retrieval of the signer's certificate from a directory service

In order to retrieve the signer's certificate through a directory service the card holder has to identify the certificate uniquely. This reference value information may be retrieved by reading the appropriate CIO.C.CH.DS from the DF.CIA(ESIGN) according to Figure 7-3.

According to ISO/IEC 7816-15 [17] Chapter "8.7.2 X.509 certificate attributes" the appropriate data object. CIO.C.CH.DS contains the following fields.

```
X509CertificateAttributes ::= SEQUENCE
{
  value ObjectValue { Certificate },
  subject Name OPTIONAL,
  issuer [0] Name OPTIONAL,
  serialNumber CertificateSerialNumber OPTIONAL,
  ... -- For future extensions
}
```

The interpretation of the components shall be as follows:

X509CertificateAttributes.value:

The value shall be a *ReferencedValue* either identifying a file containing a DER encoded certificate at the given location, or a URL pointing to some location where the certificate can be found.

8 Device authentication

This clause is related to the assumption/model that a device authentication has (to be/ been) performed. Any statements made in this paragraph are conditional to that assumption. A specification for the need of device authentication is given in 8.2 “Authentication environments” on page 8-37.

Device authentication requires mandatory steps in order to provide a secure authentication. A device authentication is mutual and combines two mechanisms

- a device verifies the existence of a certified secret key on the other part
- the devices negotiate or exchange information to establish common session keys for subsequent operations.

After the negotiation of the symmetric keys, a secure session is established. A secure session is a cryptographic protection of the messages from either side. The cryptographic protection can be a cryptographic checksum on a plain text or even an encrypted message text. Refer to 9 “Secure Messaging” on page 9-67.

For performance reasons, the keys used for this secure messaging are symmetric keys. Therefore this document describes the establishment of symmetric session keys only, and does not consider an option for asymmetric session keys.

Once the sessions keys are established, a trusted channel is available to protect or conceal the information transmitted over the interface. The application of Secure Messaging (SM) is **highly recommended** for **every subsequent operation** to ensure the provision of an entirely trusted channel. However for compatibility reasons this specification does not mandate usage of secure messaging (SM) in all cases.

This document proposes

- an asymmetric session key transport scheme
- an asymmetric session key agreement schemes
- a symmetric session key exchange scheme

in order to perform a mutual authentication protocol between IFD and ICC. The presentation of the asymmetric schemes is staged in order to trade off the security features with the required complexity of the authentication protocols.

Note: The IFD usually represents any external interface not being particularly specified

8.1 Certification authorities and certificates

8.1.1 Certificate chains

Asymmetric authentication requires the presentation of public keys in certificates. A chain of certificates might have to be presented. To establish the 'starting point of trust' the first certificate sent to the ICC shall always be verifiable with a public key already existing in the card. The most general of such keys available in the ICC can be a Root CA public key. Each certificate introduces another public key, while the presented certificate is always signed with a key, known by the ICC.

A certificate, signed by a descendant of Root A that conveys a public key that is a descendant of Root B is called a cross-certificate (see 8.1.2). The use of cross certificates allows the use of the ICC in different application environments. Cross certificates are also required for certificate expiration issues.

If a Root certificate is available, then Root CA provides certificates for the subordinate CAs or cross certificates are used in order to convey the public key(s).

Subordinate CAs issue certificates for users with the appropriate ESIGN application.

In order to use an ESIGN application a mutual authentication between card and customer terminal is required by the security policy. As PK algorithms are to be employed ICC and IFD authentication certificates are required.

The card is the carrier of the ICC authentication certificate; carrier of the IFD authentication certificate could be a plug-in card as an example for the realisation of an authentication module inside the SCA (see Figure 8-1.)

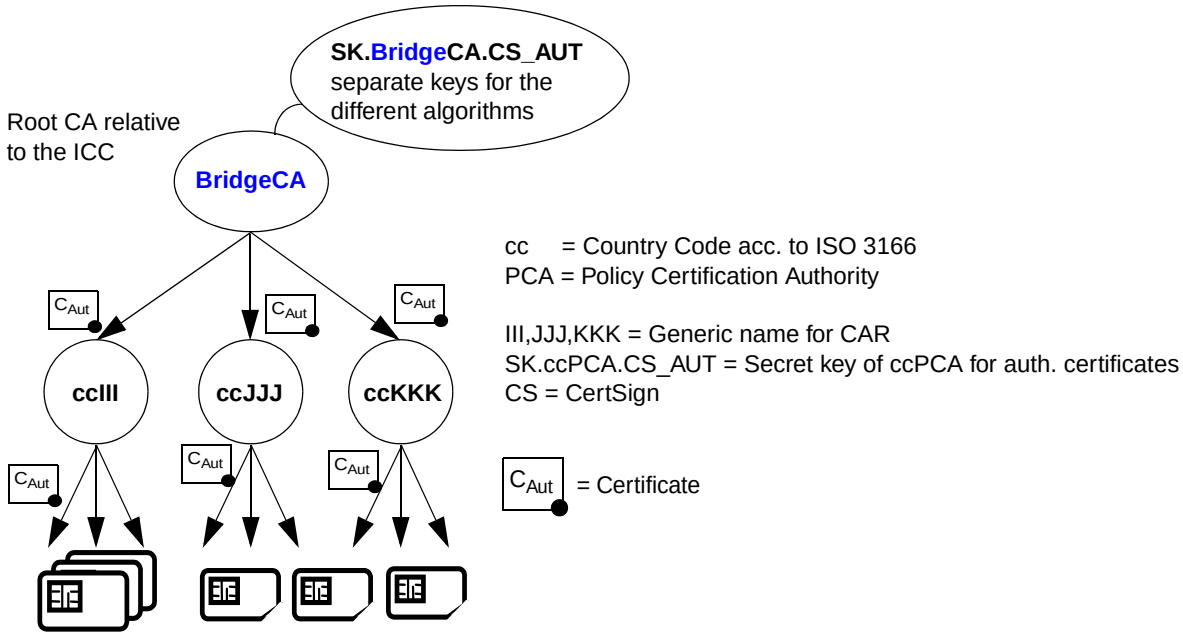


Figure 8-1. Example for certificate authorities and certificates for ICC and IFD authentication certificates

The above example repeats for every RCA key that is present in the card

8.1.2 Usage of cross certificates

A typical use of cross certificates is the link of an existing certificate chain to an existing release of a public key.

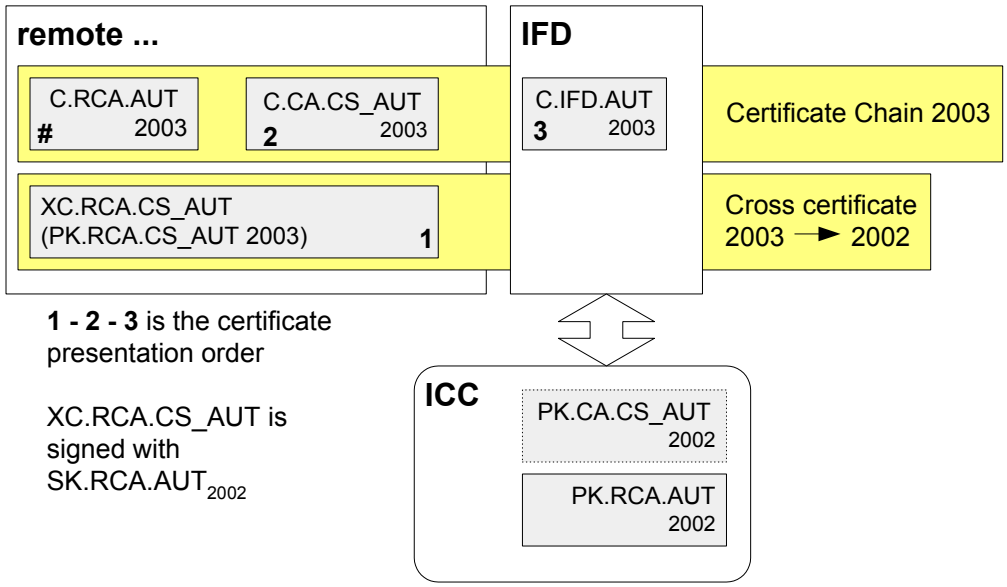


Figure 8-2. Usage of cross certificates for different key versions

The example above shows PK.CA.CS_AUT(2002) available at the ICC. This key was brought to the card in 2002, however in 2003 the root key was changed and another key chain

PK.RCA.AUT(2003) →C.CA.CS_AUT(2003) →C.IFD.AUT(2003)

was used when generating the IFD's certificate C.IFD.AUT(2003).

In order to allow authentication of the IFD, in the above example the ICC might have to receive a cross certificate XC.RCA.CS_AUT(2003) first. This certificate is signed with the SK.RCA.AUT(2002), however it transports the PK.RCA.AUT(2003).

After having verified the cross certificate XC.RCA.CS_AUT(2003) the ICC may now receive the certificates of the 2003-chain up to the C.IFD.AUT which carries the final public key required for authentication.

The ICC does not recognise whether an incoming certificate is a cross-certificate or part of a certificate chain. If it can verify the incoming certificate with an existing public key, the ICC accepts the transported public key and may use it for the next instance of the chain. The 'start of trust' is always established with the first certificate, being verified with a public key that is available in the ICC.

When verifying a certificate, the verifier has to verify the correct role of the signer. Only those roles being allowed to sign certificates may be accepted for the presentation of a certificate chain. The role of a certificate is specified in 14.7.5 "CHA-Certificate Holder Authorisation (CHA)" on page 14-109.

8.2 Authentication environments

In [4] two environments are distinguished with respect to signature creation applications.

8.2.1 SCA in trusted environment

A trusted channel is available. Device authentication is not required as the card holder knows the trusted environment that he will apply the card to. This is implicitly the case for the WIM application [32], as the WIM card is considered to be in trusted environment a priori.

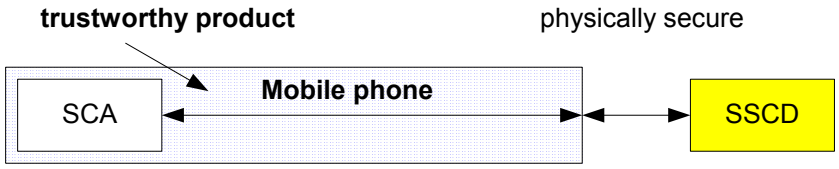


Figure 8-3. Trust of the environment

In this environment, device authentication as described in the rest of this chapter is optional.

8.2.2 SCA in untrusted environment

A device authentication shall be used if the operating environment of the card cannot be entirely trusted. This can be the case in public signature terminals or other devices, that cannot provide a trusted channel.

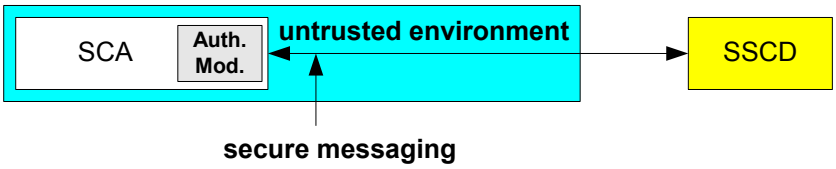


Figure 8-4. Communication in untrusted environment

Device authentication is mutual. The ICC shall authenticate the IFD and vice versa. The order of authentication, however, may differ, depending on the implemented scheme.

After successful device authentication, session keys are available on both sides to be used in subsequent transmissions. The appropriate secure messaging is in compliance with ISO/IEC 7816-4 [11] and described in 9 “Secure Messaging” on page 9-67.

Examples for an untrusted environment are

- SCA and SSCD are not at the same location, i.e. the card is remote
- usage of biometrics

8.2.3 Specification of the environment

Whether an environment is trusted or not cannot be evaluated by the IFD a priori. Therefore it must be left to the card holder to decide, whether he may trust the application environment or not. The initiation of a device authentication in the application environment is out of the scope of this specification.

8.2.4 Display message mechanism

Given, the card holder has initiated a device authentication this specification proposes a ‘display message’ mechanism in order to verify that the device authentication has been performed successfully. If used, after successful device authentication, a display message shall indicate the successful device authentication in order to inform the user about the existence of a trusted channel. The application of the display message is described in 8.11 “Reading the Display Message” on page 8-63.

8.2.5 Additional authentication environments

Device authentication may also be applied or necessary for multiple entity personalization (not in the scope of this standard).

8.3 Key transport and key agreement mechanisms

Key transport is the process of transferring a secret key, chosen by one entity (or a trusted center), to another entity, suitably protected by asymmetric techniques. Key transport requires encryption of the key part, this is typically done with the public key of the counterpart. If used, for the establisment of the secure messaging, key transport is used in both directions in order to derive a common key in the IFD and ICC.

Key agreement is the process of establishing a shared secret key between two entities A and B in such a way that neither of them can predetermine the value of the shared secret key. Key agreement mechanisms may be integrated in the device authentication.

In the context of session key establishment, ‘implicit key authentication’ means that after the execution of the mechanism only an identified entity can be in possession of the correct shared secret key.

This specification describes the following protocols

- 8.4 “Key transport protocol” on page 8-39
- 8.5 “Device authentication with privacy protection” on page 8-48
- 8.7 “Symmetric authentication scheme” on page 8-59

8.4 Key transport protocol

The following figure shows device authentication with key transport. The infomation in the fields is simplified. A detailed description of the coding is specified below.

Stage	Step	IFD	Trans-mission	ICC
1	A	**If $PK.CA_{IFD}$.CS_AUT is available in the ICC continue with step D**		
	B	Select and/or verify PK.RCA.AUT.	→ ←	Select and/or verify key. Response OK
	C	PSO:Verify Certificate C_CV.CA.CS_AUT	→ ←	Verify certificate. Store $PK.CA_{IFD}$.CS_AUT Response OK
	D	Select and/or verify $PK.CA_{IFD}$.CS_AUT.	→ ←	Select and/or verify key. Response OK
	E	PSO:Verify Certificate C_CV.IFD.AUT	→ ←	Verify certificate. Store PK.IFD.AUT Response OK
The Public Key of the IFD is now known by the ICC, and can be trusted				
2	F	**If $PK.CA_{ICC}$.CS_AUT is available in the IFD continue with step H**		
	G	READ BINARY of file $EF.C.CA_{ICC}.CS_AUT$[1]	→ ←	Read data from specified file. C.CA.AUT is in response APDU.
	H	READ BINARY of file EF.C.ICC.AUT Verify C.ICC.AUT and store PK.ICC.AUT temporarily	→ ←	Read data from specified file. C.ICC.AUT is in response APDU.
The Public Key of ICC is now known by the IFD, and can be trusted				

Table 8-1. Reference device authentication Scheme 1 with Key Transport (Part 1 of 2)

Stage	Step	IFD	Trans-mission	ICC
3	I	MSE:SET KeyId(SK.ICC.AUT) for authentication[2]	→	Select SK of ESIGN application.
			←	Response OK
		MSE:SET KeyId(PK.IFD.AUT) for encryption	→	
			←	Response OK
	J	INTERNAL AUTHENTICATE (**C**) where **C** = RND.IFD || SN.IFD	→	ESIGN application calculates: SIG = DS[SK.ICC.AUT] (...|| K_{ICC} || h(...|| K_{ICC} || **C**) ||...))
		Decrypt with SK.IFD.AUT and verify response from ICC with PK.ICC.AUT	←	E.PK.IFD(min(SIG, N.ICC - SIG) **C** is from Internal Auth. command
IFD has now authenticated the card				
4	K	GET CHALLENGE	→	
			←	RND.ICC
	L	EXTERNAL AUTHENTICATE IFD calculates: SIG = DS[SK.IFD.AUT] (...|| K_{IFD} || h(... || K_{IFD} || **S**) ...)	→	
		E[PK.ICC.AUT] (min(SIG, N.IFD - SIG) with **S** = RND.ICC || SN.ICC	→	Verify that K_{IFD} is different from K_{ICC} Verify the signature
		SN.ICC was received in the certificate of Step H.	←	OK !
Two-way authentication is now complete Key derivation according to "Then the actual session keys are built from $K_{IFD/ICC}$ according to chapter 8.8 ." on page 8-61				

Table 8-1. Reference device authentication Scheme 1 with Key Transport (Part 2 of 2)

1 This step is required only if the IFD does not have the public key of the CA.
2 Both MSE commands of this step may be realized with one MSE command that transmits a CRT with both key references

8.4.1 Authentication Steps

The following are the essential steps of authentication with key transport. The given reference provides highest achievable protection of the IFD's identiy and data until the ICC has been authenticated first.

8.4.1.1 Step A - Skip to authentication (conditional)

If the PK.CA_{IFD}.CS_AUT is available in the ICC, the steps B - C are not required to be executed.

8.4.1.2 Step B - Selection of verification key PK.RCA.AUT (conditional)

If PK.CA_{ICC}.AUT is not available in the ICC a chain of (cross-)certificates must be presented to the ICC in order to confirm the validity of each certificate. The chain of certificates ends at any key which is available in the ICC. The last possible key available in the ICC is the public key of the root CA, named PK.RCA.AUT. Steps B and C have to be performed repeatedly until the correct PK.CA.AUT is available in the ICC. This process is called multi-stage verification.

The multi-stage verification is necessary if and only (a) the CA is not the root CA (if it is the root CA, the key necessarily is present in the card), and (b) the appropriate public key of the CA is not present in the card.

Execution Flow:

```
MSE( INS = '22'              // MANAGE SECURITY ENVIRONMENT
     P1 = '81'               // SET for verification
     P2 = 'B6'               // DST
     data = PK.RCA.AUT <or other key in chain>)
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

The structure and coding of the key reference data is specified in section 12.1.2 "CRT for selection of IFD's PK.CA_{IFD}.CS_AUT" on page 12-91.

8.4.1.3 Step C - Verify Certificate C_CV.CA.CS_AUT (conditional)

In multi-stage verification the certificate of a CA's public key C_CV.CA.CS_AUT is sent to the card with the VERIFY CERTIFICATE command. The ICC verifies this certificate with the public key sent in the previous step. If the verification fails, the ICC responds with an appropriate return code. On success, the ICC stores the PK contained in the certificate and allows a further import of certificates.

The role of the certificate has to comply with its signer. The selection of PK.RCA can always be accepted to verify a chained certificate. Any other key, that was imported with certificate may be applied only, if the role of its certificate matches the purpose of verification of the next incoming certificate.

Execution Flow:

```
PSO( INS = '2A'                  // PERFORM SECURITY OPERATION
     P1 = '00'                   // response data field absent
     P2 = 'AE'                   // VERIFY CERTIFICATE
     data = C_CV.CA.CS_AUT)      // any cert in chain
```

The command and response APDUs are described in section ISO/IEC 7816-8, chapter 5.2.

The structure of the certificate is specified in section 14.9 “C_CV.CA.CS-AUT” on page 14-114.

There are two alternatives for the transmission of the certifcate

1. Transmission of the entire certificate including the termplate tag ‘7F 21’ as described in 14.8 “C_CV.IFD.AUT” on page 14-113.

2. Transmission of relevant data objects from the value field of the certificate as described in 14.8 “C_CV.IFD.AUT” on page 14-113. These data objects may be transmitted with command chaining.

Note: The ICC must verify evidence, that the role for the PK of the IFD either corresponds to signing another (subsequently transmitted) certificate or to the authentication mechanism. The role (see 14.7.5 on page 14-109) is part of the certificate when the PK is imported. The role of the last PK imported with VERIFY CERTIFICATE has to be stored in the ICC until the execution of the EXTERNAL AUTHENTICATE command in order to verification.

8.4.1.4 Step D - Selection of verification key $PK.CA_{IFD}AUT$

The MSE command sets the key ID of the CA’s public key to be used for the verification of the IFD’s authentication certificate.

Execution Flow:

```
MSE( INS = ‘22’              // MANAGE SECURITY ENVIRONMENT
     P1 = ‘81’               // SET for verification
     P2 = ‘B6’               // DST
     data = KeyRef(PK.CA_IFD.CS_AUT)
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

For the structure and coding of the key reference data is specified in section 12.1.2 “CRT for selection of IFD’s $PK.CA_{IFD}$.CS_AUT” on page 12-91.

8.4.1.5 Step E - Verify Certificate C_CV.IFD.AUT

The IFD sends a card verifiable (CV) certificate C_CV.IFD.AUT along with the VERIFY CERTIFICATE command. The ICC verifies the certificate using the public key $PK.CA_{IFD}$.CS_AUT of the certification authority CA.

The certificate contains the IFD’s PK.IFD.AUT, which is stored temporarily by the ICC

Execution Flow:

```
PSO( INS = ‘2A’                 // PERFORM SECURITY OPERATION
     P1 = ‘00’                  // response data field absent
     P2 = ‘AE’                  // VERIFY CERTIFICATE
     data = C_CV.IFD.AUT)       // any cert in chain
```

The command and response APDUs are described in section ISO/IEC 7816-8, chapter 5.2.

The structure and coding of the card verifiable certificate is specified in section 14.8 “C_CV.IFD.AUT” on page 14-113.

There are two alternatives for the transmission of the certifcate

1. Transmission of the entire certificate including the termplate tag ‘7F 21’ as described in 14.8 “C_CV.IFD.AUT” on page 14-113.

2. Transmission of relevant data objects from the value field of the certificate as described in 14.8 “C_CV.IFD.AUT” on page 14-113. These data objects may be transmitted with command chaining.

   For the transmittion of the data field, it is only necessary to send the relevant data objects of the certificate. If this is the case, only the value field of the ‘7F 21’ template may be sent, possibly with command chaining.

   The ICC stores the key in a temporary location for use in subsequent steps of the authentication, together with identification data such that the key can be recognized when specified in an MSE:SET APDU. The CHA has to be stored in order to identify the role of the key for further usage.14.7.5 “CHA-Certificate Holder Authorisation (CHA)” on page 14-109.

   The execution flow of the VERIFY CERTIFICATE is outlined in 14.7.7 “Flowchart for the verification of certificates” on page 14-111.

After Step E the IFD’s public authentication key PK.IFD.AUT can be trusted.

8.4.1.6 Step F - Skip reading chain certificates

If the public key $PK.CA_{ICC}.CS_AUT$ is available in the IFD or can be obtained by other means than reading it from the ICC, step G can be skipped.

8.4.1.7 Step G - Read $C.CA_{ICC}.AUT$ (conditional)

The IFD obtains $PK.CA_{ICC}.CS_AUT$ by reading $C.CA_{ICC}.CS_AUT$ from a file $EF.C.CA_{ICC}.AUT$ in the ICC. Then it verifies the certificate $C.CA_{ICC}.CS_AUT$ and extracts and stores the public key $PK.CA_{ICC}.CS_AUT$ temporarily for use in step H.

Execution Flow:

```
READ BINARY( INS = ‘B0’
             P1 = ‘83’          // Example SFI = 03
             P2 = ‘00’          // offset = 0
             Le = ‘00’)         // read all data
```

Response = data.EF.C.CA_{ICC}.CS_AUT

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.2.3..

The structure and coding of $C.CA_{ICC}.AUT$ may be as described in section 14.9 “C_CV.CA.CS-AUT” on page 14-114, but it could also be some other format unknown to the ESIGN application. The actual format is application specific and out of the scope of this standard.

8.4.1.8 Step H - Read ICC’s certificate C.ICC.AUT

The IFD reads the ICC’s certificate to be used for later authentication.

The received certificate C.ICC.AUT contains the ICC’s public key. The IFD verifies the certificate using the public key of the certification authority $PK.CA_{ICC}.CS_AUT$. If C.ICC.AUT can be verified successfully, the PK.ICC.AUT is stored in the IFD and can be trusted for subsequent authentication.

Execution Flow:

```
READ BINARY( INS = ‘B0’
             P1 = ‘84’          // Example: SFI = 04
             P2 = ‘00’          // offset = 0
             Le = ‘00’)         // get all data

Response = data.EF.C.ICC.AUT
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.2.3..

See section 14.10 “C.ICC.AUT” on page 14-115 for more information concerning the structure and content of this certificate.

8.4.1.9 Step I - Key selection

Before processing the INTERNAL AUTHENTICATE command the ICC’s private authentication key SK.ICC.AUT must be set by the MSE command.

Furthermore, the IFD’s public key PK.IFD.AUT needs to be selected for encryption in order to transport the ICC’s key contribution K_{ICC}. K_{ICC} is a 32 byte random number generated by the ICC.

The command MSE checks, whether the keys named (qualified with purpose of use and if needed further parameters) are available. Then the keys are set. The following commands check, whether the keys set here can be used

Execution Flow

```
MSE( INS = ‘22’         // MANAGE SECURITY ENVIRONMENT
     P1 =’C1’           // SET for internal and external authentication
     P2 =’A4’           // AUT
     data = CRT(SK.ICC.AUT) || CRT(PK.IFD.AUT)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.11.

For the coding of the key CRT content refer to 12.1.2 “CRT for selection of IFD’s $PK.CA_{IFD}$.CS_AUT” on page 12-91

8.4.1.10 Step J - Internal Authentication

The IFD performs an INTERNAL AUTHENTICATE command. The challenge sent to the ICC with this command is `RND.IFD || SN.IFD.` K_{ICC} is a 32 byte random number generated by the ICC.

The ICC then computes the signature over the challenge and returns it to the IFD encrypted with the PK.IFD.AUT.

The IFD verifies the response with with the trusted key of the ICC: PK.ICC.AUT.

Execution Flow

```
INTERNAL AUTHENTICATE(   INS = ‘88’
                         P1 = ‘00’
                         P2 = ‘00’       // key reference, none
```

data = **C** = RND.IFD || SN.IFD)

Response = E[PK.IFD.AUT](SIGMIN)

SIGMIN = min(SIG, N.ICC - SIG);

SIG = DS[1][SK.ICC.AUT]('6A' || PRND1 || K_{ICC} || h(PRND1 || K_{ICC} || **C**) || 'BC')

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.2.

The *SN.IFD* is taken from the C_CV certificate's *CHR* field (see 14.7.4 on page 14-109). The least significant 8 bytes of *SN.IFD* are used. *SN.IFD* was stored in Step E.

For the content of the response data field refer to 12.2.2 "INTERNAL AUTHENTICATE" on page 12-95.

The signature is formatted according to ISO/IEC 9796-2 (DS scheme 1) [20].

Note 1: Encryption of the signature $Sig_{SK.ICC.AUT}( )$ with PK.IFD.AUT requires some mathematical restrictions in order to produce a correct result. The length of the two RSA moduli N.ICC and N.IFD must be identical. This together with the use of min (SIG, SIG*) according to ISO 9796-2 (DS scheme 1) guarantees that the deciphering with SK.IFD.AUT delivers a unique result. For the mathematical explanation see A.2.2 "Using min(SIG, N-SIG) for the signature token" on page A-134.

8.4.1.11 Step K - Get Challenge

In order to proof its authenticity dynamically the IFD shall request a challenge from the ICC. The challenge consists of a random number large enough in order to avoid exhaustive search attacks.

Execution Flow

GET CHALLENGE(INS = '84',
Le = '08')

Response = random number RND.ICC

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3.

8.4.1.12 Step L - External authentication

The IFD computes a signature on the concatenation of the challenge with its own key token information K_{IFD} using using **SK.IFD.AUT**. The IFD encrypts the signature with PK.ICC.AUT and sends it to the ICC. K_{IFD} is a 32 byte random number generated by the IFD. The ICC verifies, that K_{IFD} is different from K_{ICC}, otherwise an error is reported.

After the ICC verified the signature with the trusted public key PK.IFD.AUT, it responds with a confirmation whether the authentication was successful or not.

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

Execution Flow

```
EXTERNAL AUTHENTICATE(   INS = ‘82’
                         P1=’00’
                         P2=’00’
                         data = E[PK.ICC.AUT](SIGMIN) )

with

    SIGMIN = min (SIG, N.IFD - SIG)

and

    SIG = DS[1][SK.IFD.AUT](’6A’ || PRND || K_IFD ||
                                h(PRND.|| K_IFD || RND.ICC || SN.ICC) || ‘BC’)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3.

For the structure and coding of the data field refer to 12.2.1 “EXTERNAL AUTHENTICATE” on page 12-94

Note 1: In order to avoid the “Bleichenbach attack” [47], the error response shall be the same for failing the verification of the encryption (E.PK.ICC.AUT) and the verification of the signature with PK.IFD.AUT.

Note 2: Encryption of the signature Sig[SK.IFD.AUT](...) with PK.ICC.AUT requires some mathematical restrictions in order to produce a correct result. The length of the two RSA moduli N.ICC and N.IFD must be identical. This together with the use of min (SIG, SIG*) according to ISO 9796-2 (DS scheme 1) guarantees that the deciphering with SK.ICC.AUT delivers a unique result.

Note 3: The ICC must verify evidence, that the role for the PK of the IFD corresponds to the external authentication mechanism. The role (see 14.7.5 on page 14-109) is part of the certificate when the PK was imported. The role has to be stored in the ICC temporarily between the execution of the VERIFY CERTIFICATE command and the EXTERNAL AUTHENTICATE command in order to allow this verification.

8.4.2 Session Key creation

One part of the authentication procedure is the agreement of session keys for building cryptograms and cryptographic checksums with TDES.

In a first step the 32-byte values K_{IFD} and K_{ICC} are exclusivly or-ed.

$$K_{IFD/ICC} = K_{IFD} \oplus K_{ICC}$$

Then the actual session keys are built from $K_{IFD/ICC}$ according to chapter 8.8

Note: Alternativly, for the key transport protocol, the session key derivation may be computed as follows

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

$K_{IFD/ICC}$ (32 bytes) is interpreted as concatenation of the 4 necessary keys (each 8 bytes).

K = **Ka**(ENC) || **Kb** (ENC) || **Ka** (MAC) || **Kb** (MAC)

This solution may be applied in compliance with existing infrastructure. In general the session key derivation according to 8.8 “Compute session keys from key seed $K_{IFD/ICC}$” is recommended.

8.5 Device authentication with privacy protection

The ‘privacy protocol’ enhances the key transport protocol by a Diffie-Hellman key negotiation, prior to the authentication. The identity of the ICC is revealed to the IFD not before successful authentication of the IFD.

Usage of the privacy protocol mandates to authenticate the IFD first.

8.5.1 Authentication steps

Stage	Step	IFD	Trans-mission	ICC
1	1	READ BINARY of file DH global parameters g, p, q.[1]	→ ←	Read DH key parameters *g*, *p* and *q* from specified file. Data is in response APDU
	2	compute $x_{IFD} = g^{r_{IFD}} \bmod p$ MSE:SET:KAT (KID = g,p,q parms || K_{IFD})	→ ←	OK
	3	GET DATA (K_{ICC}) compute $K_{IFD/ICC} = K_{ICC}^{r_{IFD}} \bmod p$	→ ←	compute $x_{ICC} = g^{r_{ICC}} \bmod p$ compute $K_{IFD/ICC} = K_{IFD}^{r_{ICC}} \bmod p$ K_{ICC}
Secure messaging keys are derived at both sides according to. Secure messaging (encryption) is activated. All following commands are protected by secure messaging with encryption as described in 9 “Secure Messaging” on page 9-67. The SSC is initialized to a start value of ‘1’.				

Table 8-2. Reference device authentication scheme 1 with Diffie-Hellman Key Exchange (Part 1 of 3)

Stage	Step	IFD	Trans-mission	ICC
2	4	**If PK.CA.AUT is available in the ICC continue with step 7**		
	5	Select and/or verify PK.RCA.AUT.	→	Select and/or verify key.
			←	Response OK
	6	PSO:Verify Certificate C_CV.CA_{IFD}.CS_AUT	→	Verify certificate.
			←	Response OK
	7	Select and/or verify PK.CA_{IFD}.CS_AUT.	→	Select and/or verify key.
			←	Response OK
	8	PSO:Verify Certificate C_CV.IFD.AUT	→	Verify certificate.
			←	Response OK
The Public Key of the IFD is now known by the ICC, and can be trusted				
3	9	MSE:SET PK.IFD.AUT Select IFD's public key and select the key parameters for the consecutive operation:	→	Select PK of IFD
			←	Response OK
	10	GET CHALLENGE	→	
			←	RND.ICC
	11	EXTERNAL AUTHENTICATE SN.IFD || DS[SK.IFD.AUT] (...|| h(...|| K_{IFD} || **S**)) with **S** = SN.IFD || RND.ICC || K_{ICC} || DH.P	→	ICC verifies signature. A's identity is implicitly verified as known by the certificate C_CV.IFD.AUT.
			←	OK
ICC has now authenticated the reader - SM (encryption) active				
4	12	READ BINARY of file EF.C.CA_{ICC}.CS_AUT[2]	→	Read data from specified file.
			←	C.CA_{ICC}.CS_AUT as response
	13	READ BINARY of file EF.C.ICC.AUT	→	Read data from specified file.
			←	C.ICC.AUT is in response APDU
The Public Key of ICC is now known by the reader, and can be trusted				

Table 8-2. Reference device authentication scheme 1 with Diffie-Hellman Key Exchange (Part 2 of 3)

Stage	Step	IFD	Trans-mission	ICC
5	14	MSE:SET SK.ICC.AUT	→	Select SK of ESIGN application.
			←	Response OK
	15	INTERNAL AUTHENTICATE (RND.IFD) IFD verifies signature. B's identity is implicitly verified as known from the certificate C.ICC.AUT	→	The ESIGN application computes SN.ICC || DS[SK.ICC.AUT] (...|| h(...|| K_{ICC} || **S**)) with **S** = SN.ICC || RND.IFD || K_{IFD} || DH.P
			←	
Two-way authentication is now complete[3]				

Table 8-2. Reference device authentication scheme 1 with Diffie-Hellman Key Exchange (Part 3 of 3)

1 If the data is not organized in a file, it may be read from data objects instead.
2 This step is required only if the reader does not have the public key of the CA.
3 The SSC will be updated on both sides according to chapter 8.9

8.5.1.1 Step 1 - Read key exchange parameters

If the IFD does not have the required key exchange information available, it may read them from a global data file EF.DH or data objects respectively. EF.DH may contain public algorithm quantities which depend on the authentication algorithm. For instance, in a Diffie Hellman Key exchange scheme the public key quantities would be the public parameters p, q and g.

Execution Flow

```
READ BINARY( INS = 'B0'        // READ BINARY
             P1 = '85'         // Example: SFI = 05
             P2 = '00'         // offset = 0
             Le = '00')        // all data

Response = data.FID.EF.DH
```

The command and response APDUs are described in ISO/IEC 7816-4, chapter 7.2.3.

The contents of EF.DH are specified in section 15.5 "EF.DH" on page 15-118.

Note 1: Unless the READ BINARY selects the file using a Short File Identifier (SFI), an appropriate SELECT(EF) command must to be executed prior to reading the file. This applies also to the following steps, where appropriate.

Note 2: The public key quantities reveal information about the authentication mechanism. As long as these quantities are used in many ICCs, the identity of an ICC cannot be determined from this information.

8.5.1.2 Step 2 - IFD selects the DH public key parameter set and sends K_{IFD}

In order to establish an early secure session, the IFD generates $K_{IFD} = g^{r_{IFD}} mod\ p$ and sends it to the ICC within a CRT(KAT) of a MSE command.

Execution Flow

```
MSE( INS = '22'                          // MANAGE SECURITY ENVIRONMENT
     P1 = '41'                           // Set for key agreement
     P2 = 'A6'                           // KAT
     Lc = L_A6                           // L_A6 = L_83 + L_91 + 4
     data = '83' L_83 CRT(DH KeyParms)   // Select public DH parms
            '91' L_91 K_IFD              // DO_RND

Response = absent
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

For the structure and coding of the key reference data is specified in section 12.1.4 “CRT KAT for selection of the public DH key parameters” on page 12-92.

8.5.1.3 Step 3 - ICC computes K_{ICC}

The opposite key portion K_{ICC} is returned in the response of a GET DATA command.

Execution Flow

```
GET DATA( INS = 'CA'                     // GET DATA
          P1 = '02'                      // indicate SIMPLE-TLV
          P2 = '4D'                      // header list
          Lc = '04'                      // Length of header list
          data = 'A6 02 91 00'           // header list with KAT template
          Le = '00')                     // get all data

Response = A6 L_A6 91 L_91 K_ICC
```

The ICC generates $K_{ICC} = g^{r_{ICC}} mod\ p$ and sends it to the IFD in the response of the command in the data field of a KAT template as DO.RND.

The command and response APDUs are generally described in section ISO/IEC 7816-4, chapter 7.4.2.

Note: Setting the security environment in the first step implicitly establishes the correct handling of the transmitted random number. The KAT context, indicated in the header list of the GET DATA command selects the correct random number generation algorithm for K_{ICC}.

IFD (after having received the response of the GET DATA command) and ICC (during the execution and the context of the GET DATA command) establish a secure session using key derivation according to 8.8 “Compute session keys from key seed $K_{IFD/ICC}$” on page 8-62. The send sequence counter for MAC computation is set to the value of ‘1’ for the next block sent by the IFD.

8.5.1.4 Step 4 - Skip reading chain certificates

If the public key PK.CA_{IFD}.CS_AUT is available in the ICC, then step 5 and 6 can be skipped.

8.5.1.5 Step 5 - Selection of verification key PK.(R)CA_{IFD}.CS_AUT (conditional)

If PK.CA_{IFD}.CS_AUT is not available in the ICC a chain of (cross-certificates) must be presented to the ICC in order to confirm the validity of each certificate. The chain of certificates ends at any key which is available in the ICC. The top key of the chain, available in the ICC is the public key of the root CA, named PK.RCA.AUT. Steps 5 and 6 have to be performed repeatedly until the correct PK.CA_{IFD}.CS_AUT is available in the ICC. This process is called multi-stage verification.

The MSE command selects the key to be used for verification of the certificate to be received in a consecutive command.

Execution Flow:

```
MSE( INS = ‘22’                 // MANAGE SECURITY ENVIRONMENT
     P1 = ‘81’                  // SET for verification
     P2 = ‘B6’                  // DST
     data=PK.RCA.AUT <or other key in chain>)
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

For the structure and coding of the key reference data is specified in section 12.1.2 “CRT for selection of IFD’s PK.CA_{IFD}.CS_AUT” on page 12-91.

8.5.1.6 Step 6 - Verify Certificate C_CV.CA_{IFD}.CS_AUT (conditional)

In multi-stage verification the certificate of a CA’s public key C_CV.CA_{IFD}.CS_AUT is finally sent to the card with the VERIFY CERTIFICATE command. The ICC verifies this certificate with the public key set in the previous step. If the verification fails, the ICC responds with an appropriate return code. On success, the ICC stores the PK contained in the certificate and allows a further import of certificates along the certificate chain.

The role of the certificate has to comply with its signer. The selection of PK.RCA can always be accepted to verify a chained certificate. Any other key, that was imported with certificate may be applied only, if the role of its certificate matches the purpose of verification of the next incoming certificate.

Execution Flow:

```
PSO( INS = ‘2A’                 // PERFORM SECURITY OPERATION
     P1 = ‘00’                  // response data field absent
     P2 = ‘AE’                  // VERIFY CERTIFICATE
     data= <certificate>)       // any cert in chain
```

The command and response APDUs are described in section ISO/IEC 7816-8, chapter 5.2.

The structure of the certificate is specified in section 14.9 “C_CV.CA.CS-AUT” on page 14-114.

There are two alternatives for the transmission of the certifcate

1. Transmission of the entire certificate including the termplate tag ‘7F 21’ as

described in 14.9 “C_CV.CA.CS-AUT” on page 14-114.

2. Transmission of relevant data objects from the value field of the certificate as described in 14.9 “C_CV.CA.CS-AUT” on page 14-114. These data objects may be transmitted with command chaining.

Note: The execution flow of the VERIFY CERTIFICATE operation is outlined in 14.7.7 “Flowchart for the verification of certificates” on page 14-111.

According to this flowchart the ICC must verify evidence, that the role or the PK of the IFD either corresponds to signing another (subsequently transmitted) certificate or to the authentication mechanism. The role (see 14.7.5 on page 14-109) is part of the certificate when the PK is imported. The role of the last PK imported with VERIFY CERTIFICATE has to be stored in the ICC until the execution of the EXTERNAL AUTHENTICATE command in order to verification.

8.5.1.7 Step 7 - Selection of verification key $PK.CA_{IFD}.AUT$

The MSE command sets the key ID of the CA’s public key to be used for the verification of the IFD’s authentication certificate.

Execution Flow:

```
MSE( INS = ‘22’              // MANAGE SECURITY ENVIRONMENT
     P1 = ‘81’               // SET for verification
     P2 = ‘B6’               // DST
     data=PK.CA_IFD.CS_AUT <or other key in chain>)
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

For the structure and coding of the key reference data is specified in section 12.1.2 “CRT for selection of IFD’s $PK.CA_{IFD}.CS_AUT$” on page 12-91.

8.5.1.8 Step 8 - Verify Certificate C_CV.IFD.AUT

The IFD sends a card verifiable (CV) certificate C_CV.IFD.AUT along with the VERIFY CERTIFICATE command. The ICC verifies the certificate using the public key $PK.CA_{IFD}.CS_AUT$ of the certification authority CA.

The certificate contains the IFD’s PK.IFD.AUT, which is stored temporarily by the ICC

Execution Flow:

```
PSO( INS = ‘2A’               // PERFORM SECURITY OPERATION
     P1 = ‘00’                // response field absent
     P2 = ‘AE’                // VERIFY CERTIFICATE
     data=C_CV.IFD.AUT)       // any cert in chain
```

The command and response APDUs are described in section ISO/IEC 7816-8, chapter 5.2.

The structure and coding of the card verifiable certificate is specified in section 14.8 “C_CV.IFD.AUT” on page 14-113.

There are two alternatives for the transmission of the certifcate

1. Transmission of the entire certificate including the termplate tag ‘7F 21’ as described in 14.8 “C_CV.IFD.AUT” on page 14-113.

2. Transmission of relevant data objects from the value field of the certificate as described in 14.8 “C_CV.IFD.AUT” on page 14-113. These data objects may be transmitted with command chaining.

   The ICC stores the key (PK.IFD.AUT) in a temporary location for use in subsequent steps of the authentication, together with identification data (CAR field) such that the key can be recognized when specified in an MSE:SET APDU.

Note: The execution flow of the VERIFY CERTIFICATE operation is outlined in 14.7.7 “Flowchart for the verification of certificates” on page 14-111.

According to this flowchart the ICC must verify evidence, that the role for the PK of the IFD either corresponds to signing another (subsequently transmitted) certificate or to the authentication mechanism. The role (see 14.7.5 on page 14-109) is part of the certificate when the PK is imported. The role of the last PK imported with VERIFY CERTIFICATE has to be stored in the ICC until the execution of the EXTERNAL AUTHENTICATE command in order to verification.

After Step 8 the IFD’s public authentication key PK.IFD.AUT can be trusted.

8.5.1.9 Step 9 - Key Selection for external authentication

External authentication involves a challenge from the ICC to be signed with the secret authentication key (SK.IFD.AUT) of the IFD. The appropriate verification of this signature in the ICC requires the appropriate public authentication key (PK.IFD.AUT).

The possible existence of different such keys in the ICC requires the selection of the particular key to be used for the verification of the authentication token. This selection is performed with the MANAGE SECURITY ENVIRONMENT (MSE) command

Execution Flow

```
MSE(  INS = ‘22’                // MANAGE SECURITY ENVIRONMENT
      P1 = ’81’                 // SET for external authentication
      P2 = ’A4’,                // AT
      data = CRT.PK.IFD.AUT)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.11.

For the structure and coding of the key reference data refer to 12.1.3 “CRT for selection of IFD’s PK.IFD.AUT” on page 12-92.

8.5.1.10 Step 10 - Get Challenge

In order to proof its authenticity dynamically the IFD shall request a challenge from the ICC. The challenge may consists of a simple random number.

Note: Cryptologically K_{IFD} and K_{ICC} have sufficient random quality. The additional request for RND.ICC is used to initialize the SSC counters (see 9.6 “Send sequence counter (SSC)” on page 9-71).

Execution Flow

```
GET CHALLENGE(  INS = ‘84’        // GET CHALLENGE
                Le = ‘08’)        // get 8 bytes of random

Response = 8 bytes random number
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3.

8.5.1.11 Step 11- External authentication

The IFD computes a signature on the concatenation of the challenge with its own key token information KT.IFD using using SK.IFD.AUT.

After the ICC verified the signature with the trusted public key PK.IFD.AUT, it responds with a confirmation whether the authentication was successful or not.

Execution Flow

```
EXTERNAL AUTHENTICATE(  INS = ‘82’
                        P1 = ’00’     // no algorithm reference
                        P2 = ’00’     // no reference data
                        data = SN.IFD || SIG)
```

with

$SIG = DS^{1}[SK.IFD.AUT]$(’6A’ || PRND2 || h[PRND2 || K_{IFD} || SN.IFD || RND.ICC || K_{ICC} || DH.P || ‘BC’)

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3.

DH.P is the concatenation of the Diffie-Hellman key parameters g, p and q. For the structure and coding of the data field refer to 12.3.1 “EXTERNAL AUTHENTICATE” on page 12-96.

Note: The ICC must verify evidence, that the role for the PK of the IFD corresponds to the external authentication mechanism. The role (see 14.7.5 on page 14-109) is part of the certificate when the PK was imported. The role has to be stored in the ICC temporarily between the execution of the VERIFY CERTIFICATE command and the EXTERNAL AUTHENTICATE command in order to allow this verification.

8.5.1.12 Step 12 - Read $C.CA_{ICC}.AUT$ (conditional)

The IFD obtains $PK.CA_{ICC}.CS_AUT$ by reading $C.CA_{ICC}.CS_AUT$ from a file $EF.C.CA_{ICC}.AUT$ in the ICC. Then it verifies the certificate $C.CA_{ICC}.CS_AUT$ and extracts and stores the public key $PK.CA_{ICC}.CS_AUT$ temporarily for use in step 13.

Execution Flow:

```
READ BINARY( INS = ‘B0’
             P1 = ‘83’          // Example SFI = 03
             P2 = ‘00’          // offset = 0
             Le = ‘00’)         // read all data
```

Response = data.EF.C.CA_{ICC}.CS_AUT

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.2.3..

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

The structure and coding of $C.CA_{ICC}.CS_AUT$ may be as described in section 14.9 “C_CV.CA.CS-AUT” on page 14-114, but it could also be some other format unknown to the ESIGN application. The actual format is application specific and out of the scope of this standard.

8.5.1.13 Step 13 - Read ICC’s certificate C.ICC.AUT

The IFD reads the ICC’s certificate to be used for later authentication.

The received certificate C.ICC.AUT contains the ICC’s public key. The IFD verifies the certificate using the public key of the certification authority $PK.CA_{ICC}.CS_AUT$. If C.ICC.AUT can be verified successfully, the PK.ICC.AUT is stored in the IFD and can be trusted for subsequent authentication.

Execution Flow:

```
READ BINARY( INS = ‘B0’
             P1 = ‘84’          // Example: SFI = 04
             P2 = ‘00’          // offset = 0
             Le = ‘00’)         // get all data

Response = data.EF.C.ICC.AUT
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.2.3..

See section 14.10 “C.ICC.AUT” on page 14-115 for more information concerning the structure and content of this certificate.

8.5.1.14 Step 14 - Key selection

Before processing the INTERNAL AUTHENTICATE command the ICC’s private authentication key SK.ICC.AUT must be set by the MSE command.

The MSE command checks, whether the keys named (qualified with purpose of use and if needed further parameters) are available. Then the keys are set. The following commands check, whether the keys set here can be used

Execution Flow

```
MSE( INS = ‘22’                 // MANAGE SECURITY ENVIRONMENT
     P1 = ’41’                  // SET for internal authentication
     P2 = ’A4’                  // AT
     data = CRT for selection of ICC’s SK.ICC.AUT)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.11.

For the coding of the key reference data refer to 12.1.1 “CRT AT for selection of ICC’s private authentication key” on page 12-91.

8.5.1.15 Step 15 - Internal Authentication

The IFD performs an INTERNAL AUTHENTICATE command. The challenge sent to the ICC with this command is RND.IFD.

The ICC then computes the signature over the challenge and the key token K_{IFD}, K_{ICC} and returns it to the IFD encrypted with secure messaging.

The IFD verifies the response with with the trusted key of the ICC: PK.ICC.AUT and gets evidence that the signer (holder of the certificate) is identical with the entity that made the key negotiation.

Execution Flow

```
INTERNAL AUTHENTICATE(INS = ‘88’, // INTERNAL AUTHENTICATE
         P1 = ‘00’                // no further information in P1,P2
         P2 = ‘00’
         Lc = ‘08’
         data = RND.IFD)
```

$$\text{Response} = \text{SN.ICC} \;||\; \text{DS}^{1}[\text{SK.ICC.AUT}](\text{'6A'} \;||\; \text{PRND} \;||\; \text{h}(K_{ICC} \;||\; \text{SN.ICC} \;||\; RND.IFD \;||\; K_{IFD} \;||\; \text{DH.P}) \;||\; \text{‘BC’})$$

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.2.

The signature is formatted according to ISO/IEC 9796-2 (DS scheme 1)[20].

DH.P is the concatenation of the Diffie-Hellman key parameters g, p and q. For the content of the response data field refer to 12.3.2 “INTERNAL AUTHENTICATE” on page 12-97.

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

8.6 Authentication summary

The above steps involve an ultimate level of security for the device authentication. They contain some mandatory aspects of the device authentication protocols to be considered in order to comply with the claims of the ESIGN-G1 [4] document.

- The public key of the IFD and ICC resp. shall not be used until successfully verified by a certificate.
- The device to be authenticated must submit its identity relevant token to the authenticator in a secure way, typically this can be achieved implicitly, if the identity is part of the certificate.
- For both sides of the authentication it is mandatory to have evidence (after authentication), that the authentication has been made with the same entity, that shares the session keys for the secure messaging.
- Latest after the device authentication, a trusted channel must have been established. Relevant subsequent commands shall use the secure messaging session.
- A display message may indicate the successful device authentication in order to inform the card holder of the presence of a secure channel. The application of the display message is described in 8.11 “Reading the Display Message” on page 8-63.

8.7 Symmetric authentication scheme

The symmetrical authentication protocol may be used where a secret key infrastructure is available. The shared secret key KK is supposed to be already present in the IFD and ICC.

8.7.1 Authentication steps

In the following we use the notation E[K_{ENC}](data) to describe the encryption of 'data' using a TDES key K_{ENC}. The notation MAC[K_{MAC}](data) describes the computation of a MAC over 'data' using a TDES keyK_{MAC}. K_{MAC} and K_{ENC} shall be different and shall be available on the IFD and the ICC side. The establishment/derivation of these keys is not in the scope of this specification.

Table 8-3. Authentication scheme based on symmetrical keys

Stage	Step	IFD	Trans-mission	ICC
1	1	READ BINARY of file EF.SN.ICC or GET DATA respectively	→	Read data from specified file.
			←	SN.ICC as response
	2	GET CHALLENGE	→	
			←	RND.ICC
	3	MUTUAL AUTHENTICATE **S** = RND.IFD || SN.IFD || RND.ICC || SN.ICC || K_{IFD} E[K_{ENC}](**S**) || MAC[K_{MAC}](E[K_{ENC}](**S**))	→	ICC decrypts input and compares *RND.ICC* with the previous response. Verify RND.ICC, SN.ICC. Generate Key K_{ICC} Generate Session Key K_{SK} Generate SSC.ICC **Return:** **R** = RND.ICC || SN.ICC || RND.IFD || SN.IFD || K_{ICC}
		Verify RND.IFD, SN.IFD Generate Session Key K_{SK} Generate SSC.IFD	←	E[K_{ENC}](**R**) || MAC[K_{MAC}](E[K_{ENC}](**R**))
Both sides authenticated and session key seeds available				

K_{ENC} is a TDES key being used in a DES in CBC mode (see 9.7 "Use of TDES" starting on page 9-71. The IV for the CBC-encryption is set to all zero '00000000 00000000'

K_{MAC} is TDES key being used according checksum calculation for secure messaging 9.5.2 “Cryptographic Checksums” on page 9-70, but the initial check block is set to zero '0000000000000000' and the MAC length is 8 bytes.

No padding is required for the encryption on either side because the data is already available as a multiple of 8.

RND.ICC and K_{ICC} are random numbers which are generated by the ICC where RND.IFD and K_{IFD} are random numbers which are generated by the off-card entity. The random numbers RND.ICC and RND.IFD are 8 bytes long.

The random numbers K_{ICC} and K_{IFD} are 32 bytes long each and are used to generate the session keys, since the XOR between K_{IFD} and K_{ICC} is 32 bytes long. SN.IFD and SN.ICC are the 8 least significant bytes of the serial numbers of the off-card entity and the card, respectively.

8.7.1.1 Step A - Read SN.ICC

The IFD requires the ICC’s identity SN.ICC for the mutual authentication token. This parameter can be read in two alternate ways

Execution Flow

```
READ BINARY( INS = ‘B0’          // READ BINARY
             P1 = ’85’           // Example: SFI = 05
             P2 = ’00’           // offset = 0
             Le = ‘00’)          // all data

Response = data.FID.EF.SN.ICC
```

The command and response APDUs are described in ISO/IEC 7816-4, chapter 7.2.3.

Note: Unless the READ BINARY selects the file using a Short File Identifier (SFI), an appropriate SELECT(EF) command must to be executed prior to reading the file. This applies also to the following steps, where appropriate.

Execution Flow (alternate)

```
GET DATA(  INS = ‘CA’            // GET DATA
           P1 = ’02’             // indicate SIMPLE-TLV
           P2 = ’5A’             // Tag
           Le = ‘00’)            // get all data

Response = SN.ICC
```

The command and response APDUs are described in ISO/IEC 7816-4, chapter 7.4.2.

The contents of EF.SN.ICC are specified in section 15.4 “EF.SN.ICC” on page 15-117.

8.7.1.2 Step B - Get Challenge

In order to proof its authenticity dynamically the IFD shall request a challenge from the ICC. The challenge may consists of a simple random number RND.ICC.

Execution Flow

```
GET CHALLENGE(  INS = ‘84’        // GET CHALLENGE
                Le = n)           // get n bytes of random

Response = n bytes random number
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3..

8.7.1.3 Step C - Mutual authentication

The MUTUAL AUTHENTICATE command authenticates ICC and IFD in one command. Therefore the IFD ecnrypts the challenge RND.ICC and its identity SN.ICC and sends it to the ICC together with its own identity SN.IFD and challenge RND.IFD.

The ICC first decrypts the message and verifies its own challenge and identity token. Then it encrypts all the parameters in reverse order and sends them back to the IFD. The IFD verifies the response after decryption.

Execution Flow

```
MUTUAL AUTHENTICATE( INS = ‘82’        // MUTUAL AUTHENTICATE
                     P1 = ‘00’         // no specific information
                     P2 = ‘xx’         // key reference denoting
                                          Kenc and Kmac see note 1 and 2
                     data = E[KENC](S) || MAC[KMAC]( E[KENC](S) )
                     Le = ‘00’)        // get all response bytes

    with S = RND.IFD || SN.IFD || RND.ICC || SN.ICC || KIFD

Response = E[KENC](R) || MAC[KMAC]( E[KENC](R) )

    with R = RND.ICC || SN.ICC || RND.IFD || SN.IFD || KICC
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.5.3.

Note 1: The keyID of the keys K_{enc}/K_{mac} may be also set by using a MSE command prior to MUTUAL AUTHENTICATE.

Note 2: The value of the keyID is either known on application level and/or described in the DF.CIA (see 16 “Cryptographic Information Application” on page 16-122). If ‘xx’ = ‘00’ the ICC knows the key ID implicitly.

8.7.2 Session Key creation

One part of the authentication procedure is the agreement of session keys for building cryptograms and cryptographic checksums with TDES.

In a first step the 32-byte values K_{IFD} and K_{ICC} are exclusivly or-ed.

$$K_{IFD/ICC} = K_{IFD} \oplus K_{ICC}$$

Then the actual session keys are built from $K_{IFD/ICC}$ according to chapter 8.8 .

8.8 Compute session keys from key seed $K_{IFD/ICC}$

Two 16-byte session keys are required for the secure messaging as described in 9 “Secure Messaging” starting on page 9-67.

K_i (ENC) describes the TDES key being used to encrypt and decrypt a cryptogram (CG).

K_i (MAC) describes the TDES key being used to compute and verify a cryptographic checksum (CC).

i = a describes the first 8 bytes of the TDES key

i = b describes the second 8 bytes of the TDES key

Key derivation from the common secret is according to Figure 8-5 (see ANSI X 9.63, Elliptic Curve Key Agreement and Transport Protocols). Let c be a 32 bit counter. Both IFD and ICC calculate.

$$HASH1 = \mathbf{h_{SM}}\,(K_{IFD/ICC} \,||\, c) \qquad \text{with } c=1$$

and

$$HASH2 = \mathbf{h_{SM}}\,(K_{IFD/ICC} \,||\, c) \qquad \text{with } c=2.$$

where the hash function $\mathbf{h_{SM}}$ is defined for the ESIGN application as SHA-1.

Bytes 1..8 of *HASH1* form the key $K_a(\mathrm{ENC})$, and bytes 9..16 build the key $K_b(\mathrm{ENC})$.

Bytes 1..8 of *HASH2* form the key $K_a(\mathrm{MAC})$, and bytes 9..16 build the key $K_b(\mathrm{MAC})$.

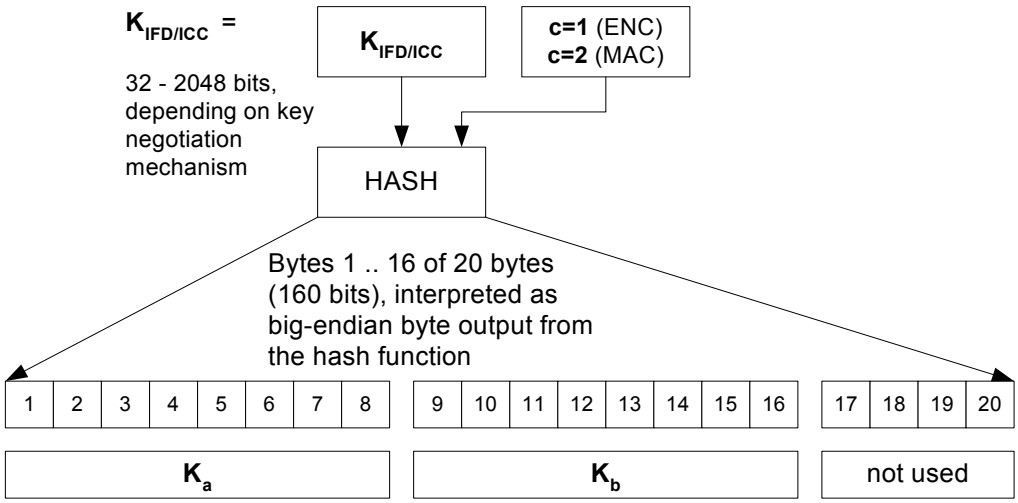


Figure 8-5. Building TDES-session keys from hash output

8.9 Compute send sequence counter SSC

After successful device authentication the start value of the send sequence counter SSC is generated according to 9.6 “Send sequence counter (SSC)” on page 9-71.

8.10 Post-authentication phase

After successful authentication, the session keys for the session are available in IFD and ICC. All further communication is done under Secure Messaging through protection (MAC) or additional encryption of the data being transmitted at the interface. Whether to use encryption or not depends on the actual access conditions of any file or data object to be accessed.

By applying Secure Messaging the format of a plain text message will change according to the definitions in ISO/IEC 7816-4 [11] when it is transmitted with secure messaging.

In the following commands the data in the data field shall be transferred as a cryptogram:

- READ BINARY (display message)
- VERIFY
- CHANGE RD
- RESET RC
- HASH

The session ends

- when another authentication is started
- when the card is depowered or reset
- after a non-recoverable violation of the access rules by an APDU input

8.11 Reading the Display Message

Signatures at a point of untrusted environment require a mutual authentication with session key agreement for secure messaging.

After the mutual authentication has been processed successfully the commands SELECT and READ BINARY must be used to read the display message (display text 8 bytes, ASCII coded), individually for each ICC, which is shown on the display, so that the user also recognizes that the mutual authentication was successful.

The access conditions to EF.DM are described in clause 15.10 “EF.DM” on page 15-121.

Note 1: The user has to be trained to understand the display message method and to associate the appearance of the display message with a valid secure messaging indication. There is no technical means, but the user’s decision whether to abort the transaction if the display message is not shown.

Note 2: The display message method is not resistant against the “Grandmaster Chess Attack” (see A.5 “The “Grandmaster Chess Attack”” on page A-144.

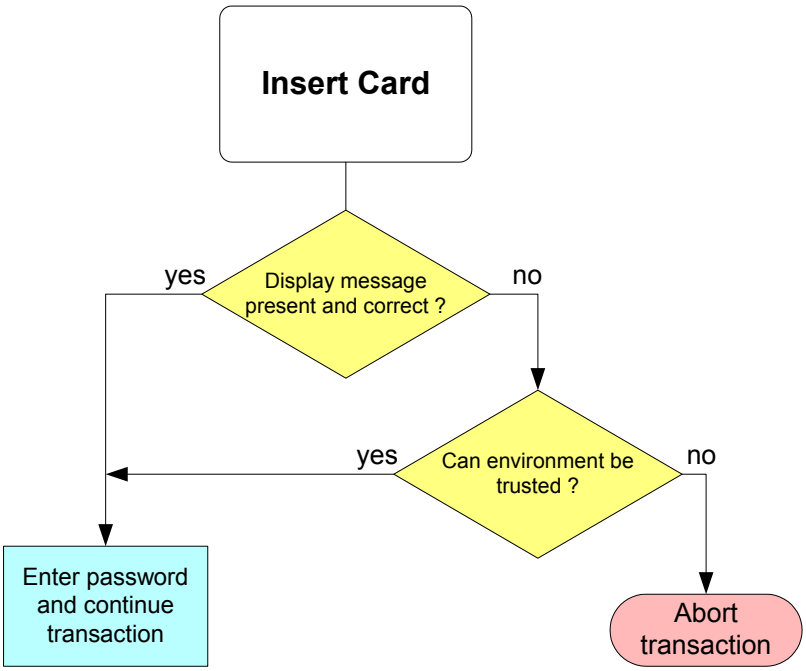


Figure 8-6. User interaction to determine continuation of transaction

The location of EF.DM can be obtained from the information given in CIO.

The commands SELECT and READ BINARY in the SM-Mode are structured as follows

Execution Flow 1 of 2

```
SELECT(  INS = ‘A4’                  // Select command
         P1 = ‘02’                   // Example: Select by FID
         P2 = ‘0C’                   // Example: No FCI information
         data = <see below>)
```

For the general coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.1.1.

T	L	V	
‘81’	‘02’	‘xxxx’	= FID of EF.DM
‘8E’	L	‘xx..xx’	= Cryptographic CheckSum

Table 8-4. Command data field for SELECT of EF.DM

The response uses secure messaging in order to send the status information to the terminal.

T	L	V	
'99'	'02'	'90 00'	= SW1 SW2 status bytes
'8E'	L	'xx..xx'	= Cryptographic CheckSum
-	-	'90 00'	= SW1 SW2 status bytes

Table 8-5. SELECT response

Execution Flow 2 of 2

```
READ BINARY(  INS = 'B0'
              P1 = '00'          // offset = 0
              P2 = '00'
              data = <see below>)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.2.3.

T	L	V	
'97'	'01'	'08'	= Le
'8E'	L	'xx..xx'	= Cryptographic CheckSum

Table 8-6. READ BINARY command to read the Display Message

The response uses secure messaging in order to send the status information to the terminal.

T	L	V	
'87'	'08'	'xx .. xx'	= cryptogram of display message
'8E'	L	'xx..xx'	= Cryptographic CheckSum
-	-	'90 00'	= SW1 SW2 or specific status bytes

Table 8-7. READ BINARY response

Note: The command READ BINARY is also allowed after user authentication on private or company terminals. In this case SM is not applied.

8.12 Updating the Display Message

Changing the Display Message is only allowed after user authentication. It is done via the command UPDATE BINARY.

Execution Flow 1 of 2

```
same as in 8.11
```

Execution Flow 2 of 2

```
UPDATE BINARY(  INS = 'B0'      // UPDATE BINARY
                P1 = '00'       //
                P2 = '00'
                data = <see below>);)
```

For the coding of the command/response APDUs refer to ISO/IEC 7816-4, chapter 7.2.3.

T	L	V	
'87'	L_{87}	'xx .. xx'	= Display message
'8E'	L_{8E}	'xx ... xx'	= CC-DO

Table 8-8. READ BINARY command to read the Display Message

The response APDU uses secure messaging in order to send the status information to the terminal.

T	L	V	
'99'	'02'	'90 00'	= SW1 SW2 status bytes
'8E'	L	'xx ... xx'	= CC-DO
-	-	'90 00'	= SW1 SW2 status bytes

Table 8-9. UPDATE BINARY response

9 Secure Messaging

After successful completion of the device authentication commands and responses are transferred in the SM mode as specified by access conditions. The derived or negotiated symmetric keys will be used to protect integrity and/or confidentiality of the information being transmitted on the interface to the external world or vice versa[1].

Static SM is an another option, using a symmetric key being reserved for secure messaging. In the case of static SM the keys are always available in the card. A key agreement/derivation method is therefore not required.

By application of Secure Messaging the format of a plain text message will change according to the definitions in ISO/IEC 7816-4 [11] when it is transmitted with secure messaging.

9.1 CLA byte

The presence of Secure Messaging is indicated in b3 and b4 of the CLA byte of the command APDU. According to ISO/IEC 7816-4, chapter 6.2.3.1 the bits b3 and b4 are set to 1 indicating that the command header is included in the message authentication

9.2 TLV coding of command and response message

If Secure Messaging is applied the command and response message shall be TLV coded according to ISO/IEC 7816-4.

Table 9-1. SM Data objects

Tag	Meaning
‘81’	Plain value (to be protected by CC)
‘87’	Padding-content indicator byte (‘01’ for ISO-Padding) followed by the cryptogram
‘8E’	Cryptographic checksum (MAC)
‘97’	Le (to be protected by CC)
‘99’	Processing status (SW1-SW2, protected by MAC)

For cryptograms the padding indicator PI is always set to ´01´, i.e. padding acc. to ISO/IEC 7816-4 (´80 ...00´).

Note: The plain value SM DOs are always set to Tag ´81´, because the structure of the data in the data field is irrelevant for the SM view.

1 If not all commands are used with secure messaging, as a consequence the unprotected messages can be forged by an attacker. For compatibility reasons to existing applications the usage of secure messaging for any command cannot be mandated, although it is highly recommended.

The cryptographic checksum shall integrate any secure messaging data object having an odd tag number.

9.3 Treatment of SM-Errors

When the ICC recognizes an SM error while interpreting a command, then the status bytes must be returned without SM. In ISO/IEC 7816-4 the following status bytes are defined to indicate SM errors:

- ´6987´: Expected SM data objects missing
- ´6988´: SM data objects incorrect

Note: Further SM status bytes can occur in application specific contexts.

When the ICC returns status bytes without SM DOs or with an erroneous SM DO the ICC deletes the session keys. As a consequence the secure session is aborted.

9.4 Padding for checksum calculation

The padding mechanism acc. to ISO/IEC 7816-4 [11] (´80 ...00´) is applied.

9.5 Message structure of Secure Messaging APDUs

9.5.1 Cryptograms

Cryptograms are build with TDES in CBC-Mode with the Null vector as Inital Check Block.

A cryptogram (Tag = ‘87’x) is always followed by a cryptographic checksum with Tag = ‘8E’x. Encryption must be done first on the data, followed by the computation of the cryptographic checksum on the encrypted data. This order is in accordance with ISO/IEC 7816-4 [11] and has security implications as described in [39].

The command header shall be included in to the cryptographic checksum.

The actual value of Lc will be modified to Lc’ after application of secure messaging. If required, an appropriate data object may optionally be included into the APDU data part in order to convey the original value of Lc.

The following figure shows an example of secure messaging when using encryption.

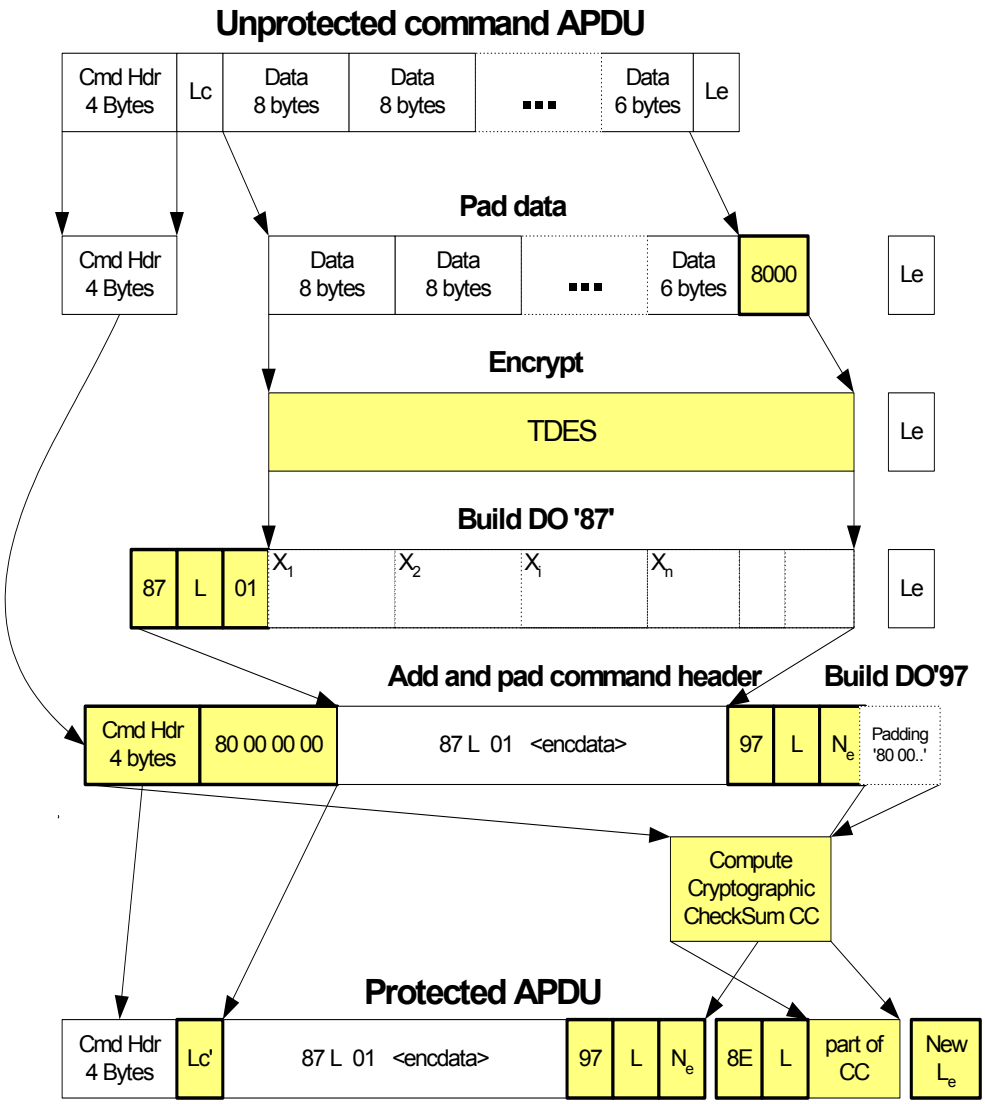


Figure 9-1. Example of computation of a CG and its related CC in a command APDU

Figure 9-1 shows a reference example how an unprotected command APDU is encrypted with the appropriate computation of the related cryptographic checksum. If encryption is not required, the data object ‘87’ is replaced with a plain text data object ‘81’ that conveys the plain data (no padding) in its value field.

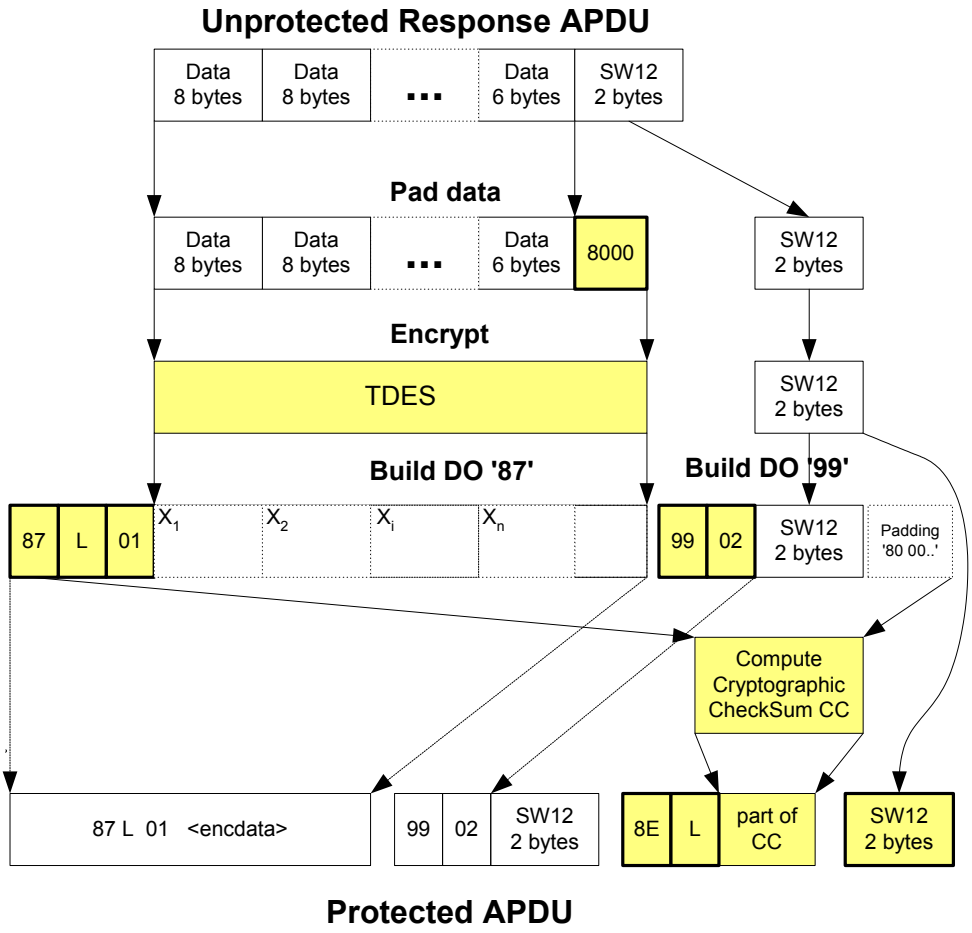


Figure 9-2. Example of computation of a CG and its related CC in a response APDU

Figure 9-2 shows a reference example how an unprotected response APDU is encrypted with the appropriate computation of the related cryptographic checksum. If encryption is not required, the data object ‘87’ is replaced with a plain text data object ‘81’ that conveys the plain data (no padding) in its value field.

Note: Some existing applications transmit the DO ‘99’ (secured SW12) only if no data is present in the response. If the DO ‘99’ is not present in the response, the IFD shall correctly process using the unprotected SW12 at the end of the response. An attacker, however, cannot remove DO ‘99’ from the response because the verification of the CC (MAC) would fail.

9.5.2 Cryptographic Checksums

The data part is split in data blocks with 8-bytes length each. Figure 9-1 and Figure 9-2 indicate the 8-byte subblocks with the notation X_i.

Cryptographic checksums are built acc. to ISO/IEC 7816-4 (chapter 6.2.3.1) as follows (the basic mechanism is to build a MAC according ISO/IEC 9797-1 with the block cipher DES, padding method 2, MAC algorithm 3, MAC length of at least four bytes):

- Initial stage: The initial check block Y0 is E[Ka] (SSC).

- Sequential Stage: The check blocks Y1, .. , Yn are calculated using Ka.
- Final Stage: The cryptographic checksum is calculated from the last check block Yn as follows: E[Ka](D[Kb](Yn)).

Here E[K]( ) means single encryption with DES and key K, respectively D[K]( ) decryption with DES. Figure 9-5 on page 9-72 illustrates this mechanism.

9.6 Send sequence counter (SSC)

The send sequence counter is an 8-byte number that is initialized immediately after successful device authentication.

The send sequence counter SSC must be increased (+1) each time before a MAC is calculated, i.e. if the starting value is x, in the next command the value of SSC is x+1. The SSC value of the first response is then x+2.

The starting value for the SSC is:

```
SSC = RND.ICC(4 least significant bytes) || RND.IFD(4 least significant bytes).
```

The RND.ICC and RND.IFD are taken from the values of the device authentication protocols as described in 8 “Device authentication” on page 8-34.

9.7 Use of TDES

The following figure shows the application of keys in TDES (see also ISO 11568-2).

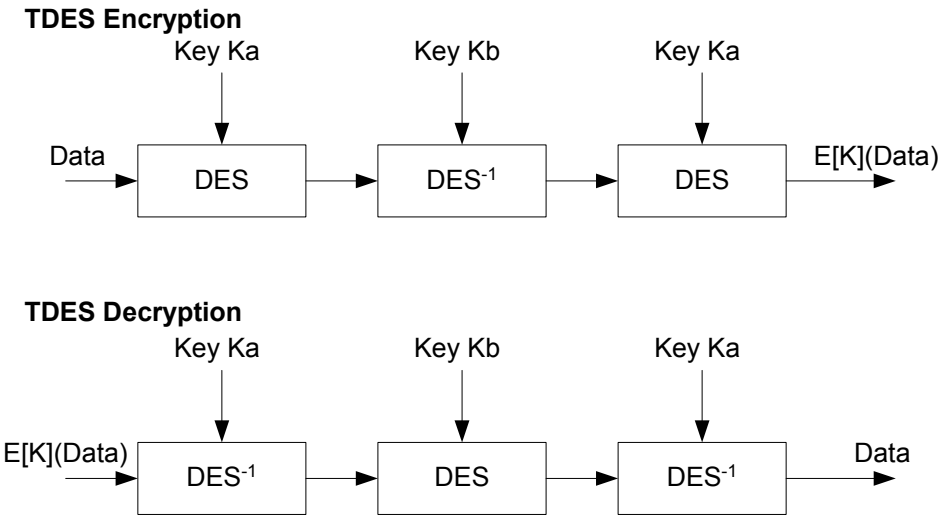
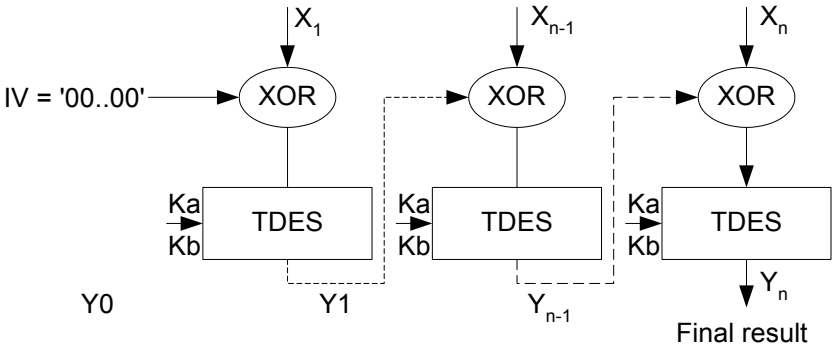


Figure 9-3. TDES Encryption/Decryption



IV = zero initialization vector

'X_1 ||...|| X_n' = plain text (message to encrypt) where each block Xi is 64-bit long

'Y_1 ||...|| Y_n' = resulting cryptogram (encrypted message), where each block Yi is 64-bit long

Figure 9-4. TDES Encryption/Decryption

The encryption is started with the initial value which is set to a zero vector (8 bytes). The IV is xor-ed with the first 8 byte text block of the APDU. The result of this encryption is processed accordingly as shown in Figure 9-4.

The cryptographic checksum (CC) is calculated as retail MAC according to Figure 9-5.

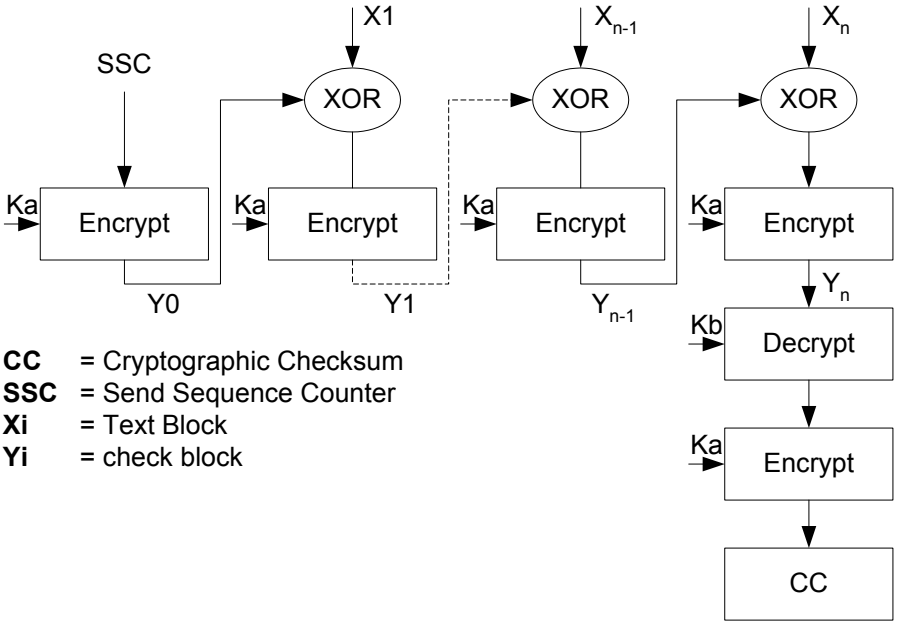


Figure 9-5. Retail MAC computation

The first step performs a single DES encryption with they key Ka on the send sequence counter. The 8-byte result Y0 is xor-ed with the first 8-byte block X1 from the actual data to be protected. Figure 9-1 and Figure 9-2 illustrate how the text blocks Xi are built from the actual APDU data. Then the xor-result is encrypted again with the key Ka.

The second to the last step continue up to the last encryption which results in Yn. Then the final TDES step is performed on Yn.

10 Key Generation

This clause defines command sequences for the key parameter generation/import in the usage phase of the card. Key parameters are a set of public and secret quantities to perform an appropriate algorithm. Key parameter generation/import is required if a new set of key parameters requires to be installed in the ICC e.g. the installation of signature keys.

Note: This chapter describes some alternative methods to establish key parameters in the ICC. Key generation and import are in conformance with the protection profile(s) as described in [6].

Device authentication is mandatory prior to any key generation

After a set of key parameters was generated it requires to be exported (with additional information) to a CA in order to receive a qualified certificate for the generated key. This key exportation shall always use authenticity and integrity in order to guarantee the genuineness and origin of the key and to ensure the conformance to the protection profile as specified in [6].

Three different key export methods are applicable

- Using the ICC's authentication key SK.ICC.AUT
- Using dynamic SM (asymmetric or symmetric techniques)
- Using static SM (symmetric secret key techniques)

  Static SM is another option, using a symmetric key being reserved for secure messaging. In the case of static SM the secure messaging key is always available in the card. A secure messaging key generation method is therefore not required.

Note: There may be national regulations with respect to key length and key attributes.

10.1 Key generation and export using SK.ICC.AUT

For guaranteeing the authenticity, the PK of the generated key pair for digital signature is signed by using SK.ICC.AUT . The authentication key may be implicitly known or explicitely set by a MANAGE SECURITY ENVIRONMENT command.

Key export is allowed only for **public** key parameters.

10.1.1 Selection of authentication key

The ICC's authentication key has to be selected in order to sign the key parameters of step 3.

Execution Flow 1 of 4

```
MSE( INS = '22'       // = MANAGE SECURITY ENVIRONMENT
     P1 = '41'        // = SET for internal authentication
     P2 = 'A4'        // = AT
     data = see below)
```

Table 10-1. MSE SET command data field

T	L	V	
'84'	L	'xx'	= DO KeyRef for SK.ICC.AUT
'80'	L	'xx'	= DO AlgID (optional)

Note: The DO AlgID (see Table 13-1 on page 13-98) is optional

```
Response = empty
```

10.1.2 Key generation

Execution Flow 2 of 4

```
GENERATE AKP(   INS = '46'   // GENERATE ASYMMETRIC KEY PAIR
                P1 = '84'    // Generate key pair, response empty
                P2 = ´xx´    // Key reference of SK.CH.DS
                data = absent or CRT DOs)
```

The optional data field may contain a CRT with information for the SK.CH.DS e.g. a key reference that cannot be expressed in one byte in P2. If the data field contains one or more CRTs associated to the key generation, P2 should be set to '00'. INS = '47' should be used, if the data field contains data objects. (compare ISO FCD 7816-8 [14]).

```
Response : empty
```

Note: The key related information (e.g. keyID) is already present in the ICC. The key generation will only fill the actual key parameters with the generated values.

10.1.3 Export key for certification

In order to provide a certificate, the IFD reads the public key and related information from the ICC. The data is signed by the ICC using its authentication key SK.ICC.AUT.

Execution Flow 3 of 4

```
GENERATE AKP( INS = '46','47'   // GENERATE AKP
              P1 = '83'         // Public key export response data field
                                   according to an extended headerlist
              P2 = 'xx'         // Key reference of SK.CH.DS
              data = absent or CRT DOs
              Le = '00')        // get all

Response : see table below
```

The optional data field may contain a CRT with information for the SK.CH.DS e.g. a key reference that cannot be expressed in one byte in P2. If the data field contains one or more CRTs associated to the key generation, P2 should be set to '00'. INS = '47' should be used, if the data field contains data objects. (compare ISO FCD 7816-8 [14]).

Table 10-2. Data field of GENERATE ASYMMETRIC KEY PAIR response

APDU response data field of GENERATE ASYMMETRIC KEY PAIR						
‘A2’	L_{A2}	Input for the verification of digital cryptographic checksum				
		‘B6’	L_{B6}	DO DST as specified in the following data objects		
				‘83’	L_{83}	‘xxxx’ = DO KEYREF of PK.CH.DS
		‘7F49’	L_{7F49}	‘xxxx’ = DO PK see 11.2 “Public Key parameters” on page 11-86		
		‘9E’	L_{9E}	‘xxxx’ = Digital Signature		

10.2 Key generation and export with dynamic SM

As first step, an authentication procedure has to be performed according to 8 “Device authentication” on page 8-34, in order to produce the SM session keys. Then the GENERATE ASYMMETRIC KEY PAIR command follows.

Execution Flow

```
GENERATE AKP ( INS = ‘46’        // GENERATE ASYMMETRIC KEY PAIR
               P1 = ‘82’         // Generate key pair, response data
                                 // field according to an extended
                                 // headerlist
               P2 = ´xx´         // Key reference of SK.CH.DS
               data = empty or CRT DOs and/or proprietary DOs)

Response : see table below
```

Table 10-3. GENERATE ASYMMETRIC KEY PAIR response

APDU response data field of GENERATE ASYMMETRIC KEY PAIR						
‘A8’	L_{A8}	Input for the verification of digital signature				
		‘B6’	L_{B6}	DO DST as specified in the following data objects		
				‘83’	L_{83}	‘xxxx’ = DO KEYREF of PK.CH.DS
		‘7F49’	L_{7F49}	‘xxxx’ = DO PK see 11.2 “Public Key parameters” on page 11-86		
		‘8E’	L_{8E}	‘xxxx’ = DO CC (Cryptographic Checksum)		

Note: The functionality of this command can be split in two commands as demonstrated in the former clause.

For the public key export see 10.4 “Public Key Export” on page 10-80

10.3 Key parameter import

10.3.1 Key Import using PUT DATA

In this section a concept is described concerning the import of an externally generated signature key (denoted by SK.CH.DS) into an ICC. Therefore the ISO/IEC 7816-4 [11] command PUT DATA is used. This import can be secured optionally by means of a digital signature.

Confidentialy shall be used in secure messaging when the key parameters are sent to the ICC.

The Chinese Remainder Theorem format of a private key is used in modern smart cards and considered in this concept.

10.3.1.1 Private Key Parameters for RSA

Private key parameters are used in order to import an externally generated key. An RSA private key requires the following parameters in Chinese Remainder Threorem format:

Table 10-4. Private Key Data Objects for RSA

T	L	V
'92'	L_{92}	prime p, p < q
'93'	L_{93}	prime q, p < q
'94'	L_{94}	q - 1 mod p
'95'	L_{95}	d mod p-1
'96'	L_{96}	d mod q-1
'97'	L_{97}	public exponent e

10.3.1.2 Private key parameters for other algorithms

Refer to ISO/IEC 7816-8

10.3.1.3 Using Header Lists to Import a Private Key

An extended header list, denoted by tag ‘4D’, is used to import a private key. The tag ‘7F48’ is used to denote a private key object. Using these tags the extended header list for a private key in CRT-format has the following structure:

Table 10-5. Extended header list for private key import

<table>
<tr><td rowspan="9">‘4D’</td><td rowspan="9">L_{4D}</td><td colspan="5">extended header list</td></tr>
<tr><td rowspan="8">‘A8’</td><td rowspan="8">L_{A8}</td><td colspan="4">Input for the verification of a digital signature (optional)</td></tr>
<tr><td rowspan="7">‘7F48’</td><td rowspan="7">L_{7F48}</td><td colspan="3">T-L pair to indicate a private key data object</td></tr>
<tr><td>‘92’</td><td>L_{92}</td><td>T-L pair for parameter p</td></tr>
<tr><td>‘93’</td><td>L_{93}</td><td>T-L pair for parameter q</td></tr>
<tr><td>‘94’</td><td>L_{94}</td><td>T-L pair for parameter q-1 mod p</td></tr>
<tr><td>‘95’</td><td>L_{95}</td><td>T-L pair for parameter d mod p-1</td></tr>
<tr><td>‘96’</td><td>L_{96}</td><td>T-L pair for parameter d mod q-1</td></tr>
<tr><td>‘97’</td><td>L_{97}</td><td>T-L pair for parameter e</td></tr>
<tr><td>‘9E’</td><td>L_{9E}</td><td colspan="3">T-L pair to indicate digital signature (optional)</td></tr>
</table>

In the data structure presented above also filler tags ‘00’ can be used in order to denote bytes in the incoming data stream that shall be read but ignored by the ICC.

10.3.2 Key Import Execution Flow

10.3.2.1 Step 1 (optional)

A digital signature can be used in the extended header list, (see Table 10-5, “Extended header list for private key import,” on page 10-78), in order to protect the input data for change and for non-repudiation services. SM shall be used to protect the import data for evesdropping (Encryption functionality) and to ensure a trusted channel.

The ICC needs the corresponding public key to verify this signature. If this key is not already present in the ICC, the command VERIFY CERTIFICATE is used to transmit this public key to the ICC. The key used to verify the certificate has to be selected by means of an MSE:SET command.

10.3.2.2 Step 2 - Select card holder private signature key (mandatory)

Using the MSE:SET command the private signature key of the cardholder is selected:

Execution Flow

```
MSE ( INS = ‘22’     // MANAGE SECURITY ENVIRONMENT
      P1 = ‘41’      // SET for verification
      P2 = ´B6´      // DST
      data = see table below

Response : data field absent
```

The DST contains at least the following data object:

Table 10-6. DST (only mandatory data objects are presented)

T	L	V
‘84’	L_{84}	Key reference SK.CH.DS (the key to be imported)
‘83’	L_{83}	Key reference to a public key used for signature verification (optional)

10.3.2.3 Step 3 - Send key parameters (mandatory)

Using the command PUT DATA with command chaining according to ISO/IEC 7816-4 the externally generated private key SK.CH.DS is imported. Confidentiality (SM encryption) shall be used to transmit the key parameters

Execution Flow

```
PUT DATA ( INS = ‘DB’    // PUT DATA, odd INS code
           P1 = ‘3F’     // ‘3FFF’ indicates the current DF
           P2 = ´FF´     //
           data = see table below)

Response : data field absent
```

The command data is presented in the following table

Table 10-7. PUT DATA command data (example)

‘4D’	L_{4D}	extended header list						
		‘A8’	L_{A8}	Input for the verification of a digital signature (optional)				
				‘B6’	L_{B6}	T-L pair to indicate a DST		
						‘84’	L_{84}	T-L pair to indicate a key reference to SK.CH.DS
				‘7F48’	L_{7F48}	T-L pair to indicate a private key data object		
						‘92’	L_{92}	T-L pair for parameter p
						‘93’	L_{93}	T-L pair for parameter q
						‘94’	L_{94}	T-L pair for parameter q-1 mod p
						‘95’	L_{95}	T-L pair for parameter d mod p-1
						‘96’	L_{96}	T-L pair for parameter d mod q-1
						‘97’	L_{97}	T-L pair for parameter e
				‘9E’	L_{9E}	T-L pair to indicate digital signature (optional)		
‘5F48’	L_{5F48}	Concatenation of the key parameter data elements according to the extended header list. Data elements that are associated to filler tags’ 00’ in the extended header list are read, but ingored.						
‘9E’	L_{9E}	Digital signature over all DO’s (from ‘4D’ to the end of ‘5F48’).						

10.4 Public Key Export

Two alternatives are available for key export.

- Public Key Export using GENERATE ASYMMETRIC KEY PAIR
- Public Key Export using GET DATA

10.4.1 Key Export using GENERATE ASYMMETRIC KEY PAIR

The key export may can be done with the GENERATE ASYMMETRIC KEY PAIR command. This version is described in 10.2 “Key generation and export with dynamic SM” on page 10-76.

10.4.2 Key Export using GET DATA

In this section a concept is described concerning the export of a public signature key (denoted by PK.CH.DS) that has been generated according to ISO/IEC 7816-8 [14] The authenticity is guaranteed by means of a digital signature using the device authentication secret key (denoted by SK.ICC.AUT).

The data structure to be exported has the following logcal structure.

Table 10-8. DOs for PK coding present in the ICC[1]

‘A8’	L_{A8}	T-L pair to indicate a template for digital signature verification			
		‘B6’	L_{B6}	DO DST as specified in the following data objects	
				‘83’	L_{83}	‘xxxx’ = Key reference to PK.CH.DS
		‘7F49’	L_{7F49}	‘xxxx’ = DO PK see 11.2 “Public Key parameters” on page 11-86	
				‘81’	L_{81}	modulus
				‘82’	L_{82}	public exponent
		‘9E’	L_{9E}	Digital signature (signed are all bytes of the digital signature verification template preceeding tag ‘9E’	

1 The actual physical representation of the logical stucture in Table 10-8 is implementation specific and not in the scope of this specification

In order to retrieve this data structure the GET DATA command according to ISO/IEC 7816-4 [11] is used as described in 10.1.3 “Export key for certification” on page 10-75.

10.4.3 Key Export Execution Flow

The following steps characterize the retrieval of the public key information and the corresponding signature. The steps 2 to 4 do not need to follow the order as proposed in the steps.

10.4.3.1 Step 1: Select Key

With th MSE command, the PK to be retrieved is selected:

Execution Flow 1 of 4

```
MSE ( INS = ‘22’     // MANAGE SECURITY ENVIRONMENT
      P1 = ‘41’      // SET for verification
      P2 = ´B6´      // DST
      data = CRT(DST)

Response : data field absent
```

The command and response APDUs are described in section ISO/IEC 7816-4, chapter 7.5.11..

The structure and coding of the key reference data is specified in section 12.1.5 “CRT for selection key parameters for an imported key” in volume 1 page 12-93.

10.4.3.2 Step 2 - Get modulus

The command GET DATA is used in order to retrieve the DST data object and – dependent on the key length - the first part of the modulus. The remainder of the modulus is read in the next step.

Execution Flow 2 of 4

```
GET DATA (INS = ‘CB’     // GET DATA, odd INS code
          P1 = ‘3F’      // ‘3FFF’ indicates the current DF
          P2 = ´FF´      // see above
          data = see table Table 10-9

Response : see Table 10-10.
```

The command data consists of a concatenation of the extended header list, (see Table 10-9), and the tags denoting the data objects to be retrieved. Then the GET DATA command is used in three steps whereby the data fields shown in the following tables, are

Table 10-9. GET DATA command data : Extended header list for public key export

<table>
<tr><td rowspan="6">‘4D’</td><td rowspan="6">‘0B’</td><td colspan="6">T-L pair to indicate an extended header list</td></tr>
<tr><td rowspan="5">‘A8’</td><td rowspan="5">‘09’</td><td colspan="5">T-L pair to indicate a template for digital signature verification</td></tr>
<tr><td rowspan="2">‘B6’</td><td rowspan="2">‘02’</td><td colspan="3">T-L pair to indicate a DST data object</td></tr>
<tr><td>‘83’</td><td>‘00’</td><td>T-L pair that indicates a public key reference</td></tr>
<tr><td rowspan="2">‘7F49’</td><td rowspan="2">L_{7F49}</td><td colspan="3">T-L pair to indicate a public key data object</td></tr>
<tr><td>‘81’</td><td>L_{81}</td><td>T-L pair to indicate the modulus</td></tr>
</table>

The response data has the following structure

Table 10-10. GET DATA response data, export of modulus

<table>
<tr><td rowspan="5">‘A8’</td><td rowspan="5">L_{A8}</td><td colspan="4">T-L pair to indicate a template for digital signature verification</td></tr>
<tr><td rowspan="2">‘B6’</td><td rowspan="2">L_{B6}</td><td colspan="3">DO DST as specified in the following data objects</td></tr>
<tr><td>‘83’</td><td>L_{83}</td><td>‘xxxx’ = Key reference to PK.CH.DS</td></tr>
<tr><td rowspan="2">‘7F49’</td><td rowspan="2">L_{7F49}</td><td colspan="3">public key</td></tr>
<tr><td>‘81’</td><td>L_{81}</td><td>modulus</td></tr>
</table>

10.4.3.3 Step 3 - Read public exponent

This step is used to read the public key. As presented in step 1 a GET DATA command is used.

Execution Flow 3 of 4

```
GET DATA (INS = 'CB'     // GET DATA, odd INS code
          P1 = '3F'      // '3FFF' indicates the current DF
          P2 = ´FF´      // see above
          data = see table Table 10-11

Response : see Table 10-12.
```

For the structure of the command data please refer to the following table:

Table 10-11. GET DATA command data : Extended header list for public exponent

'4D'	'07'	T-L pair to indicate an extended header list						
		'A8'	'05'	T-L pair to indicate a template for digital signature verification				
				'7F49'	'02'	T-L pair to indicate a public key data object		
						'82'	'00'	T-L pair to indicate the public exponent

The response data has the following structure

Table 10-12. GET DATA response data, export of public exponent

'A8'	L_{A8}	T-L pair to indicate a template for digital signature verification				
		'7F49'	L_{7F49}	public key		
				'82'	L_{82}	public exponent

10.4.3.4 Step 4 - Retrieve digital signature

This step is used in order to retrieve the digital signature data object by means of the GET DATA command.

Execution Flow 4 of 4

```
GET DATA (INS = 'CB'     // GET DATA, odd INS code
          P1 = '3F'      // '3FFF' indicates the current DF
          P2 = 'FF'      // see above
          data = see table Table 10-13

Response : see Table 10-14.
```

For the structure of the command data please refer to the following table:

Table 10-13. GET DATA command data : Extended header list for DS DO

'4D'	'04'	T-L pair to indicate an extended header list				
		'A8'	'02'	T-L pair to indicate a template for digital signature verification		
				'9E'	'00'	T-L pair to indicate digital signature data object

The response data has the following structure

Table 10-14. GET DATA response data, return of digital signature

‘A8’	L_{A8}	T-L pair for digital signature data object		
		‘9E’	L_{9E}	digital signature

10.5 Write certificate

After the public key is certified, the appropriate certificate is sent to the ICC and written into the corresponding EF.

Execution Flow 4 of 4

```
UPDATE BINARY(  INS = ‘D0’      // UPDATE BINARY (file select omitted)
                P1 = ‘xx’       // according to ISO 7816-4
                P2 = ´xx´       // ....
                data = certificate data
                Le = ‘00’       // get all

Response : see table below
```

Notes:

1. For transmision of the DO DSI in a confidential way SM can be applied

2. According to ISO/IEC 7816-8 it is possible instead of retrieving the public key or the extended public key DO, to store the public key in a file to be extracted when appropriate , whereby the corresponding file identifier is implicitly known by the application (or stored in the DF.CIA and the format of the public key may be application specific (e.g. extended public key DO as described here or according to PKCS #7. In this case the GENERATE AKP command 2 is replaced e.g. by a READ BINARY(file identifier).

3. In the case the length of the public key cannot be coded in one byte and the card does not support the extended length field concept according to ISO/IEC 7816-3 it is possible to use the GET RESPONSE command instead to retrieve the remainder part of the public key.

10.6 Setting keys in static secure messaging

Static SM does not require a key exchange as the SM keys have to be available on both sides already. However, the appropriate keys for secure messaging might have to be selected. This happens in schemes where ‘region’ keys are used, i.e. several regions or applications have different SM keys. The ICC, however, has all possible keys and requires a selection prior to secure messaging.

The selection of the SM keys is done by a MSE command as follows.

Execution Flow 1 of 2

```
MSE( INS = ‘22’            // = MANAGE SECURITY ENVIRONMENT
     P1 = ‘41’             // = SET for computation of CC
     P2 = ‘B4’             // = CCT
     data = see below
```

Table 10-15. MSE SET command data field

T	L	V	
‘83’	L	‘xx’	= DO KeyRef for SM key
‘80’	L	‘xx’	= DO AlgID (optional)

Note: The DO AlgID (see Table 13-1 on page 13-98)) is optional, its value determined by the application

```
Response = empty
```

For the description of the MSE command, see ISO/IEC 7816-4, chapter 7.5.11.

Execution Flow 2 of 2

```
MSE( INS = ‘22’            // = MANAGE SECURITY ENVIRONMENT
     P1 = ‘41’             // = SET for computation of CT
     P2 = ‘B8’             // = CT
     data = see below
```

Table 10-16. MSE SET command data field

T	L	V	
‘83’	L	‘xx’	= DO KeyRef for SM key
‘80’	L	‘xx’	= DO AlgID (optional)

Note: The DO AlgID (see Table 13-1 on page 13-98) is optional, its value determined by the application

```
Response = empty
```

For the description of the MSE command, see ISO/IEC 7816-4, chapter 7.5.11.

11 Key identifiers and parameters

A key is a public or secret quantity in the ICC that is used in conjunction with an algorithm to process data with secrecy or integrity. Keys consist of several parameters depending on the algorithm the key is used with. (e.g. an RSA secret key might consist of the parameter *p*, *q*, and the secret exponent *d* the appropriate public key would consist of modulus *N* and the public exponent *e*).

A key is referenced by its KID which can either be a 1-byte number key identifier KID or a key name as it is the case of certificates.

The following chapters describe the key identifiers and the data objects that store the appropriate key parameters of the ESIGN application. The key parameters are important whenever public parameters of a key (like modulus N in the case of a public RSA key) will be read as part of a card verifiable certificate.

11.1 Key identifiers (KID)

There may be one or more key files with private keys present in the ICC. The KIDs of the following private keys are specified.

11.1.1 Secret and private keys

Table 11-1. Key names for private keys

Key Name	Meaning
PIN.CH.DS	digital signature password
PUK.CH.DS	digital signature unblocking code
PIN.CH.ADMIN	administrator password
PUK.CH.ADMIN	administrator unblocking code
SK.CH.DS	digital signature key
SK.ICC.AUT	private authentication key

Note 1: The KID can be choosen individually with respect to 6.1.1.1 “Password reference value (KID)” on page 6-22. The actual values have to be indicated in DF.CIA (see 16 “Cryptographic Information Application” on page 16-122.

Note 2: For the password presentation format refer to 6.1.2 “Presentation formats” on page 6-23.

11.2 Public Key parameters

Public key parameters are used in CV_certificates and during key generation when exporting the public part of a key to the CSP (certificate service provider) .

The public key PK.IFD.AUT (or PK.ICC.AUT) in a CV certificate (refer to 14 “CV_Certificates and Key Management” on page 14-103) consists of a concatenation of parameters. Which PK parameters at what length are present in the CV certificate is described in the CPI field. The CPI field is part of the C_CV certificate, refer to 14.7.2 “CPI-Certificate Profile Identifier” on page 14-108. C_CV certificates are only used in device authentication.

The tags of the parameters are context specific. Their interpretation depends on the algorithm identified by the OID. The values of the parameters have to be coded as an byte string.

11.2.1 RSA public key parameters

An RSA public key requires the following parameters

Table 11-2. Key parameters for an RSA public key

T	L	V
‘81’	L_{81}	modulus N
‘82’	L_{82}	public exponent e

11.2.2 DH key exchange parameters used from DSA

A DSA verify operation requires the following parameters

Table 11-3. Key parameters for DSA verify operation

T	L	V
‘81’	L_{81}	DSA/DH:p
‘82’	‘14’	DSA/DH:q
‘83’	L_{83}	DSA/DH:g
‘84’	L_{84}	DSA:y

11.3 DSA with ECC public key parameters

DSA with ECC is used for signature generation only. The device authentication has not been defined for the use with DSA (ECC). For purpose of key generation the secret and public key parameters have to be computed in the ICC.

11.3.1 Public Key ECC-p

An elliptic curve E over a field F with p elements, p prime (p > 3), is given by the equation

$$Y^2 = X^3 + aX + b \qquad \text{with a, b} \in \text{F} \tag{11-1}$$

with

$$4a^3 + 27b^2 \neq 0 \ mod \ p \tag{11-2}$$

The curve E consists of all points (x,y) x in F, y in F, satisfying the previous equation with a special point O called the point at infinity.

Let k be the smallest number with p $\leq 2^{8k}$. L.F is a number $\geq$ k.

The number *p* and the elements in F are represented as byte sequence of length L.F, whereby the leading (L.F-k) bytes are zero bytes. *A* point *P* on *E* (except the point at infinity) is represented as the concatenation of the sequence of bytes which represents the x-component of *P*, followed by the sequence of bytes which represents the y-component of *P*. The length of this concatenated sequence is equal to 2*L.F

The sequence of bytes representing the y-component of a point *P* could be replaced by a single bit allowing the recovery of this y-component (this is the point -compression notion)., reducing the size from *2*LF* to *LF+1*.

Table 11-4. ECDSA domain parameters

T	L	V
'81'	length.LF	**p** positive odd prime integer
'82'	length.LF	**a** coefficient of curve E, a in F
'83'	length.LF	**b** coefficient of curve E, b in F
'84'	length 2*LF or LF+1	x_a and y_a in F which define a curve point G of order n
'85'	length.LF	**n** a positive prime integer
'86'	length 2*LF or LF+1	x_a and y_a in F which define the public key point Q in E
'87'	L_{87}	cofactor **h** = #E / n (generally 1,2,3 or 4)

An ECDSA key pair is associated with a set of EC domain parameters (P, a, b, n, G, h). The public key Q is a random multiple of the base point $Q = dG$, while the private key is the integer d used to generate the multiple.

11.3.1.1 Public Key ECC-2

An (non-supersingular) elliptic curve E over a field F with 2^m elements is given by the equation

$$Y^2 + X \cdot Y = X^3 + aX^2 + b \qquad \text{with a, b} \in \text{F} \tag{11-3}$$

with $b \neq 0$. The curve E consists of all points (x,y), x in F, y in F, satisfying the previous equation, with a special point *O* called the point at infinity.

Let k be the smallest number with m $\leq$8k. L.F is a number $\geq$ k.

An element in *Z* is represented as the concatenation of a sequence of zero bytes, length (L.F-k), followed by z, represented as bytes sequence of length *k* with respect to a basis as described in ANSI X.9.63. A point *P* on *E* (except the point at infinity) is represented as the concatenation of the sequence of bytes which represents the x-component of *P*, followed by the sequence of bytes which represents the y-component of *P*. The length of this concatenated sequence is equal to 2*L.F.

The number m is represented as byte sequence of length L.m.

The sequence of bytes representing the y-component of a point P could be replaced by a single bit allowing the recovery of this y-component (this is the point -compression notion), reducing the size from 2*LF to LF+1.

Table 11-5. ECDSA domain parameters

T	L	V
'81'	length.LF	2^m, m a positive integer
'82'	length.LF	**a** coefficient of curve E, a in F
'83'	length.LF	**b** coefficient of curve E, b in F
'84'	length 2*LF or LF+1	x_a and y_a in F which define a curve point G of order n
'85'	length.LF	**n** a positive prime integer
'86'	length 2*LF or LF+1	x_a and y_a in F which define the public key point Q in E
'87'	L_{87}	cofactor **h** = #E / n (generally 1,2,3 or 4)

An ECDSA key pair is associated with a set of EC domain parameters (P, a, b, n, G, h). The public key Q is a random multiple of the base point $Q = dG$, while the private key is the integer d used to generate the multiple.

There are many different bases of F (with 2^m elements) over {0,1}, normal bases and polynomial bases.

case a, optimal normal basis:

Tag '82'-'87'	as above
Tag '88'	m (length L.m)

case b, polynomial basis with respect to a trinomial:

Tag '82'-'88'	as in case a
Tag '89'	value **k** of trinomial $X^m + X^k + 1$ (length L.m)

case c, polynomial basis with respect to a pentanomial:

Tag '82' - '88'	as in case a
Tag '89'	value k1 of pentanomial $X^m + X^{k3} + X^{k2} + X^{k1} + 1$ (length L.m)
Tag '8A'	value k2 of pentanomial $X^m + X^{k3} + X^{k2} + X^{k1} + 1$ (length L.m)
Tag '8B'	value k3 of pentanomial $X^m + X^{k3} + X^{k2} + X^{k1} + 1$ (length L.m)

case d, polynomial basis with respect to any polynomial:

Tag '82' - '88'	as in case a
Tag '89'	Give all coefficients

11.4 RSA Diffie-Hellman key exchange parameters

A Diffie-Hellman key exchange protocol requires the following public parameters which are coded as primitive DOs in the ‘7F 49’ PK template.

Table 11-6. Key parameters for Diffie-Hellman key exchange

T	L	V
‘86’	L_{86}	DSA/DH:p
‘87’	20	DSA/DH:q
‘88’	L_{88}	DSA/DH:g

12 APDU data structures

This chapter describes the data structures used in the ESIGN application.These data structures may describe elements being found in files or as part of a command APDU. The appropriate chapters will refer to these descriptions.

12.1 CRTs

The MANAGE SECURITY ENVIRONMENT command (refer to ISO/IEC 7816-4, chapter 7.5.11.) is used in applications to reference keys or to set security environments. In particular the device authentication makes use of this command (refer to 8.4.1.11 “Step K - Get Challenge” on page 8-45 and 8.5.1.7 “Step 7 - Selection of verification key PK.CA_{IFD}.AUT” on page 8-53)

The appropriate key reference information is coded in CRTs (Control Reference Templates). The following paragraphs describe the contents of relevant CRTs.

12.1.1 CRT AT for selection of ICC’s private authentication key

This CRT refers the private key of the ICC and is submitted in the data field of the MANAGE SECURITY ENVIRONMENT command to specify the signing key. The actual CRT tag for authentication ‘A4’ is submitted in the parameter P2 of the command.

Table 12-1. Structure and content of CRT to select ICC’s private authentication key

‘A4’[1]	L_{A4} [2]	CRT for authentication			
		‘83’	L83	‘xx’	= KeyRef of PK.IFD.AUT
		‘84’	L84	‘xx’	= KeyRef of SK.ICC.AUT

1 sent in P2 of the APDU
2 sent in Lc of the APDU

At the same time, the public key PK.IFD AUT is selected in the same command in order to encrypt the signature sent to the IFD.

Typically the public key was sent to the ICC in a preceeding PSO:VERIFY CERTIFICATE command. Then the *KeyRef* can be found in the CHR field (14.7.4 “CHR-Certificate Holder Reference (Subject Key Identifier)” on page 14-109) of the CV certificate that was sent to the ICC prior to the key selection.

12.1.2 CRT for selection of IFD’s PK.CA_{IFD}.CS_AUT

This CRT refers a public key that verifies a certificate in the key chain to the IFD’s authentication key PK.IFD.AUT. It is submitted in the data field of the MANAGE SECURITY ENVIRONMENT. The actual CRT tag for Authentication ‘B6’ is submitted in the parameter P2 of the command.

Table 12-2. Structure and content of CRT for cert signing key selection

‘B6’[1]	L_{B6} [2]	CRT for digital signature			
		‘83’	L_{83}	‘xx’	= *KeyRef* of PK.CA_{IFD}.CS_AUT

1 sent in P2 of the APDU

2 sent in Lc of the APDU

If the key was sent to the ICC in a preceeding PSO:VERIFY CERTIFICATE command, the *KeyRef* can be found in the CHR field (14.7.4 “CHR-Certificate Holder Reference (Subject Key Identifier)” on page 14-109) of the CV certificate that was sent to the ICC prior to the key selection.

If the key is statically stored in the ICC (e.g. a PK.RCA.AUT) the *KeyRef* information may be found in the CIO.PK.CH.AUT in the appropriate EF.PuKD (see Figure 16-2 on page 16-123).

12.1.3 CRT for selection of IFD’s PK.IFD.AUT

This CRT refers a public key that verifies the authentication certificate of the ICC It is submitted in the data field of the MANAGE SECURITY ENVIRONMENT. The actual CRT tag for Authentication ‘B6’ is submitted in the parameter P2 of the command.

Table 12-3. Structure and content of CRT for cert signing key selection

‘B6’[1]	L_{B6} [2]	CRT for digital signature			
		‘83’	L_{83}	‘xx’	= *KeyRef* of PK.IFD.AUT

1 sent in P2 of the APDU
2 sent in Lc of the APDU

12.1.4 CRT KAT for selection of the public DH key parameters

This CRT refers the public key parameters for the key agreement with Diffie-Hellman (DH). The actual CRT tag for DH key agreement ‘A6’ is submitted in the parameter P2 of the command. The data field contains the key reference DO for the selection of the DH public key parameters and the DH key token $K_{IFD} = g^{r_{IFD}} mod\ p$ of the IFD.The key token is actually send in a data object for a random number. This is appropriate as it is actually of random qualitThe key reference for the DH parameters may be retrieved from DF.CIO unless already known..

Table 12-4. Structure and content of CRT to select DH public key parameters

‘A6’[1]	L_{A6} [2]	CRT for key agreement			
		‘83’	L_{83}	‘xx’	= KeyRef of DH key parameters
		‘91’	L_{91}	‘xx’	= $K_{IFD} = g^{r_{IFD}} mod\ p$

1 sent in P2 of the APDU
2 sent in Lc of the APDU

12.1.5 CRT for selection key parameters for an imported key

This CRT refers the parameters of the key to be imported key and is submitted in the data field of the MANAGE SECURITY ENVIRONMENT command

Table 12-5. Structure and content of CRT for an imported key

<table>
<tr><td rowspan="4">‘A4’[1]</td><td rowspan="4">L_{A4} [2]</td><td colspan="4">CRT for authentication</td></tr>
<tr><td>‘83’</td><td>L83</td><td>‘xx’</td><td>= KeyRef of PK.IFD.AUT</td></tr>
<tr><td>‘84’</td><td>L84</td><td>‘xx’</td><td>= KeyRef of SK.ICC.AUT</td></tr>
<tr><td colspan="4">other key parameters that specify the usage of the imported key</td></tr>
</table>

1 sent in P2 of the APDU
2 sent in Lc of the APDU

12.2 Key transport device authentication protocol

12.2.1 EXTERNAL AUTHENTICATE

Command data

The following table shows an example for the transmission of the authentication token in the EXTERNAL AUTHENTICATE command

The signature is formatted according to ISO/IEC 9796-2 (DS scheme 1)[20].

Table 12-6. APDU command data field for EXTERNAL AUTHENTICATE

E[PK.ICC.AUT]**([SIGMIN)** with **SIGMIN** = min (**SIG**, N.IFD - **SIG**) and **SIG** = DS[1][SK.IFD.AUT] (**'6A'** = Padding acc. to ISO 9796-2 (DS scheme2) **PRND2** = 'XX ... XX' random padding bytes generated by IFD. PRND2 has to be such a length that the term '6A' - 'BC' has the same length as the modulus. $\mathbf{K_{IFD}}$ = key transport token **h[**PRND2 || K_{IFD} || RND.ICC || SN.ICC[2] **]** **'BC'** = Padding according to ISO 9796-2 (option 1))	SIGMIN needs to be computed in order to create a unique encryption (see A.2.2 "Using min(SIG, N-SIG) for the signature token" on page A-134) SIG(SIGIN) Signature of length N The block '6A .. BC' forms the DSI (digital signature input) SIGIN. The signing key is SK.IFD.AUT

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

2 Only the 8 least significant bytes are included

Response data

SW12 = 9000, on error an appropriate return code is sent

12.2.2 INTERNAL AUTHENTICATE

Command data

Table 12-7. APDU command data field for INTERNAL AUTHENTICATE command

'xx..xx'	RND.IFD (8 bytes)
'xx..xx'	SN.IFD (least significant 8 bytes)

Response data

The signature is formatted according to ISO/IEC 9796-2 (DS scheme 1)[20].

Table 12-8. APDU response data field for INTERNAL AUTHENTICATE

E[PK.IFD.AUT] (**SIGMIN**) with **SIGMIN** = min (**SIG**, N.ICC - **SIG**) and **SIG** = DS[1][SK.ICC.AUT] (**'6A'** = Padding acc. to ISO 9796-2 (DS scheme 1) **PRND1** = 'XX ... XX' random padding bytes generated by ICC. PRND1 has to be such a length that the term '6A' - 'BC ' has the same length as the modulus. $\mathbf{K_{ICC}}$ = key transport token (see on page 8-61) **h[**PRND1 || K_{ICC} || RND.IFD || SN.IFD[2] **]** **'BC'** = Padding according to ISO 9796-2 (option 1))	SIGMIN needs to be computed in order to create a unique encryption (see A.2.2 "Using min(SIG, N-SIG) for the signature token" on page A-134) SIG(SIGIN) Signature of length N The block '6A .. BC' forms the DSI (digital signature input) SIGIN. The signing key is SK.ICC.AUT

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

2 Only the 8 least significant bytes are included

12.3 Privacy device authentication protocol

12.3.1 EXTERNAL AUTHENTICATE

Command Data

The following table shows an example for the transmission of the authentication token in the EXTERNAL AUTHENTICATE command.

The signature is formatted according to ISO/IEC 9796-2 (DS scheme 1) [20].

Table 12-9. APDU command data field for EXTERNAL AUTHENTICATE

SN.IFD	Identity of IFD
DS[1][SK.IFD.AUT] **(** **'6A'** = Padding according to ISO 9796-2 (DS scheme 1) **PRND2** = 'xx..xx' random padding bytes generated by ICC. PRND2 has to be such a length that the term '6A' - 'BC' has the same length as the modulus. **h[**PRND2 || K_{IFD} || SN.IFD || RND.ICC || K_{ICC} || DH.P] **'BC'** = Padding according to ISO 9796-2 (option 1) **)**	SIG(DSI) Signature of Length N, this example uses 1024 bit modulus. The block '6A .. BC' forms the DSI (digital signature input). The signing key is SK.IFD.AUT

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

DH.P is the concatenation of the Diffie-Hellman key parameters.

DH.P = g || p || q

The parameters are concatenated without any structural information. The length of these parameters is already determined by the used algorithm.

Response data

SW12 = 9000, on error an appropriate return code is sent

12.3.2 INTERNAL AUTHENTICATE

Command data

The INTERNAL AUTHENTICATE command sends a challenge RND.IFD to the ICC[1].

Table 12-10. APDU command data field for INTERNAL AUTHENTICATE command

'xx..xx'	RND.IFD (8 bytes)

Response data

The following table shows an example for the transmission of the authentication token in the INTERNAL AUTHENTICATE command.

The signature is formatted according to ISO/IEC 9796-2 (DS scheme2) [20]

Table 12-11. APDU response data field for INTERNAL AUTHENTICATE

SN.ICC	Identity of ICC
DS[1][SK.ICC.AUT] (**'6A'** = Padding according to ISO 9796-2 (DS scheme 1) **PRND1** = 'xx..xx' random padding bytes generated by ICC **h[**PRND1 || K_{ICC} || SN.ICC || RND.IFD || K_{IFD} || DH.P] **'BC'** = Padding according to ISO 9796-2 (option 1))	SIG(DSI) Signature of Length N, this example uses 1024 bit modulus. The block '6A .. BC' forms the DSI (digital signature input). The signing key is SK.ICC.AUT

1 DS is to be understood as plain RSA exponentiation. It does not imply a layer of input formatting other than already specified here

DH.P is the concatenation of the Diffie-Hellman key parameters.

DH.P = g || p || q

The parameters are concatenated without any structural information. The length of these parameters is already determined by the used algorithm.

1 From a cryptological point of view RND.IFD is not required as K_A has sufficient random quality to act as challenge, however RND.IFD is sent to provide maximum symmetry between the device authentication protocols.

13 AlgIDs, Hash- and DSI Formats

This chapter describes parameters that are used in different functions, e.g. device authentication and signature creation.

13.1 Algorithm Identifiers and OIDs

In general, algorithms are defined by object identifiers. Object identifiers are standardized values presented in a certificate. (Refer to 14.8 “C_CV.IFD.AUT” on page 14-113).

An object identifier, however, cannot be interpreted by the ICC, i.e. they are mainly used to inform the external world about the use of an algorithm. Instead, the ICC uses algorithm IDs in order to specify a particular algorithm locally. Hence with a known object identifier, the IFD may use an algorithm ID in order to tell the ICC which algorithm to use. Unless an algorithm is implicitly Hdefined or already specified through a key selection, it has to be supplied to the ICC with a MSE command (SET).

The following table specifies the algorithm ID for the use with the ESIGN application. In general, national algorithm or European catalogs will have to be applied to signature cards.

Table 13-1 shows the AlgIDs relevant for use with the ESIGN application for the digitial signature function

Table 13-1. AlgIDs for the use in the digital signature function

AlgID	Meaning
Algorithm ID’s for the digital signature function	
‘0x’	No indication of a hash-function, the hash function is defined implicitly or not applicable.
‘1x’	SHA-1, OID = {1 3 14 3 2 26 }
‘2x’	RIPEMD-160, OID = { 1 3 36 3 2 1 }
‘x1’	RSA with DSI acc. to ISO/IEC 9796-2 (DS scheme 1)
‘x2’	RSA with DSI acc. to PKCS #1, (parameter DigestInfo, if x = 0)
‘x4’	DSA with ELC (parameter hash value, if x = 0)

Table 13-2 shows the AlgIds relevant for the use in the device authentication as described in 8 “Device authentication” on page 8-34.

Table 13-2. AlgIDs for the use in ESIGN device authentication

AlgID	Meaning
‘17’	RSA with SHA-1 session key exchange according to 8.4 “Key transport protocol” on page 8-39 DSI acc. to ISO/IEC 9796-2 (DS scheme 1) Session key derivation with SHA-1
‘1B’	RSA with SHA-1 DH session key agreement according to 8.5 “Device authentication with privacy protection” on page 8-48 DSI acc. to ISO/IEC 9796-2 (DS scheme 1) Session key derivation with SHA-1
‘1C’	Symmetric session key transport protocol Session key exchange according to 8.7 “Symmetric authentication scheme” on page 8-59 Session key derivation with SHA-1
‘1D’	RSA with SHA-1 session key exchange according to 8.4 “Key transport protocol” on page 8-39 Session key derivation as specified in the note of 8.4.2 on page 8-46. DSI acc. to ISO/IEC 9796-2 (DS scheme 1)

13.2 Hash Input-Formats

When hashing is performed partly inside and partly outside the card, the data field of the command PSO: HASH for the hash algorithms SHA-1 and RIPEMD-160 should be structured as follows:

13.2.1 PSO:HASH without command chaining

Table 13-3. data field for PSO:HASH function without command chaining

T	L	V
‘90’	‘1C’	20 Bytes intermediate hash value Counter of 8 bytes (number of bits already hashed)
‘80’	‘xx’	text still to be hashed without padding (length of text up to 64 bytes)

Note: If the P2 parameter of the PSO:HASH command is 0x80, then the data can also be presented as a plain value (without TLV-coding).

13.2.2 PSO:HASH with command Chaining

The following table show the content of the first command's data field of the PSO:HASH function with command chaining.

Table 13-4. Data field of first command of PSO:HASH function with command chaining

T	L	V
'90'	'1C'	20 Bytes intermediate hash value Counter of 8 bytes (number of bits already hashed)

The following table show the content of the second to (n-1)th first command's data field of the PSO:HASH function with command chaining.

Table 13-5. Data field of 2nd to (n-1)th command of PSO:HASH function with command chaining

T	L	V
'80'	'40'	text to be hashed

Table 13-6. Data field of last command of PSO:HASH function with command chaining

T	L	V
'80'	'xx'	last textblock without padding

Note: If the P2 parameter of the PSO:HASH command is 0x80, then the data can also be presented as a plain value (without TLV-coding).

13.3 Formats of the Digital Signature Input (DSI)

In the following text ESIGN-G conform DSI formats of the digital signature are described for different signature algorithms. It is mandatory to support at least one of these formats, i.e. if only the RSA algorithm is available in a card, then the card must be able to compute a signature on the basis of the ESIGN-G formats for RSA. Optionally the card may be able to support other signature formats.

The given DSI formats describe the string, that is used directly by the signature algorithm to compute the digital signature.

13.3.1 DSI according to ISO/IEC 9796-2 (DS scheme 1)

The DSI format based on ISO/IEC 9796-2 (DS scheme 1) and integrating a random number has the following structure (this example assumes a 1024 bit modulus):

Table 13-7. Example: Digital Signature Input (DSI) - Format acc. to ISO/IEC 9796-2

T	L	V
-	-	'**60**' = flags as of ISO 9796-2 (DS scheme 1) '00..00 01' = Padding according to ISO 9796 (DS scheme 1) 0-bits with last bit = '1'. The padding has to be such a length that the term '60' - 'BC' has the same length as the modulus. RandomNo = random number inserted by the card (8 bytes) Hash field = 'XX ...XX' '**BC'** = Padding according to ISO 9796 (option 1) The block '60 .. BC' forms the DSI (digital signature input)

Contrary to the original algorithm ISO/IEC 9796-2 (DS scheme 1) the random number as internal message is not integrated into the calculation of the hash value. Also a recoverable string (see ISO/IEC 9796-2 (DS scheme 1) is not produced, i.e the intermediate string is used directly as DSI.

13.3.2 DSI according to PKCS #1 V 2.x

The DSI format according to PKCS #1 V 2.1 has the following structure

Table 13-8. Digital Signature Input (DSI) - Format acc. to PKCS #1 V2.1 (Chapter 9.2.1 "EMSA-PKCS1-v1_5")

T	**L**	**V** (Example with 1024-bit modulus)

-	-	'00'= Start byte
		'01' = Block type
		'FF..FF'= Padding String PS (minimum of 8 bytes)
		'00' = Separator
		DigestInfo = ASN.1-Sequence of OID and parameter) and digest (ASN.1-DO has value)
		{
		SHA1 : 30 21 30 09 06 05 2b 0e 03 02 1a 05 00 04 14
		RIPEMD-160 : 30 21 30 08 06 05 2B 24 03 02 01 04 14
		}
		
		'xx..xx' = Hash

13.3.3 DSA with DH key parameters

The DSI consists of the hash value calculated using SHA-1 or RIPEMD-160.

13.3.4 DSA with Elliptic Curves

The DSI consists of the hash value which was calculated using SHA-1 or RIPEMD-160. If the DSI is longer than the hash value (e.g. if q is longer than 160 bits), then the DSI should be filled with leading zero bits.

14 CV_Certificates and Key Management

The following chapters describe structure and usage of card verifiable certificates. Card verifiable certificates are a mandatory requirement for the device authentication (refer to 8 “Device authentication” on page 8-34.

However, there are other implications (see next chapter) which might require card verifiable certificates either read from the card or sent to the card for verification and key transport.

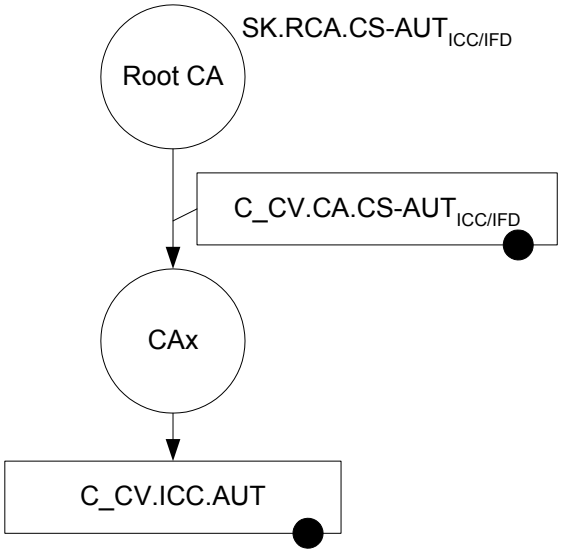
The last section of this chapter describes the mandatory card verifiable certificates to be supported by the ICC in order to apply device authentication.

14.1 Level of trust in a certificate

The existence of a valid certificate gives the ICC evidence for a trusted partner, represented through the IFD. If a more granular specification of trust is required, additional schemes require to be applied. e.g. the comparison of a CHA field with a reference in the ICC. The definition of these additional methods is out of the scope of this standard

14.2 CV Certificates in the ICC

For particular application purposes card verifiable certificates might be required.



The Card verifiable Certificate for the ICC will be produced by a CA, e.g. a Card manufacturer.

The public key of the related CA is certified by a Root CA in a CV certificate.

Figure 14-1. CAs and CV certificates

14.3 The role of the CA

The CA (certification authority) issues certificates for the public key of an entity.

For the public key of a signature key pair, the CA is an institution that is entitled to issue certificates, according to the specifications in ESIGN-G1 [2].

For the public key of a device authentication key pair, the role of the CA is typically represented by the card manufacturer (CM) or the card issuer (CI). This could also be the case for the 'root CA' as referenced below.

The following chapters do not distinguish these different entities. They will generally refer to CA as the authority who created the appropriate certificate.

14.4 CV_Certificates and key management

Device authentication may be performed with one stage authentication or multiple stage authentication.

The following figures show the key management, when one step and two stage procedure of certification verification applies. The treatment of other data elements, which are present in the certificate (i.e. CHA), is omitted in these figures.

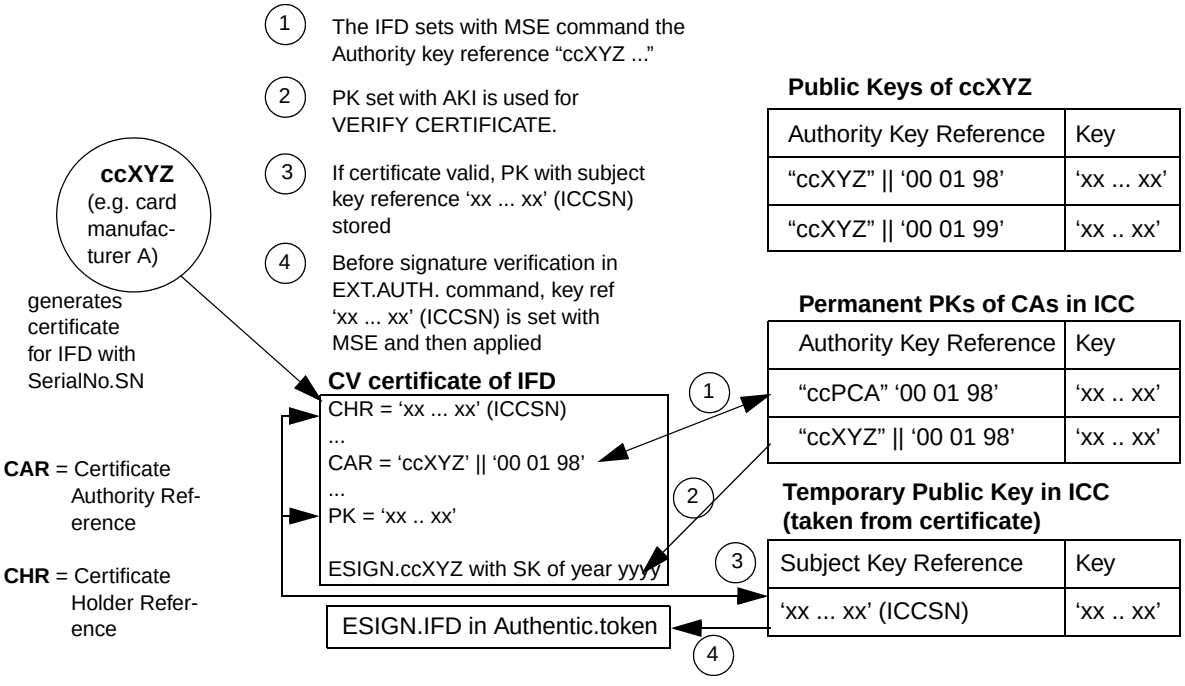


Figure 14-2. One Stage Procedure (Precondition: PK from ccXYZ present in the ICC)

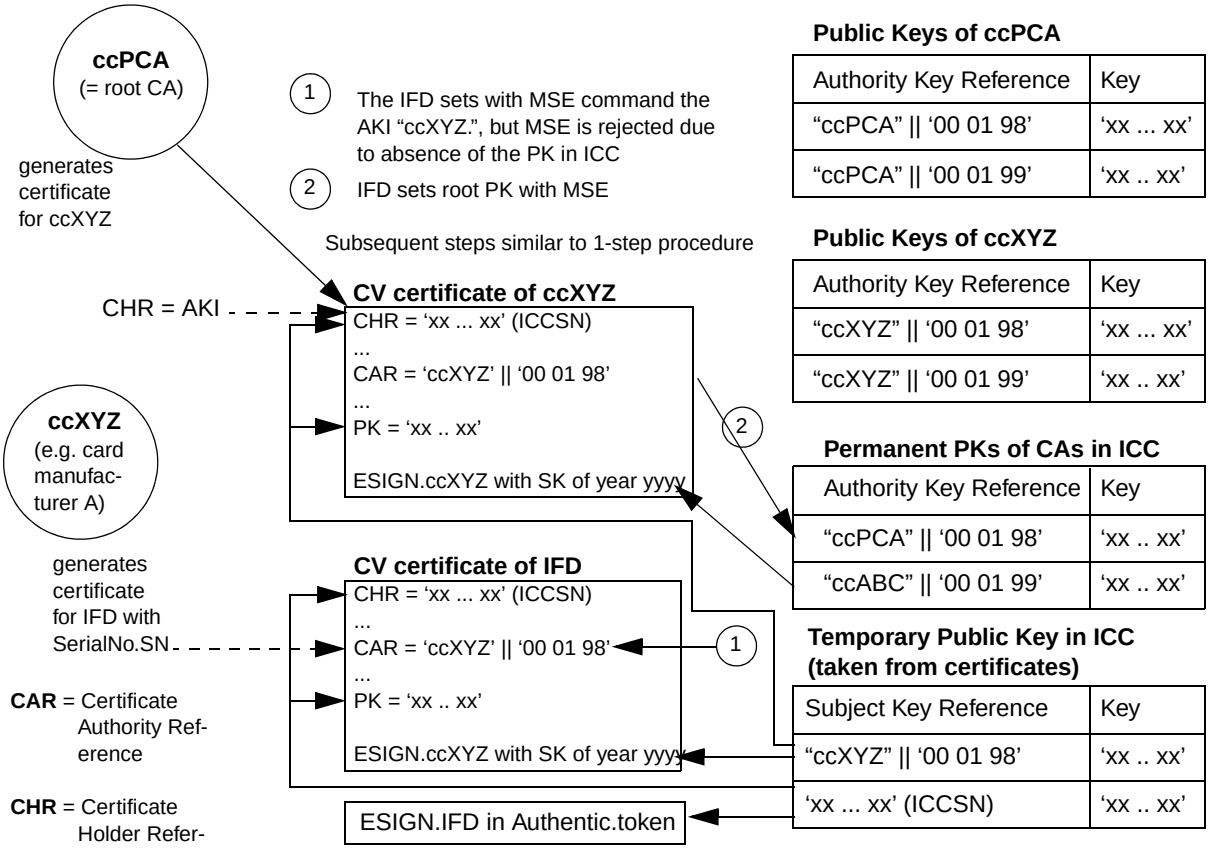


Figure 14-3. Two Stage Procedure (custormer terminal has certificate from ccXYZ and not from ccABC)

14.5 Signature-Certificates

Signature certificates are provided in transparent files (see 15 “Files” on page 15-116). The verification of a DS certificate is carried out by the recipient of a signed document. Security software is usually used for this. The structure of a DS certificate (see interoperability specification for signature certificates) is not interpreted to the card.

14.6 Authentication Certificates

Authentication certificates are provided in transparent files (see 15 “Files” on page 15-116). The verification of a DS certificate is carried out by the IFD (refer to 8 “Device authentication” on page 8-34) or by a server in case of a client/server authentication.

The following paragraphs only describe the structures and content of card verifiable certificates as used during device authentication.

The evaluation of some fields is optional and may be done in addition to those fields which are required to be checked during device authentication with the VERIFY CERTIFICATE command.

14.7 Certificate Data Elements

The following table shows data elements relevant to CV certificates (Card Verifiable Certificates) and thus for terminal and ICC authentication certificates. The tags are assigned by different ISO standards. Data structures using the data objects of Table 14-1 do not necessarily have to include the tag and length bytes, when coding the value part. However, in this case, it can be appropriate to provide a header list to be transmitted together with the appropriate structure in order to allow the receiver a correct interpretation of the structure, if this is not known a priori.

Table 14-1. Interindustry DOs for CV Certificates

Tag	Data element	Defined in
‘7F21’	CV certificate, constructed, e.g. cardholder certificate	ISO/IEC 7816-6
‘5F4E’	Certificate content	ISO/IEC 7816-8
‘5F29’	Interchange profile descriptor, e.g. Certificate profile Identifier(**CPI**) (see 14.7.2)	ISO/IEC 7816-6
‘42’	Certification authority reference (**CAR**), e.g. name or ID of issuer autority (see 14.7.3)	ISO/IEC 7816-6/8
‘5F20’	Certificate holder reference (**CHR**), e.g. cardholder name or SN.ICC (see 14.7.4)	ISO/IEC 7816-6/8
‘5F49’	Certificate holder public key, e.g. **PK.IFD** (primitive DO)	ISO/IEC 7816-6/8
‘7F49’	Certificate holder public key, e.g. **PK.IFD** (constructed DO)	ISO/IEC 7816-8 (proposed)
‘06’	Object Identifier (**OID**) for signature algorithm of certificate holder (see 14.7.8)	ISO/IEC 7816-6

Table 14-1. Interindustry DOs for CV Certificates

Tag	Data element	Defined in
'5F4A'	Public Key of CA or its reference (Authority Key Identifier **AKI**)	ISO/IEC 7816-6
'5F4C'[1]	Certificate holder authorisation (**CHA**)	ISO/IEC 7816-9
'5F37'	Signature of a certificate, produced by the related CA	ISO/IEC 7816-6/8
'5F38'	PK-Remainder	ISO/IEC 7816-6

1 ISO7816-6 has assigned this tag for the certificate holder authorisation due to a collision of two data objects with the same tag (see [13]).

In application specific contexts further data elements may be present, such as

Expiry Date
: The expiry date delimits the period in which the public key is bound to the key holder (cannot be checked in a ICC at the moment).

Validity Label
: The validity label denotes the actual status of a valid certificate by means of a number. The use of these data elements will be described in a later version of this specification.

14.7.1 Service Indicator

A service indicator is optionally used in order to specify a service a particular key is allowed to be used. The service indicator is submitted in the CAR-field (14.7.3) of a C_CV certificate. The C_CV certificates described in this standard are used for device authentication purposes only.

Table 14-2. Values for Service Indicator

Value	Service
'0'	Digital Signature
'1'	Entity Authentication (IFD/ICC)
'2'	Key Encipherment
'3'	Data Encipherment
'4'	Key Agreement
'5'	Entity Authentication (C/S)
'6'	CertSign (no service indicated) = signing a certificate
'7'	CertSign for Authentication and Key Encipherment
'8'	CertSign for Authentication and Key Agreement
Other values RFU	

14.7.2 CPI-Certificate Profile Identifier

The certificate profile identifier (CPI) delineates the exact structure of an authentication certificate. It can be used as a card internal identifier of the relevant headerlist. Thus several headerlists can be supported and the matching headerlist to an incoming certificate can be found dynamically. A certificate headerlist describes the concatenation of DEs within one certificate. The coding scheme for CPI is shown below

Table 14-3. CPI coding according to DIN 66291-1

CPI coding	Meaning
'00'	RFU
'01' - '7E'	CPIs standardized in this specification (14.7.8 "Coding of Certificates" on page 14-112)
'80' - 'FE'	Application specific CPIs
'FF'	Escape, subsequent bytes contain the CPI

Possible values for the CPI are given in 14.7.8 "Coding of Certificates" on page 14-112.

14.7.3 CAR-Certification Authority Reference (Authority Key Identifier)

The "Certification Authority Reference" (CAR) has the purpose of identifying the certificate issuing CA in such a way that the DE can be used at the same time as an authority key identifier. The CAR consists of

- the CA name, i.e. the country code according to ISO 3166 (2 Bytes) followed by an acronym of the CA (3 Bytes, ASCII characters and
- an extension for key referencing (3 Bytes).

Table 14-4. Structure of the Certification Authority Reference (Autority Key Identifier)

CA Name (5 Bytes)	Extension for key referencing 3 Bytes

The extension has the following structure:

Table 14-5. Structure of the extension for key referencing

Service Indicator (1 BCD)	Discretionary Data binary coded (1 BCD)	Algorithm reference (2 BCD)	Date (last two digits of key generation year) (2 BCD)

The Service Indicator (Key usage) has the value 1 = entity authentication (see also Table 14.7.1 on page 14-107).

The Discretionary Data may have a value at the discretion of the related CA.

The algorithm Reference/Id can be individually assigned by a CA for distinguishing different PK algorithms.

The date consists of the last two digits of the year, in which the key pair for certificate signing was produced. If in one year more than one keypair has been generated for the same algorithm, the keypairs may be distinguished by using the discretionary data field.

When a certificate is presented, the CAR field is temporarily stored in order to reference the certificate's public key in later step. (see 14.7.7).

14.7.4 CHR-Certificate Holder Reference (Subject Key Identifier)

The "Certificate Holder Reference" (CHR) is used to denote the certificate holder uniquely in such a way that the DE can be used as an subject key identifier to reference the PK of the certificate holder. The field has a fixed length of 12 bytes. The value depends on the entity for which the certificate is issued. The CHR consists of

CV-certificate for CS

Filler (4 bytes) || CAR (8 Bytes)

CV-certificate for ICC and IFD

Filler (0-4 Bytes) || SN.ICC (8-12 Bytes)

Table 14-6. Structure of the Certificate Holder Reference, if Certificate Holder is a CA

Filler (4 Bytes)	CA Name (5 Bytes)	Extension for key referencing (3 Bytes)

A filler-byte is coded ´00´.

The "Extension for key referencing" has the same structure as shown in table Table 14-5. The field "date" contains the last two digits of that year, in which the PK certified in the CV-certificate (PK.CA.CS_AUT) was issued.

Table 14-7. Structure of Certificate holder Reference, if Certificate Holder is a ICC

Filler pads entire CHR to 12 bytes (0..4 Bytes)	SN.ICC or SN.IFD (8-12 Byte)

During verification of the CV certificate the PK is stored temporarily, whereby the unique certificate holder reference is used as key reference (for the certified PK) in the MSE command.

14.7.5 CHA-Certificate Holder Authorisation (CHA)

The "Certificate Holder Authorisation" (CHA) describes access rights of the certificate holder. It is a general DE in a CV certificate, which is be used to identify (i.e. access-) rights of the certificate holder. The meaning of CHA can be compared with a role based key identifier when applying symmetrical authentication algorithms.

The CHA-DE consists of the application reference/ID for which the authorisation is valid and the role identifier.

Table 14-8. Structure of Certificate Holder Authorisation for the ESIGN application

Prefix	**Value**
AID of related DF (6 most significant bytes)	Role Identifier

14.7.6 Role identifier

C_CV certificates for device authentication are not created by a legacy CA, but by the card manufacturer (CM) or by the card issuer (CI). The specification of a role identifier of a authentication certificate chain is specified by one of these instances.

If no specific role is assigned within a certificate chain, the following notion of roles shall be applied

Table 14-9. Role Id specification

Role ID	Meaning	Used in
'21'	Role of 'RCA_{IFD}', may sign cross certificates for IFD	XC_CV.CA_{IFD}.CS_AUT
'22'	Role of CA_{IFD}.CS, may sign certificates in a certificate chain for IFDs	C_CV.CA_{IFD}.CS_AUT
'01'	Role of IFD, key may be used for authentication	C.IFD.AUT
'31'	Role of 'RCA_{ICC}', may sign cross certificates for ICC	XC_CV.CA_{ICC}.CS_AUT
'32'	Role of CA_{ICC}.CS, may sign certificates in a certificate chain for ICCs	C_CV.CA_{ICC}.CS_AUT
'00'	Role of ICC, key may be used for authentication	C.ICC.AUT

When a device (IFD/ICC) a certificate, it verifies that the public key, used for the verification of the certificate, is associated with a role ID, that allows the verification of the certificate. If the verification is successful, then the device stores the public key of the certificate together with its KeyID and role ID. In general, a VERIFY CERTIFICATE command shall provide evidence, that a certificate to be verified has been signed by a private key, that is associated with a role, that allows this (certificate) signature.

A certificate chain may use a notion of a role ID, which is different from the one proposed. Then the VERIFY CERTIFICATE command needs to recognize the role correctly.

On performing EXTERNAL AUTHENTICATE the role of the IFD's PKshall be verified to correspond to authentication. The ICC stores the role id temporarily from the key import until is usage in EXTERNAL AUTHENTICATE.

14.7.7 Flowchart for the verification of certificates

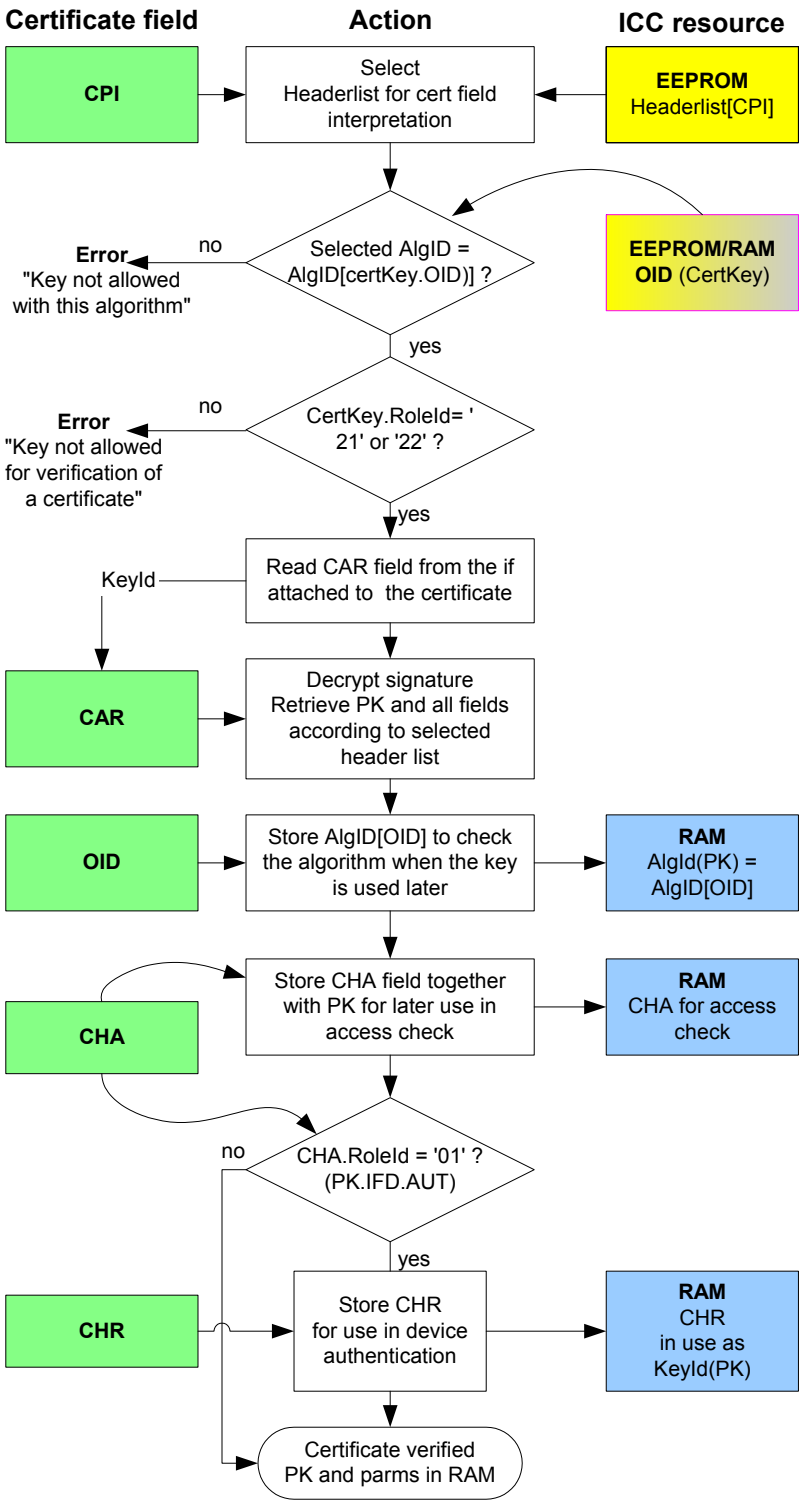


Figure 14-4. Execution flow of certificate verification (example)

Figure 14-4 shows the typical execution flow of a PSO:VERIFY_CERTIFICATE command. Prior to this execution, it is assumed that the appopriate certifying key (CertKey) has already been selected with a MANAGE SECURITY ENVIRONMENT command.

The notation of EEPROM includes the definition of any persistant storage like ROM, Flash EEPROM etc. The figure also demonstrates the usage of the particular fields of a card verifiable certificate.

As the figure is an example for the delivery of a public key in a key chain of certificates for device authentication, it uses some of the key names related to device authentication. The general idea of the execution flow, however, may be used as a reference for the execution of any card verifiable certificate.

14.7.8 Coding of Certificates

The following table shows the defined CPIs with the corresponding certificate coding. It should be noted, that the OID in the certificate of the CA identifies the signature algorithm, whereas the OID in the certificates C.ICC or C.IFD identifies the authentication procedure including key transport or key agreement.

Table 14-10. CPI values and CV field values

CPI	CAR	CHR	CHA	OID*	PK	Remark
'03'	...	...	AID || '21' AID || '22' AID || '31' AID || '32'	'2B 24 03 04 02 02 01'	Modulus (128 Bytes) || Exponent (4 Bytes)	C.CA: RSA, 1024 bit, SHA-1, 9796-2
'04'			AID || '00' AID || '01'	'2B 24 07 02 01 01' '2B 24 07 02 01 01'	as immediately above	C.ICC: RSA, 1024 bit, SHA-1, 9796-2 C.IFD: RSA, 1024 bit, SHA-1, 9796-2
Values will be added in a further maintenance document						privacy protocol acc to 8.5 "Device authentication with privacy protection" on page 8-48
						other key length and algortihms

14.8 C_CV.IFD.AUT

C_CV.IFD.AUT codes the IFD's authentication certificate issued by certification authority It is used in order to deliver the public key of its appropriate parameters to the ICC. The certificate is card verifiable and is delivered as signature with message recovery.

Table 14-11. Structure and content of C_CV.IFD.AUT

'7F 21'	**'81 CD'**	C_CV Certificate		
		'5F 37'	**'81 80'**	Signature of Length N, this example uses 1024 bit modulus. **'6A'** = Padding according to ISO 9796-2 (scheme 1) **CPI** = '04' **CAR** = Defined by Card Manufacturer **CHR** = Certificate Holder Reference **CHA** = AID[1..6] || Role Identifier **OID** = '2B 24 07 02 01 01' **PK_part1** = 'xx..xx' (MSB ⇒LSB) **Hash** = 'xx..xx' **'BC** The block '6A .. BC' forms the DSI (digital signature input) The fields above describe the DSI, however the actually stored bytes represent the actual signature with the same length
		'5F 38'	**'3C'**	**PK_part2** = 'xx..xx' (MSB ⇒LSB) **PK_exp** = '00 01 00 01'
		'42'	**'08'**	**CAR** see 14.7.3 on page 14-108

Since the PK cannot be completely integrated, the rest can be found in the PK remainder.

Note: CAR must be listed separately, because CAR is hidden by the signature. PK.CA must, however, be identifiable in order to be selected uniquely for the verification of the certificate

Table 14-12. Key parameters for an RSA public key

TagName	**T-L**	**Value**
CPI	'5F 29 01'	Certificate Profile Identifier, refer to 14.7.2
CAR	'42 08'	Certificate Authority reference, refer to 14.7.3
CHR	'5F 20 0C'	Certificate Holder Reference, refer to 14.7.4

Table 14-12. Key parameters for an RSA public key

TagName	T-L	Value
CHA	'5F 4C 07	Certificate Holder Authorisation, refer to 14.7.5
OID	'06 0x	Object Identifier, < to be assigned > currently RSA with Diffie-Hellman has not been assigned refer to 14.7.8
PK	'7F 49' L	'81' L_{81} N '82' L_{82} PK = e

14.9 C_CV.CA.CS-AUT

C_CV.CA.CS-.AUT codes the CA's certificate issued by root certification authority This certificate is only applicable in a two-stage authentication process (refer to 8.6 "Authentication summary" on page 8-58) It is used in order to deliver the public key of the certification authority which is required to verify the C_CV.IFD.AUT as described in the previous paragraph.

The certificate is card verifiable and is delivered as signature with message recovery.

Table 14-13. Structure and content of C_CV.CA.CS_AUT (Example with N=1024 bits)

'7F 21'	'81 CD'	C_CV certificate		
		'5F 37'	**'81 80'**	Signature of length(N), this example uses 1024 bit modulus. **'6A'** = Padding according to ISO 9796-2 **CPI** = '03' (e.g. for 1024 bit RSA) **CAR** = certification authority reference **CHR** = card holder reference **CHA** = AID[1..6] || Role Identifier **OID** = '2B 24 03 04 02 02 01' **PK_part1** = 'XX ... XX' (MSB ⇒LSB) **Hash** = 'XX ...XX' '**BC'** The block '6A .. BC' forms the DSI (digital signature input) The fields above describe the DSI, however the actually stored bytes represent the actual signature with the same length
		'5F 38'	**'3C'**	**PK_part2** = 'XX ... XX' (MSB ⇒LSB) **PK_exp** = '00 01 00 01'
		'42'	**'08'**	**CAR** see 14.7.3 on page 14-108

Since the PK cannot be completely integrated, the rest can be found in the PK remainder.

Note: CAR must be listed separately, because CAR is hidden by the signature. PK.CA must, however, be identifiable in order to be selected uniquely for the verification of the certificate.

The tag and length indicator of this headerlist, however are not part of the actual signature input for C_CV.PK.CA.AUT.

Table 14-14. Key parameters for an RSA public key

TagName	T-L	Value
CPI	'5F 29 01'	Certificate Profile Identifier, refer to 14.7.2
CAR	'42 08'	Certificate Authority reference, refer to 14.7.3
CHR	'5F 20 0C'	Certificate Holder Reference, refer to 14.7.4
CHA	'5F 4C 07	Certificate Holder Authorisation, refer to 14.7.5
OID	'06 0x	Object Identifier, refer to 14.7.8
PK	'7F 49' L	'81' L_{81} N '82' L_{82} PK = e

14.10 C.ICC.AUT

C.CA.ICC.AUT is the card's certificate. It is read by 8.5.1.11 "Step 11- External authentication" on page 8-55.

The format of this certificate is not specified. It can be specifed application dependent. This allows the storage of any certificate type. However, as the card might not be able to verify its own certificate it must be assured by the installation procedures, that the certificates are stored securely.

The actual format of C.CA.ICC.AUT is not in the scope of this specification. However, the certificate needs to contain the cryptological parameters as required for device authentication.

15 Files

Data required for the execution of the ESIGN application is stored in files. The actual location of these files is described in the relevant data objects in DF.CIA.

15.1 File structure

The file organisation in the ICC is designed to be compatible with ISO/IEC 7816-4. The file structure of the card is shown in the following figure. The characteristics of the individual files are described in the following chapters.

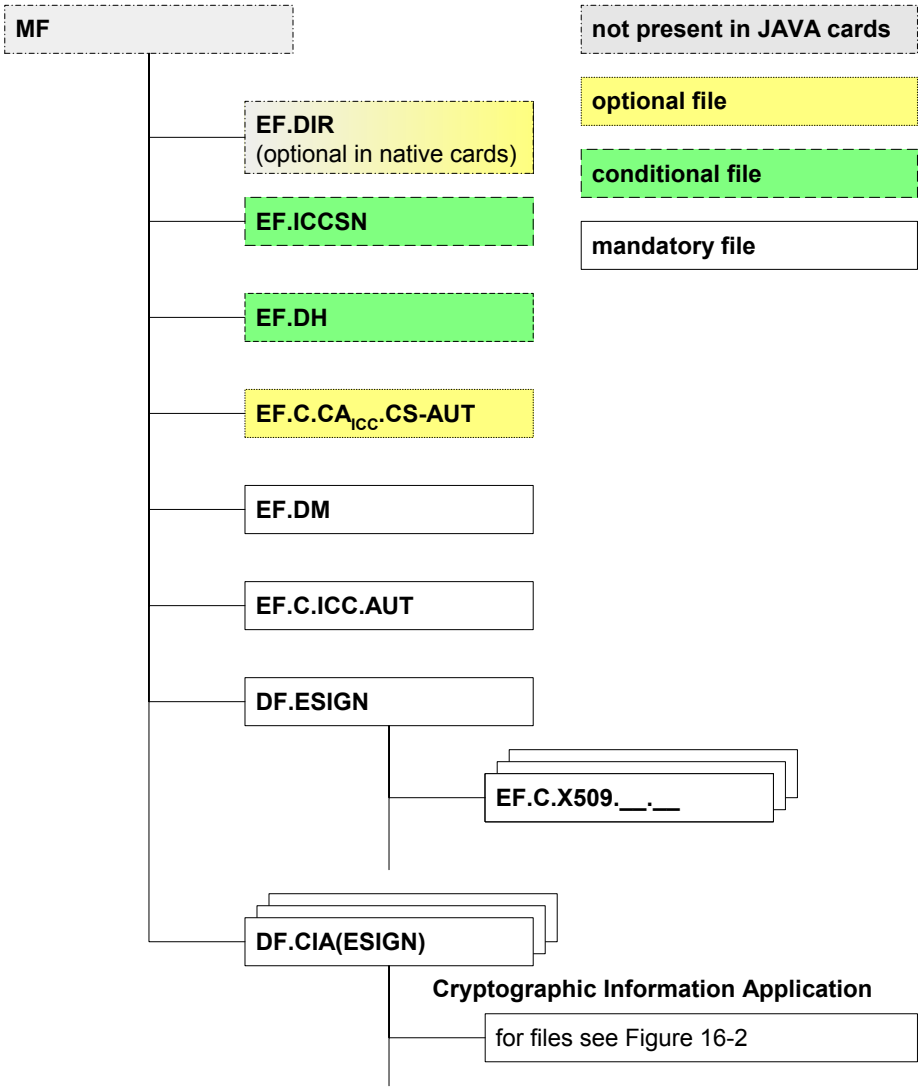


Figure 15-1. Example of file structure of an ESIGN application

Note: Figure 15-1 is an example. In particular the location of DF_ESIGN may be different from the proposal in the figure. The actual location is specified by the definition in the DF.CIA(ESIGN).

In the example in Figure 15-1 the files for the device authentication are placed under the MF as device authentication is considered a device specific mechanism. For JAVA cards, for compatibility reasons the files for device authentication may be located under a separate DF.AUT.

Any file related to the signature application is placed under DF.ESIGN. This avoids icompatibilities with any other application.

15.2 FileIDs

File IDs are at the discretion of the application provider unless specified in the following chapters. The file IDs may either be defined implicitly (i.e. the application in the IFD knows the file ID a priori) or it may be read from the DF.CIA if present. For further information on the cryptographic data objects refer to 16 “Cryptographic Information Application” on page 16-122.)

15.3 EF.DIR

EF.DIR indicates a list of applications supported by the card. It contains a set of application templates and / or application identifier data objects, in any order. It determines which commands shall be performed in order to select the indicated applications.

According to ISO/IEC 7816-4 the path '3F002F00' references EF.DIR. At MF level, the short EF identifier 30, i.e., 11110 in binary, references EF.DIR if present.

Reference

EF.DIR is described in ISO/IEC 7816-4 Chapter 9.2.1.3ff.

Presence

EF.DIR is an optional file.It may be used to indicate the presence and names of applications in the ICC. If not present, the application names (AIDs) need to be known by the IFD.

Access conditions

Read: Always. If present, EF.DIR should be readable before successful device authentication as it may contain information how to find the DF.CIA(ESIGN)

Write: Typically the card issuer has write access to update the EF.DIR.This may vary depending on the actual application environment. Updating EF.DIR is not in the scope of this specification.

15.4 EF.SN.ICC

EF.SN.ICC contains the serial number SN.ICC of the ICC. For some device authentication protocols the serial number SN.ICC is required to accomplish the identity information of the ICC. As some ICCs store only a fraction of SN.ICC in their certificate (typically 8 bytes), the full ICC.SN may be retrieved from EF.SN.ICC. This information might be required by the actual application to uniquely identify the ICC

Presence

The existence of EF.SN.ICC is conditional and depends on the implemented device authentication protocol. The “Device authentication with privacy protection” on page 8-48 does explicitly forbid reading EF.SN.ICC before successful device authentication as the content of EF.SN.ICC reveals the identity (privacy) of the card (holder).

Access conditions

Read: Depends on the type of device authentication protocol. Typically EF.SN.ICC may be read always.

If the “Device authentication with privacy protection” is used, EF.SN.ICC is not allowed to be read until the IFD is successfully authenticated. This is a security requirement in order to keep the identity of the ICC secret until the IFD can be trusted.

Write: Never. The SN.ICC shall be constant during the entire card life cycle of the ICC

15.5 EF.DH

EF.DH contains the public key parameters for the Diffie-Hellman key exchange protocol if used in device authentication. If the IFD knows these parameters implicitly, it does not need to read the information from EF.DH

The response to a READ BINARY APDU contains data objects for the ICC’s key exchange parameters.

Tag	L	Value				
‘7F49’	‘xx’	DO_1	DO_2	...	....	DO_n

Table 15-1. Data content of EF.DH

The coding of the relevant data objects in the value part is described in 11.4 “RSA Diffie-Hellman key exchange parameters” on page 11-90 .

Presence

EF.DH is optional and conditionally depending on the type of device authentication. It may be only required, if Diffie-Hellman key agreement is implemented for device authentication. It is then only required, if the public quantities for Diffie-Hellman key agreement are only partly available at the IFD. Coding all necessary quantities in EF.DH is the choice of highest interoperability.

Access conditions

Read: Always

Write: Typically the quantities of the Diffie-Helllman key agreement are not subject to be changed. nor are they exposed to any security thread. If, however, the need arises for change these parameters (e.g. for a system migration) then only the card issuer shall have the access right to update this information.

15.6 EF.C.ICC.AUT

EF.C.ICC.AUT contains the certificate(s) for the card's public authentication key PK.ICC.AUT. For different applications, different certificates of the same PK are likely to exist. This will prove the application, that the presented public key is authentic to be used within its scope.

The EF.C.ICC.AUT is usually located under the MF (see Figure 15-1). In DF.CIA, there should be a CIO describing the file ID (or path) and access conditions.

The data field of EF.C.ICC.AUT contains one or more certificates for the same card's public key.

Different certificates of EF.C.ICC.AUT are distinguished from each other by the CAR-field, which is the last field in the certificate. (Refer to 14.7.3 "CAR-Certification Authority Reference (Authority Key Identifier)" on page 14-108). This allows the selection of the appropriate certificate without the need of the decryption of the signature.

Tag	L	Value
'7F21'	'xx'	encapsulates certificate 1 over PK.ICC.AUT (issued by CA 1)
'7F21'	'yy'	encapsulates certificate 2 over PK.ICC.AUT (issued by CA 2)
'7F 21'	'zz'	...
...	...	...

Table 15-2. Data content of EF.C_CV.ICC.AUT

The coding of the certificate is described in 14.10 "C.ICC.AUT" on page 14-115.

15.7 EF.C.CA_{ICC}.CS-AUT

EF.C.CA_{ICC}.CS-AUT contains the certificate of the CA(s) that generated the certificate for the ICC's public key. If different CA's are used for the certification of the public key, the appropriate CA's certificates are stored together in this file.

The data field of EF.C.C.CA_{ICC}.CS-AUT contains one or more certificates for the same card's public key.

Tag	L	Value
'7F21'	'xx'	encapsulates certificate over PK.CA(1).CS (issued by RCA)
'7F21'	'yy'	encapsulates certificate over PK.CA(2).CS (issued by RCA)
'7F 21'	'zz'	...
...	...	...

Table 15-3. Data content of EF.C.CA_{ICC}.CS-AUT

The EF.C.C.CA_{ICC}.CS-AUT certificates are preferably stored as self-descriptive certificates, i.e. the signature part of the certificate allows message recovery and the possible PK-remainder is stored in a separate data field, if necessary.

The coding of the certificate is described in 14 “CV_Certificates and Key Management” on page 14-103.

Different certificates of EF.C.C.CA_{ICC}.CS-AUT are distinguished from each other by the CAR-field, which is the last field in the certificate. (Refer to 14.7.3 “CAR-Certification Authority Reference (Authority Key Identifier)” on page 14-108). The separate storage of CAR allows the selection of the appropriate certificate without the need of the decryption of the signature

Presence

The presence of EF.C.CA_{ICC}.CS-AUT is conditional. If the ICC is used in a system that has PK.CA_{ICC}.CS-AUT already available or where this public key can be retrieved through other channels (e.g. an Internet connection), EF.C.CA_{ICC}.CS-AUT does not need to exist. However, in many applications this might not be the case. Then EF.C.CA_{ICC}.CS-AUT shall be present in order to allow the IFD the verification of the ICC’s authentication certificate.

Access conditions

Read: Always

Update: Typically the content of certificates is not subject to changes. As an exception, writing this file might be allowed for administrational purposes in order to update the certificate. The update of certificates is considered application dependent and not described in this specification

15.8 EF.C_X509.CH.DS

The EF.C_X509.CH.DS contains the DS-certificate(s) of the cardholder. This certificate contains the public key for the card holders signature key.

Presence

The presence of this file is optional. If another format than X.509 is required by the application, then the content of this file might be replaced by an appropriate certificate.

Access conditions

Read: Always

Update: Typically the content of certificates is not subject to changes. As an exception, writing this file might be allowed for administrational purposes in order to update the certificate. The update of certificates is considered application dependent and not described in this specification

15.9 EF.C_X509.CA.CS (DF.ESIGN)

The EF.C_X509.CA.CS contains the X.509v3 certificate of the CA, which issued the cardholder certificates. The format of this certificate is described in other standards and is out of scope of this standard.

Presence

The presence of EF.C_X509.CA_{ICC}.CS-DS is conditional. If the ICC is used in a system that has PK.CA_{ICC}.CS-DS already available or where this public key can be retrieved through other channels (e.g. an Internet connection), EF.C_X509.CA_{ICC}.CS-DS does not need to exist. However, in many applications this might not be the case. Then EF.C_X509.CA_{ICC}.CS-DS should be present in order to allow the IFD the verification of the card holder's signature certificate.

Access conditions

Read: Always

Update: Typically the content of certificates is not subject to changes. As an exception, writing this file might be allowed for administrational purposes in order to update the certificate. The update of certificates is considered application dependent and not described in this specification

15.10 EF.DM

EF.DM contains the display message for the use during device authentication as described in 8.11 "Reading the Display Message" on page 8-63.

Presence

mandatory

Access conditions

Read: Only encrypted under secure messaging. The fact, that EF.DM is read, is used as a proof of evidence for an active secure messaging session.

Additional read access after the presentation of the related password is an optional application specific feature.

Update: Updating is allowed only after presentation of the related password. The reference to the related password is specified in the relevant CIO in DF.CIA.

This function is described in 8.12 "Updating the Display Message" on page 8-65.

Note: If the DM is located under the MF as in the example of Figure 15-1, only the global password can be used. If the local password ("signature PIN"), is intended to be used, then general national regulations have to be respected.

16 Cryptographic Information Application

This chapter describes the cryptographic information application.

In order to standardize interface for using the card, we need to specify all APDUs addressing cryptographic information during usage phase.

For that reason, it is important to standardize the way to access from a terminal data that can be read or written, this data are public key (to get a certificate), certificate, PIN….

A minimum file system for the ESIGN application respecting ISO 7816-15 has the following structure:

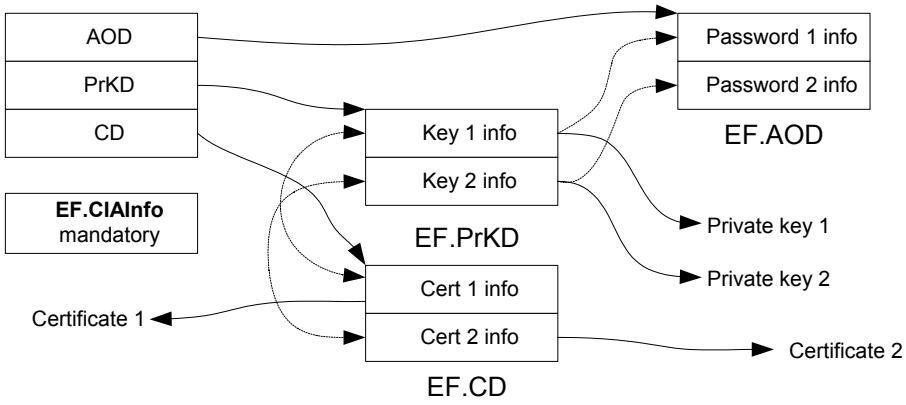


Figure 16-1. File relationships in DF.CIA.

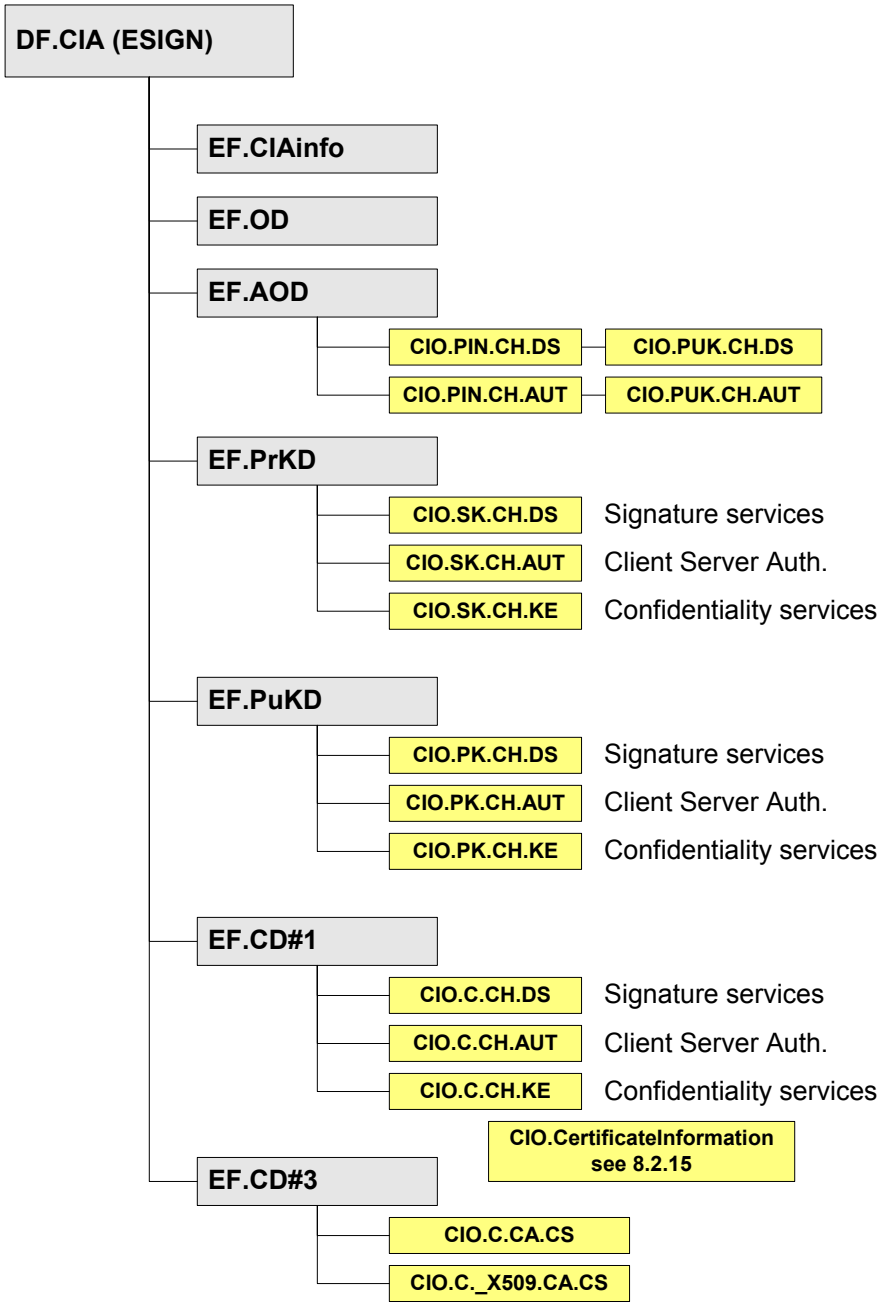


Figure 16-2. CIA file structure (example)

It is mandatory that each DF.CIA contains one EF.OD and one EF.CIAInfo.

16.1 ESIGN cryptographic information layout example

```
ESIGNK
DEFINITIONS ::= BEGIN
IMPORTS
   CIAInfo, CIOChoice, AuthenticationObjectChoice,
   PrivateKeyChoice, CertificateChoice,
   PublicKeyChoice, SecretKeyChoice,
   DataContainerObjectChoice
    FROM
   CryptographicInformationFramework;


-- Definition of ISO7816-15 Directory Files
EFOD ::= SEQUENCE OF CIOChoice
EFAOD ::= SET OF AuthenticationObjectChoice
EFPrKD ::= SEQUENCE OF PrivateKeyChoice
EFCD ::= SEQUENCE OF CertificateChoice
EFPuKD ::= SEQUENCE OF PublicKeyChoice
EFSKD ::= SEQUENCE OF SecretKeyChoice
EFDCOD ::= SEQUENCE OF DataContainerObjectChoice

-- EF.CIAInfo
EF.CIAInfo CIAInfo ::=
{
  version v2,
  serialNumber ''H,
  manufacturerID "ACME",
  label "ESIGNK Application",
  cardflags { readonly, authRequired, prnGeneration }
}

-- EF.OD
EFOD EFOD ::=
{
  privateKeys : path : {efidOrPath '4001'H},
  publicKeys : path : { efidOrPath '4002'H },
  certificates : path : {efidOrPath  '4005'H },
  trustedCertificates : path : { efidOrPath '4004'H },
  dataContainerObjects : path : { efidOrPath '4006'H },
  authObjects : path : { efidOrPath '4003'H }
}

-- EF.AOD
EFAOD EFAOD ::=
{
-- DO.PIN #1
  pwd : -- PIN.CH.DS
  {
    commonObjectAttributes
    {
      label "PIN.CH.DS",
      authId '9A'H -- Reference to PUK.CH.DS
    },
```

```
    classAttributes { authId '99'H },
    typeAttributes
    {
      pwdFlags
      {
        local, initialized
      },
      pwdType bcd,
      minLength 6,
      storedLength 8,
      maxLength 8,
      pwdReference 153,
-- 0x99 Key Id
      padChar '00'H,
-- Path of the DF.ESIGN were PWD resides
      path
      {
        efidOrPath '3F 00 45 00'H
      }
    }
  },

-- DO.PIN #2
  pwd : -- PIN.CH.ADMIN
  {
-- Reference to PUK.CH.ADMIN
    commonObjectAttributes
    {
      label "PIN.CH.ADMIN",
      authId '9C'H
    },
    classAttributes { authId '9B'H },
    typeAttributes
    {
      pwdFlags
      {
        local, initialized
      },
      pwdType bcd,
      minLength 6,
      storedLength 8,
      maxLength 8,
-- 0x9B Key Id
      pwdReference 155,
      padChar '00'H,
-- Path of the DF.ESIGN were PWD resides
      path { efidOrPath '3F 00 45 00'H }
    }
  },

-- EF.PrKD
EFPrKD EFPrKD ::=
{
-- CIO.PrK #1
  privateRSAKey : -- SK.CH.DS
```

```
  {
    commonObjectAttributes
    {
      label "SK.CH.DS",
      flags { private },
-- Reference to DO.PIN #1
      authId '99'H,
      userConsent 1,
    },
-- shall share the same iD with Public Key and Certificate
    classAttributes
    {
      iD '01'H,
      usage { sign, nonRepudiation },
      native TRUE,
      accessFlags
      {
        sensitive,
        alwaysSensitive,
        neverExtractable,
        cardGenerated
      },
-- 0x80 KID in the card
      keyReference 128,
    },
    typeAttributes
    {
      value indirect : path : {efidOrPath '3F 00 45 00'H},
      modulusLength 1024
    }
  },

-- CIO.PrK #2
privateRSAKey : -- SK.ICC.AUT
{
  commonObjectAttributes
  {
    label "SK.ICC.AUT",
    flags { private },
  }
-- shall share the same iD with Public Key and Certificate
  classAttributes
  {
    iD '02'H,
    usage {decipher, sign},
    native TRUE,
    accessFlags
    {
      sensitive,
      alwaysSensitive,
      neverExtractable,
      cardGenerated
    },
-- 0x81 KID in the card
    keyReference 129,
```

```
  },
  typeAttributes
  {
    value indirect : path : {efidOrPath '3F 00 45 00'H },
    modulusLength 1024
  }
},

-- EF_PuKD
EFPuKD EFPuKD ::=
{
-- PK.CH.DS
  publicRSAKey :
  {
    commonObjectAttributes
    {
      label "PK.CH.DS",
      objectHandle 7
    },
-- shall share the same iD with Private Key and Certificate
    classAttributes
    {
      iD '01'H,

-- shall share the same iD with Private Key and Certificate
      usage { verify},
      native TRUE,
      accessFlags { cardGenerated },
      keyReference 131
-- 0x83 KID in the card
    },
    typeAttributes
    {
      value indirect : path : { efidOrPath '3F 00 45 50'H },
      modulusLength 1024
    }
  },


-- PK.ICC.AUT
  publicRSAKey :
  {
    commonObjectAttributes
    {
      label "PK.ICC.AUT"
    },
-- shall share the same iD with Private Key and Certificate
    classAttributes
    {
      iD '02'H,
      usage { verify},
      native TRUE,
      accessFlags { cardGenerated },
      keyReference 132 -- 0x84 KID in the card
    },
```

```
    typeAttributes
    {
      value indirect : path : { efidOrPath '3F 00 45 50'H },
       modulusLength 1024
    }
  },
}

-- EF.CD#1 Card Holder certs 'EF: 3F 00 40 00 40 04'
EFCD1 EFCD ::=
{
-- C.CH.DS
  x509Certificate :
  {
    commonObjectAttributes
    {
      label "C.CH.DS",
       objectHandle 4
    },
-- shall share the same id with Private Key and Public Key
    classAttributes
    {
      iD '01'H,
      authority FALSE
    },
    typeAttributes
    {
 -- Path to EF.C_X509.CH.DS
      value indirect : path : {efidOrPath '3F 00 45 00 C0 00'H
    },
  }
},


-- C.ICC.AUT
cvCertificate :
{
  commonObjectAttributes
  {
    label "C.ICC.AUT"
  },
-- class attributes shall share the same id with Private Key and Public Key
  classAttributes
  {
    iD '02'H,
    authority FALSE
  },
  typeAttributes
  {
-- Path to EF.C.ICC.AUT
    value indirect : path :{efidOrPath '3F 00 45 00 C0 01'H}
  }
},
```

```
-- EF.CD#3 Trusted certs 'EF: 3F 00 40 00 40 05'
EFCD3 EFCD ::=
{
-- C.CA.CS
  x509Certificate :
  {
    commonObjectAttributes
    {
      label "C.CA.CS",
      objectHandle 7
    },

-- shall share the same id with Private Key and Public Key
    classAttributes
    {
      iD '04'H,
      authority TRUE
    },
    typeAttributes
    {
      value indirect : path :
      {
-- Path to EF.C_X509.CA.CS
        efidOrPath '3F 00 45 00 C0 03'H
      },
    }
  }
}
END
```

Annex A. Dev. auth. - cryptographic view

A.1 Algorithms for authentication with key exchange or key negotation

The list below shows the algorithms being available for the use as authentication algorithms. Being applied to authentication two algorithms are generally available for the existence proof of a secret key, i.e. for the verification of a public key's certificate.

- RSA
- DSA

The currently supported padding schemes used for device authentication purposes are

- Padding according to ISO/IEC 9796-2 (DS scheme 1)
- Padding according to PKCS #1 (V2.0)

The symmetrical session keys to be established may be obtained from two methods

- Diffie Hellman key negotiation
- RSA key exchange

A.2 Device authentication with key transport

A.2.1 Conformance to ISO/IEC 11770-3

For device authentication with key transport, the Key transport mechanism 6 of ISO/IEC 11770-3 [27] is applied.

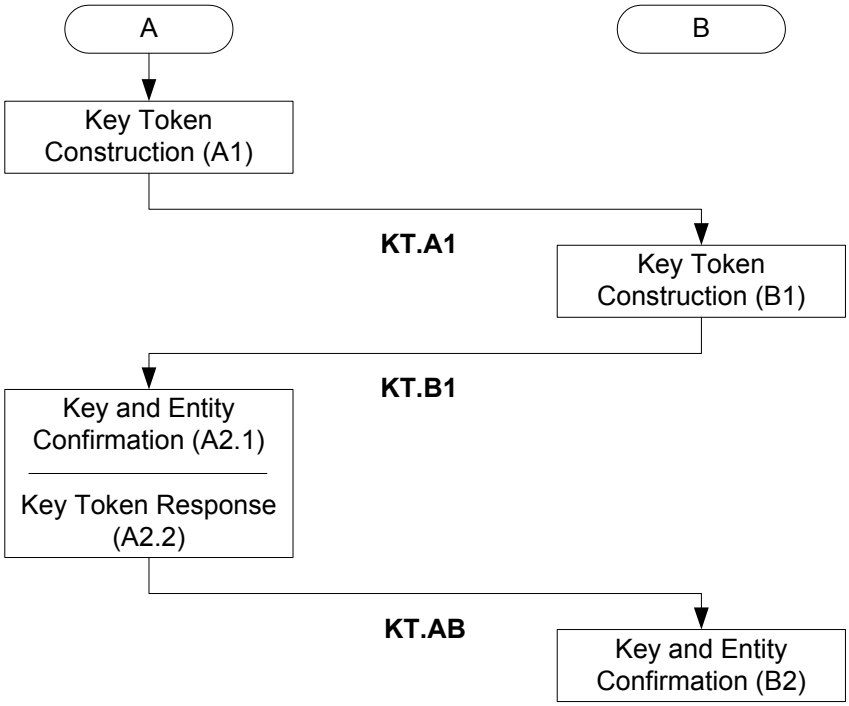


Figure A-1. Key transport mechanism 6 as described in ISO/IEC 11770-4

Key transport mechanism 6 (Extract from ISO/IEC 11770-3)

This key transport mechanism securely transfers in three passes two secret keys, one from A to B and one from B to A. In addition, the mechanism provides mutual entity authentication and mutual key confirmation about their respective keys. This mechanism is based on the following requirements:

1. Each entity X has an asymmetric encipherment system (EX,DX).

2. Each entity has access to an authenticated copy of the public encipherment transformation of the other entity. This may be achieved using the mechanisms of clause 8 of ISO/IEC 11770-3.

Key Token Construction (A1): A has obtained a key KA and wants to transfer it securely to B. A selects a random number rA and constructs a key data block consisting of its distinguishing identifier A, the key KA, the number rA and an optional data field Text1. Then A enciphers the key block using B's public encipherment transformation EB, thereby producing the enciphered data block

BE1 = EB (A || KA || rA || Text1)

A constructs the token KTA1, consisting of the enciphered data block and some optional data field Text2

KTA1 = BE1 || Text2

A sends the token to B.

Key Token Construction (B1): B extracts the enciphered key block BE1 from the received key token KTA1 and deciphers it using its private decipherment transformation DB. Then B verifies the sender identity A.

B has obtained a key KB and wants to transfer it securely to A. B selects a random number rB and constructs a key data block consisting of the distinguishing identifier B, the key KB, the random number rB, the random number rA (as extracted from the deciphered block) and an optional data field Text3. Then B enciphers the key block using A's public encipherment transformation EA, thereby producing the enciphered data block

BE2 = EA (B || KB || rA || rB || Text3)

Then B constructs the key token KTB1, consisting of the enciphered data block BE2 and an optional data field Text4,

KTB1 = BE2 || Text4

B sends the token to A.

Key and Entity Confirmation (A2.1) : A extracts the enciphered key block BE2 from the received key token KTB1 and deciphers it using its private decipherment transformation DA. Then A checks the validity of the key token through comparison of the random number rA with the random number rA contained in the enciphered block BE2. If the verification is successful, A has authenticated B and at the same time obtained confirmation that KA has safely reached entity B. Key Token Response (A2.2) A extracts the random number rB from the deciphered key block and constructs the key token KTA2, consisting of the random number rB and an optional data field Text5,

KTA2= rB ||Text5.

A sends the token to B.

Key and Entity Confirmation (B2) : B verifies that the response rB extracted from KTA2 is consistent with the random number rB sent in enciphered form in KTB1. If the verification is successful, B has authenticated A and at the same time has obtained confirmation that KB has safely reached entity B.

Note: This Key Transport Mechanism has the following properties:

1. Number of passes: 3.

2. Entity authentication: this mechanism provides

mutual entity authentication, implicit key authentication of KA from B to A and implicit key authentication of KB from A to B..

3. Key confirmation: this mechanism provides mutual key confirmation.

4. Key control: A can choose the key KA, since it is the originating entity. Similarly, B can choose the key KB. Joint key control can be achieved by each entity by combining the two keys KA and KB on both sides to form a shared secret key KAB. However, the combination function must be one-way, otherwise B can choose the shared secret key. This mechanism could then be classified as a key agreement mechanism.

5. Key usage: this mechanism uses asymmetric techniques to mutually transfer two secret keys, KA from A to B and KB from B to A. The following cryptographic function separation may be derived from the mechanism: A uses its key KA to encipher messages for B and to verify authentication codes from B. B in turn uses the received key KA to decipher messages from A and generate authentication codes for A. The cryptographic functions of KB may be separated in an analogous manner. In such a way, the asymmetric basis of the key transport mechanism may be extended to the usage of the secret keys.

6. Example: this mechanism is derived from the three pass protocol known as COMSET (see Brandt et al. in the Bibliography).

7. Background: this mechanism is based on zero-knowledge techniques. From the execution of the mechanism neither of the entities learns anything that it could not have computed itself.

A.2.2 Using min(SIG, N-SIG) for the signature token

Classical computation - a contradiction ....

Let N.ICC be the modulus of the ICC

An encrypted value

$$X = M^{e_{ICC}} \bmod N.ICC \tag{A-1}$$

should satisfy the condition M < N in order to be decrypted correctly because the decryption

$$M' = X^{d_{ICC}} \bmod N.ICC \tag{A-2}$$

will always produce a result with M' < N.ICC. Hence M' can only be identical to M, if $M < N$ was given in the encryption.

Let N.IFD be the modulus of the IFD. As usual SIG.IFD is defined as, whereas M is the digital signature input.

$$\text{SIG.IFD} = Z = M^{d_{IFD}} \bmod N.IFD \tag{A-3}$$

then SIG.IFD is a number < N.IFD. In order to always be encryptable with PK.ICC, according to (A-2)

$$\text{N.IFD} < \text{N.ICC} \tag{A-4}$$

would be required as a sufficient condition.

However, in the next protocol step, the roles are swapped and the corresponding condtion would require

$$\text{N.ICC} < \text{N.IFD} \tag{A-5}$$

for proper encryption, **This is a contradiction to (A-4)**.

The solution to the 'contradiction'

However, if a signature is defined, as

$$\text{SIG} = \min(Z, \text{N.IFD} - Z) \tag{A-6}$$

with Z from (A-3), then it is always valid

$$\text{SIG} < \frac{\text{N.IFD}}{2} \tag{A-7}$$

Now the roles may be changed and we obtain the condition

$$\frac{\text{N.ICC}}{4} < \frac{\text{N.IFD}}{2} < \text{N.ICC} \tag{A-8}$$

which is always true, if the lengths of the moduli N.ICC and N.IFD are equal. After decryption with SK.ICC, checking the signature with

$$\text{SIG} = min(M^{\text{d.IFD}} \bmod \text{N.IFD},\ \text{N.IFD} - M^{\text{d.IFD}} \bmod \text{N.IFD}) \tag{A-9}$$

is performed as

$$T = \mathrm{SIG}^{e.IFD} \bmod N.IFD \quad \text{(A-10)}$$

If the last byte of T is identical to ‘BC’, then set

$$Z = T. \quad \text{(A-11)}$$

Otherwise set

$$Z = N.IFD - T \quad \text{(A-12)}$$

Verify the equation

$$Z = (6A,\ PRND1,\ HASH,\ BC) \quad \text{(A-13)}$$

to evaluate the correctness of the signature.

A.3 Device authentication with key negotiation

The reference authentication scheme described in A.3.1 “Diffie-Hellman Key Exchange” on page A-136 uses the terms KT.A = KT.A1 and KT.B = KT.B1 as described in [27]. These terms are required in order to negotiate and derive the session keys for the trusted channel to be established.

The following chapters describe the computation of the terms KT.A and KT.B with respect to the appropriate key agreement mechanisms.

A.3.1 Diffie-Hellman Key Exchange

Diffie-Hellman was the first public-key algorithm published, in 1976 [34]. It gets its security from the difficulty of calculating discrete logarithms in a finite field, as compared with the ease of performing exponentiation calculations in the same field.

A.3.1.1 Simple Diffie-Hellman Key Exchange, No Authentication

The general form of an unauthenticated Diffie-Hellman key exchange is as follows: in this, A is the card reader and B is ESIGN application.

Both A (the card reader) and B (ESIGN application) share the public quantities p, q and g where:

- p is the modulus, a prime number; for security, this number should be of the order of 1024 bits or more.
- q is a prime number in the range of 159-160 bits
- g is a generator of order q, that is $g^q = 1 \bmod p$ and

Then the protocol goes as follows:

1. A chooses random number x with $1 \leq x \leq q-1$

   B chooses random number y with $1 \leq y \leq q-1$

2. A computes $K_A = g^x \bmod p$ and sends it to B

   B computes $K_B = g^y \bmod p$ and sends it to A

3. A computes $K_{AB} = (K_B)^x \bmod p$

   B computes $K_{BA} = (K_A)^y \bmod p$

4. A deletes the random number x

   B deletes the random number y

A and B have derived the same number because

$$K_{AB} = (K_B)^x = (g^y)^x = (g^x)^y = (K_A)^y = K_{BA}$$

A.3.1.2 Authenticated Diffie Hellman

The algorithm described above in section A.3.1.2 produces an agreed secret key; however, there is no authentication included. ISOI/EC 11770-3 [27] section 6.7 "Key agreement mechanism 7" uses a Diffie-Hellman style of key exchange, and also involves mutual authentication. The following shows the general flow of such a scheme:

Stage 1. The reader, A, calculates K_A as above, and concatenates it with a certificate containing A's indentity A and public key PK_A. A transmits this to B:

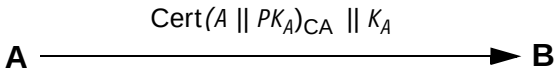


Figure A-2. Key Agreement Mechanism 7 of ISO 11770-3, Part 1

Stage 2. The card, B, checks the certificate received from A. B then calculates K_B and K_{BA} as above, and generates a message for A consisting of the following items concatenated together:

i. a certificate containing B's identity B and public key PK_B

ii. a signature of the concatenation of K_B, K_A and A's indentity A, signed using B's secret key SK_B corresponding to the public key PK_B

Note that this signature consists of just the signature value - it need not include the quantities being signed, since A already has them (they were sent to B in Stage 1 above), or are included as part of this message.

iii. a cryptographic check, using the crypto function **f** with the key K_{BA}, of K_B, K_A and A's indentity A.

B then transmits this to A:

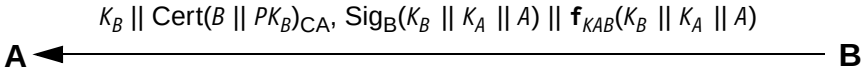


Figure A-3. Key Agreement Mechanism 7 of ISO 11770-3, Part 2

Stage 3. A now:

- checks the certificate received from B
- verifies the signature (using B's public key PK_B contained in the certificate)
- calculates K_{AB}, and uses this to verify cryptographic check using the function **f** and the key K_{AB}

A then generates a message consisting of the following items concatenated together:

i. a signature of the concatenation of K_A, K_B and B's indentity B, signed using A's secret key SK_A corresponding to the public key PK_A

Note that this signature consists of just the signature value - it need not include the quantities being signed, since B already has them.

ii. a cryptographic check, using the crypto function **f** with the key K_{AB}, of K_A, K_B and B's indentity B.

A then sends this to B:

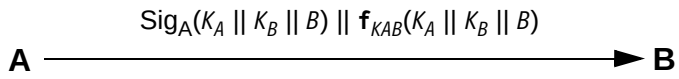


Figure A-4. Key Agreement Mechanism 7 of ISO 11770-3, Part 3

Stage 4. Finally, B:

- verifies the signature (using A's public key PK_A transmitted in the certification in Stage 1)
- verifies cryptographic check using the function **f** and the key K_{BA}

If the two verifications succeed, B signifies his acceptance to A:

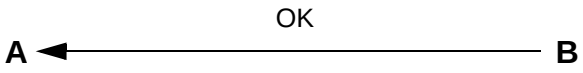


Figure A-5. Key Agreement Mechanism 7 of ISO 11770-3, Part 4

A.4 Device authentication with privacy protection

The protocol based on ISO 11770-3, discussed in section 3.2 above, has the disadvantage that the card ("*B*" in the discussion) is revealing his identity and certificate before he has verified the credentials of the reader "*A*". This could be viewed as a violation of the privacy of the card holder - the identity and certificate of the card *B* are revealed, not just to the reader *A*, but also to anyone eavesdropping on the communications between the reader and the card. The reader *A* might not be physically co-located with the card, but actually connected via a network of some sort. The DIN standards [40][41] for the digital signature cards suffer from this potential privacy problem.

Two steps are taken to counter the potential privacy problems:

A discloses its identity and credentials to *B* first; *B* reveals its identity and credentials only after verifying those of *A*.

The session key parameters are exchanged very early in the protocol, even before the authentication has been completed. In this way, most of the protocol can be protected from eavesdropping. Using the session key parameters before authentication still leaves us open to a man-in-the-middle attack. However, once the authentication is complete, any potential man-in-the-middle will have been discovered, the session can be aborted with only a potential one-time privacy problem, and *A* can take appropriate action concerning the presence of a man-in-the-middle intruder.

The protocol based on ISO 11770-3 [27], discussed in section 3.2 above, also has the disadvantage that the number of bits transmitted in all the stages is somewhat larger than necessary. Minimizing the total number of bits transmitted is important, because some smart card readers will only communicate at 9600bps, and even ignoring the cost of computing the cryptographic operations, the time needed to transmit all the bits could become a serious problem in response time to the card holder. By contrast, the protocol used in the Internet Key Exchange (IKE) standard [39] for the Internet transmits fewer bits in total. Furthermore, the IKE protocol has been completely proven correct [43], which is a major benefit in any system planning to be evaluated at the highest levels of the Common Criteria. Therefore, the privace protecting authentication protocol is based on IKE to take advantage of both the performance and formal proof benefits.

This section contains a cryptographic description of the authentication protocol used by the ESIGN device authentication with privacy protection. This differs from the protocol described in A.3 , in that it starts as in unauthenticated Diffie-Hellman, and then authenticates *A* before *B* exposes his identity.

Stage 1. *A* chooses a random number a with $1 \leq a \leq q-1$, computes a key token $K_A = g^a \bmod p$, and transmits it to *B*

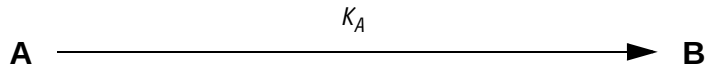


Figure 1 Authentication Stage 1: A sends a key token to B

Stage 2. *B* chooses a random number *b* with $1 \leq b \leq q\text{-}1$, computes a key token $K_B = g^b \bmod p$., and transmits it to *A*.

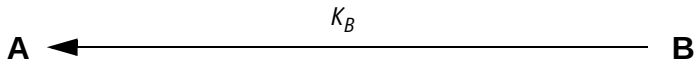


Figure 2 Authentication Stage 2: B sends a key token to A

At this point, neither *A* nor *B* has revealed his identity. However, they now can compute a mutual key K_{AB} as in A.3.1.1 . Using the mutual key K_{AB}, they can derive additional keys K_{A1}, K_{A2}, and K_{A3}, as specified in 8.8 "Compute session keys from key seed $K_{IFD/ICC}$" on page 8-62..

Stage 3a. *A* now computes the session keys according to A.3.1.1 and 8.8 . The session key for encryption will be K_E. A sends its certificate to B

A now computes E_{01} as shown below:

$$E_{01} = \text{3DES_Encrypt}_{K_E}(\text{Cert}(A)) \qquad \text{(A-14)}$$

A now transmits E_{01} and a hash of the certificate to *B*.



Figure 3 Authentication Stage 3: A sends certificate to B

Stage 3b. B responds with a challenge.

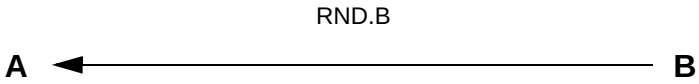


Figure 4 Authentication Stage 4: B sends challenge to A

Stage 3c. *A* now computes E1 as shown below:

$$= \text{3DES_Encrypt}_{K_E}(\text{ A} \,||\, \text{Sig}_{SK_A}[K_A \,||\, \text{A} \,||\, \text{RND.B} \,||\, K_B \,||\, \text{DH(g} \,||\, \text{p} \,||\, \text{q)}) \qquad \text{(A-15)}$$

A now transmits E1 and a hash of his identity, certificate, and the value E1 to *B*. The signature is a signature with message recovery, so all parameters in the signature can considered to be recoverable. The Diffie-Hellman key parameters are part of the signature in order to provide authenticity of the parameters.



Figure 5 Authentication Stage 3: authenticate A

Stage 4a. *B* now decrypts E_1, verifies the MAC, and verifies the signature using *A*'s public key PK_A. *B* has now authenticated *A* and knows that K_A and K_B are fresh and authentic. However at this point, while *B* knows there is no man-in-the-middle because *B* checked the signature from *A*, *A* does not know who he is talking to, and hence is unsure if there may be a man-in-the-middle attack.

B computes E_{02}

$$E_{02} = \text{3DES_Encrypt}_{K_E}(\text{Cert}(B)) \qquad \text{(A-16)}$$

B and sends its certificate to A



Figure 6 Authentication Stage 4a: B sends certificate to A

Stage 4b. A sends a challenge to B

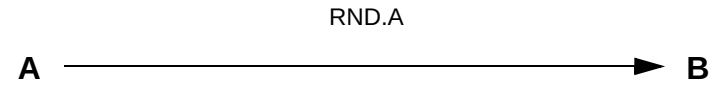


Figure 7 Authentication Stage 4b: send challenge to B

Stage 4c: *B* now computes E_2 as shown below:

$$_2 = \text{3DES_Encrypt}_{K_E}(\text{ B} \,||\, \text{Sig}_{SK_B}[K_B \,||\, B \,||\, \text{RND.A} \,||\, K_A \,]$$

The signature is a signature with message recovery, so all parameters in the signature can considered to be recoverable.

B now transmits E_2 and a hash of his identity, certificate, and the value E2 to A.

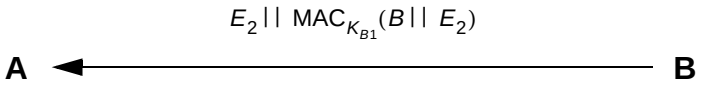


Figure 8 Authentication Stage 4c: authenticate B

A can now decrypt E_2 and verify the MAC. Using the chain of certificates back to the root CA, *A* can verify the certificate from the IC manufacturer for *B*, which contains *B*'s identify *B* and public key PK_B. Thus *A* knows, and can trust, *B*'s public key PK_B. Hence A can now authenticate *B* by verification of the signature

$$g_{SK_A}[K_A \,||\, \text{A} \,||\, \text{RND.B} \,||\, K_B \,||\, \text{DH(g} \,||\, \text{p} \,||\, \text{q}\,. \qquad \text{(A-17)}$$

A.4.1 The authenticity of the public DH parameters

The proof given in [43] precludes, that the public DH parameters are authentically known. This assumption does not necessarily hold for signature cards, therefore the public DH parameters may be retrieved from a file EF.DH prior to the device authentication.

As a consequence, the authenticity of these public key parameters is not given implicitly. Therefore the public key parameters need to be signed by the IFD, which allows the ICC to verify, that the IFD used its correct parameters.

The following scenario demonstrates, how this weakness could be used for an attack, given, the public DH parameters were not signed by the IFD

Step	IFD		ICC
1		←	p, q, g
2	$K_{IFD} = g^{r_{IFD}} \bmod p$	→	
3		←	$K_{ICC} = g^{r_{ICC}} \bmod p$
4			$K_{IFD/ICC} = K_{IFD}^{\ r_{ICC}} \bmod p$
5		←	RND.ICC
6	$\tilde{K}_{IFD/ICC} = K_{ICC}^{\ r_{IFD}} \bmod \tilde{p}$		
7	$SIG_{IFD}(K_{IFD} \mid\mid$ RND.ICC $\mid\mid K_{ICC}$	→	
8	RND.IFD	→	
9		←	$SIG_{ICC}(K_{ICC} \mid\mid$ RND.IFD $\mid\mid K_{IFD}$

As soon as $K_{IFD/ICC}$ is available it is used to secure the communcation between IFD and ICC (after session key generation).

We suggest, that an attacker changes the transmission such, that he replaces p by $\tilde{p}$. We obtain

Step	IFD		ICC
1		←	$\tilde{p}, q, g$
2	$\tilde{K}_{IFD} = g^{r_{IFD}} \bmod \tilde{p}$	→	
3		←	$K_{ICC} = g^{r_{ICC}} \bmod p$

4			$\tilde{K}_{IFD/ICC} = \tilde{K}_{IFD}^{\ r_{ICC}} \bmod p$
5		←	RND.ICC
6	$\hat{K}_{IFD/ICC} = K_{ICC}^{\ r_{IFD}} \bmod \tilde{p}$		

In general, $\tilde{K}_{IFD/ICC}$ and $\hat{K}_{IFD/ICC}$ are different. However, if the attacker chooses $\tilde{o} = c \cdot p$ with a small number c (e.g. c = 2), an attacker observe the case

$$K_{IFD/ICC} \equiv 1 \bmod c \tag{A-18}$$

with an expected probability of approx. $\frac{1}{c}$. In this case it is easy to prove that the equation

$$\tilde{K}_{IFD/ICC} = \hat{K}_{IFD/ICC} = K_{IFD/ICC} \tag{A-19}$$

is valid exactly if

$$K_{IFD/ICC} \equiv 1 \bmod c \tag{A-20}$$

If IFD and ICC may communicate with secure messaging without error indication, the attacker obtains information about the negotiated key. This may be in contradiction with the terminology of 'provable security'.

Reason:

For $\tilde{o} = c \cdot p$ it is always valid

$$\tilde{K}_{IFD/ICC} = (g^{r_{IFD}} \bmod pc)^{r_{ICC}} \bmod p = g^{r_{IFD} \cdot r_{ICC}} = K_{IFD/ICC} \tag{A-21}$$

As $0 \leq K_{IFD/ICC} < c \cdot p$ and $0 \leq K_{IFD/ICC} < p$, then $\hat{K}_{IFD/ICC} = K_{IFD/ICC}$ is true if this equality is *mod p* and *mod c*. As the equality *mod p* is trival and $\hat{K}_{IFD/ICC} \equiv K_{ICC}^{\ r_{IFD}} \bmod c \equiv 1$ the assumption is confirmed.

A.5 The “Grandmaster Chess Attack”

Device authentication without additional physically secured devices is not resistant agains the “Grandmaster Chess Attack” also called the “Mafia Attack”,

The attack scenario can be illustrated by the following figure

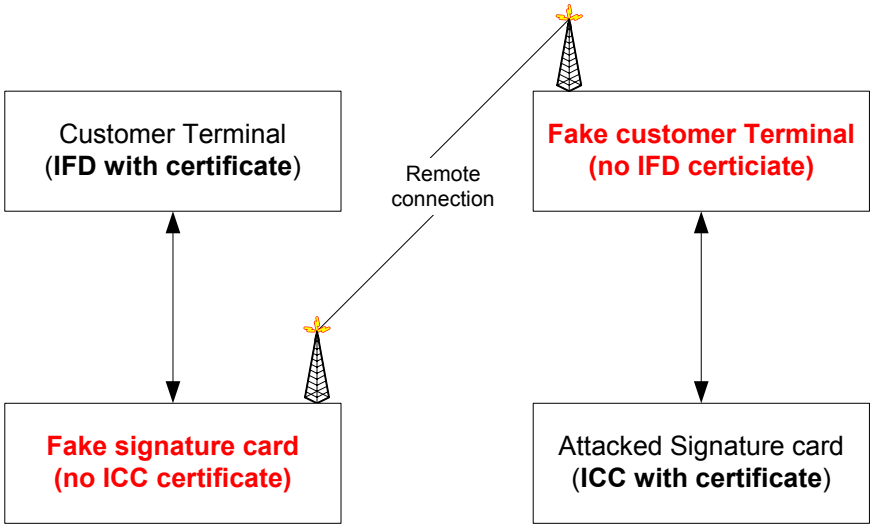


Figure A-6. Grandmaster Chess Attack

The attackers run a fake customer terminal without IFD certificate and use a fake signature card without ICC certificate. The terminals are locally seperated and connected through a remote connection. The fake IFD transmits all data transparently to the fake ICC, that is connected to the customer terminal.

The fake devices cannot tamper nor change the messages, indeed the signature card will establish a correct secure session with the customer terminal. However the display message can be taken from the customer terminal and indicated to the card holder through the fake customer terminal. Then the card holder will enter her password into the fake terminal.

The password is recorded by the fake terminal, the session is interrupted and the fake terminal starts a session with the ICC without device authentication, entering the stored PIN.

The exposure of this attack is, however, at the same level as ‘looking over the shoulder’. Due to the nature of knowledge based identification methods, this problem cannot be reduced easily. Biometric methods can improve the situation.

Annex B. Personalisation scenarios

The following figures demonstrate a typical personalisation scenario, where the ICC generates the asymmetric key pair.

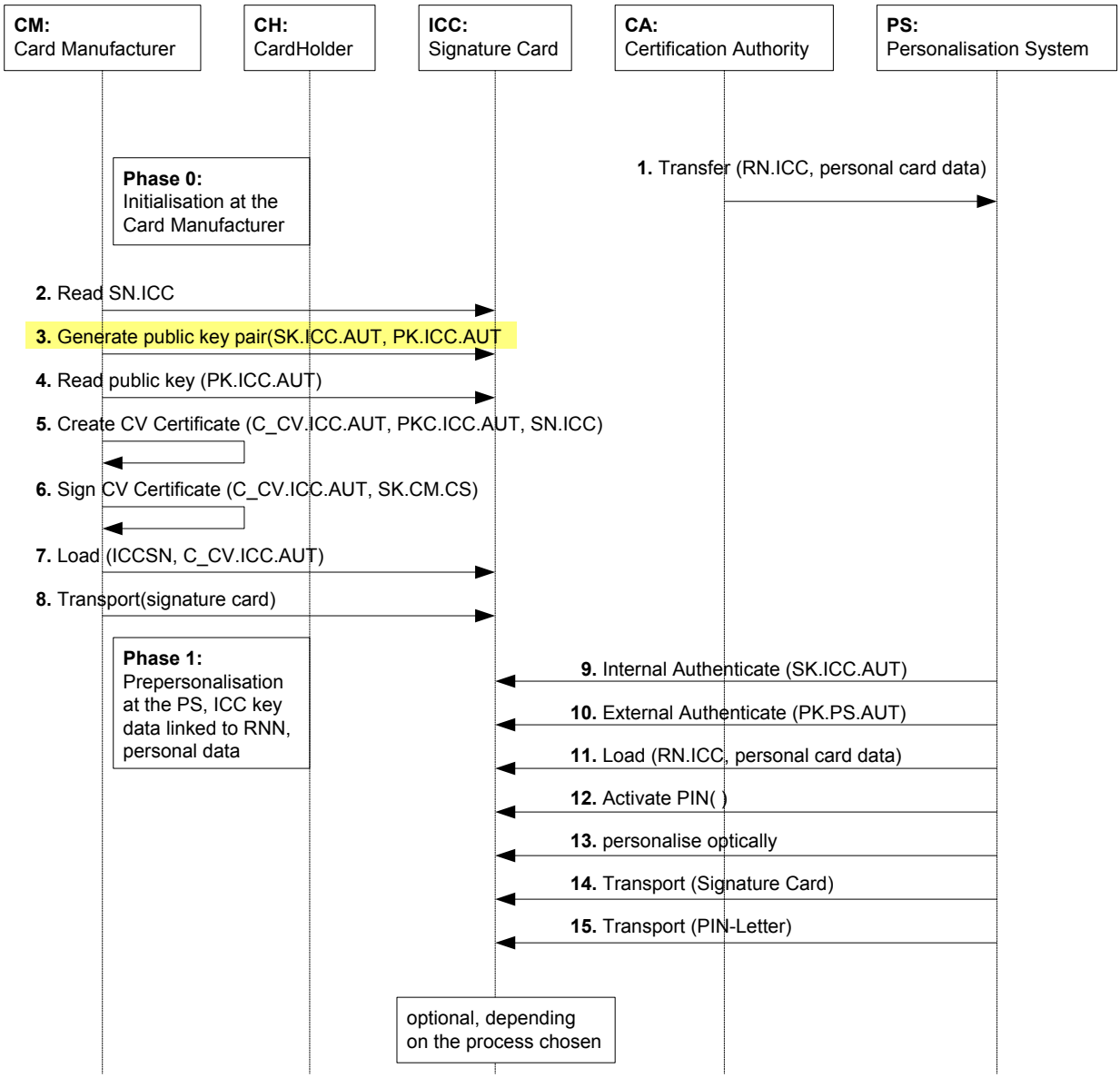


Figure B-1. Initialisation and pre-personalisation

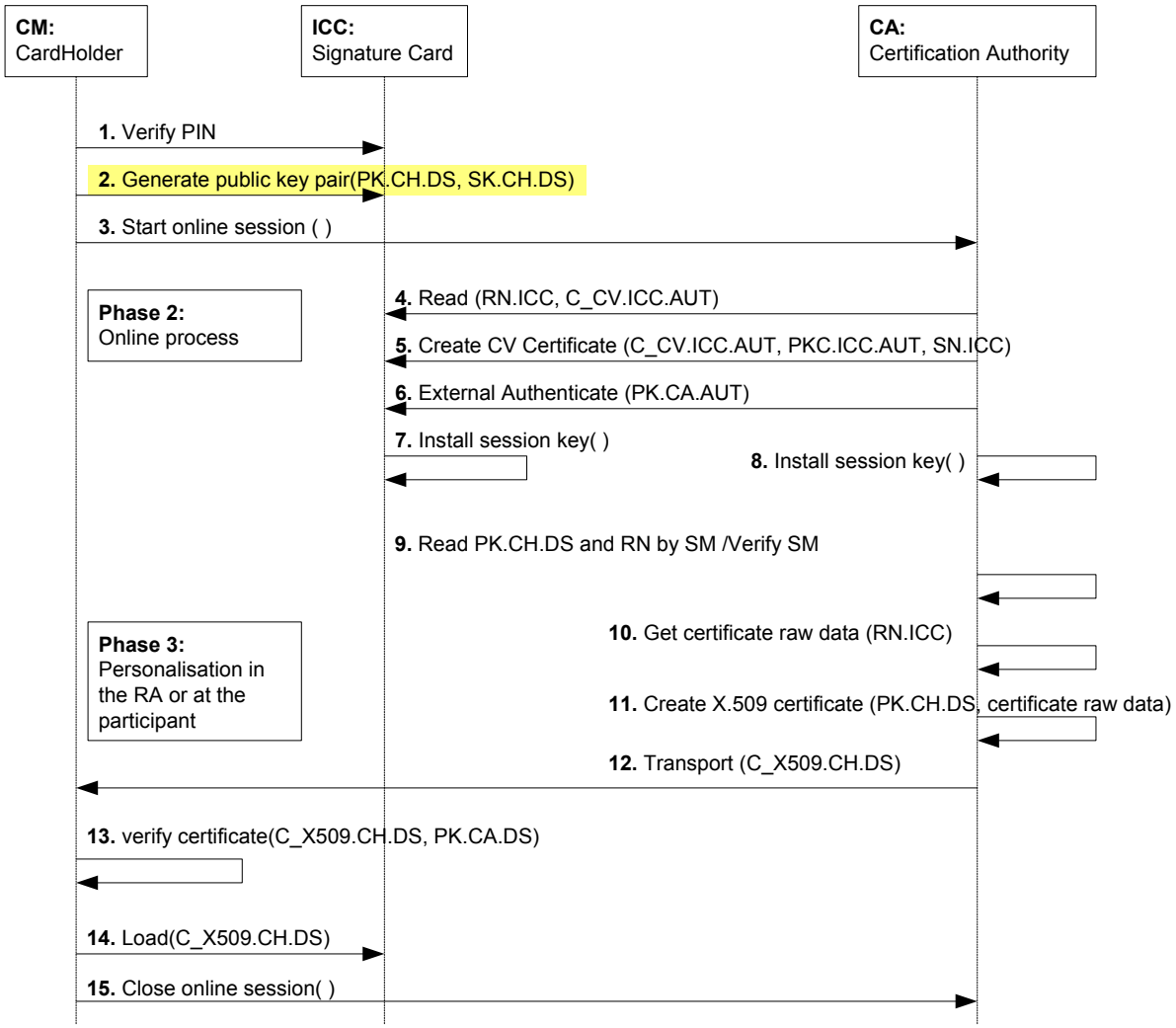


Figure B-2. On-line process and personalisation

Index

A

Access Conditions 67
Algorithm
  Selection 30
Answer-to-Reset 14
Application
  Concurrent usage 20
  Flow 17
Application Flow 17
Attack
  Grandmaster Chesss 144
Authentication
  Certificate
    Certificate
      Authentication 106
  Device 34, 131
  Environment 37
  environment 37
  external 55
  internal 57
  Post phase 63
  privacy protocol 48, 139
  symmetric 59

C

Certificate
  Card verifiable 103
  chain 34
  cross 36
  CV 103
  Signature 106
Certificates
  read 32
Certification Authoritiy 34
Certification authority 104
Channel
  Security 20
  Selection 20
command-response pair 14
Counter
  Send Sequence 71
Cryptographic CheckSum 70
Cryptographic Token Information 122

D

Data
  Structures 91
data elemen 14
data object 14
data unit 14
DES
  Triple 71
Device authentication 34
DF name 14
Diffie-Hellman 136
  key exchange parameters 90
Digital Signature
  Input Format, AlgIDs and OIDs 98
Display Message 63
  Updating 65
Display message 38

E

Elliptic Curve
  Key parameters 87
Elliptic curve 102
  public key parameters 87
Environment
  Authentication 37
  trusted, untrusted 16
EU directive 7, 16

F

File
  control parameters 14
  dedicated 14
  directory 14
  elementary 14, 15
  identifier 14
  internal 14
  management data 14
  Master 14
  parent 14
  working 15
Files 116
Format
  biometric verification data 26
  Certificate 112
  Digital Signature Input 101
  Message structure 68
Formats
  presentation 23
  Presentation formats 22

H

Hash
  Input Format 99

K

Key
  Agreement, Transport 38
  Identifiers and parameters 86

Management 103
private 15
Public 86
Selection 21
Verification function 139
Key Generation 74

M

message 14

O

Object Identifier 113

P

Padding
for checksum calculation 68
Password 22
Change 24
password 14
path 15
Post authentication 63
Privacy protection 48
Protocol
Key Transport
Authentication transport protocol 39
Key transport 131
privacy protection 139
provider 15

R

record 15
identifier 15
number 15
Retry counter 24
Role
Identifier 109

S

Scope 7
Secure Messaging 67
Security
Basic Service 21
Security Environment
Selection 20
Selection
Algorithm 30
cryptographic information application 19
ESIGN application 19
Input Formats 30
Key 21, 30
Security Environment 20
Send Sequence Counter
Counter
Send Sequence 63
Session Key
computation 62
Session key
transport scheme 34
Signature
Activation 27
Certificate 106
Creation 29
Signature application
concurrent use 20

T

terminology 16
Trusted channel 15
Trusted environment 16, 37
trusted path 15, 37

U

Untrusted environment 15
User Verification 22
User verification
Biometric 25
Knowledge based 22